
CSCI 659 Lecture 1

Fall 2022

Instructor: Haipeng Luo

1 Overview of Online Learning

Online learning (also known as online optimization, sequential decision making, etc.) has been playing a crucial role in machine learning and many real-life applications. Its foundation is supported by elegant and deep theory, which is the main topic of this course, but before even telling you what online learning is, let me start with a couple of impactful applications of online learning to convince you of its importance:

- ADAGRAD [Duchi et al., 2011], ADAM [Kingma and Ba, 2015], and several other most popular optimization algorithms for training machine learning models including neural nets, are derived from an online learning framework;
- regret minimization, the central goal of online learning, is arguably the most efficient and scalable way for solving games (e.g., finding Nash equilibria) — for example, it is the cornerstone for superhuman AI for Poker [Bowling et al., 2015], a game with 10^{13} states;
- multi-armed bandits, an online learning framework fundamental for studying exploration versus exploitation tradeoff, has been used in building recommendation systems in many tech companies [Agarwal et al., 2016];
- machine learning algorithms that preserve privacy are gaining increasing attention in recent years — it turns out that, theoretically speaking, private learning and online learning are equivalent [Alon et al., 2022].

Hopefully now you are convinced that online learning is important. Below let's start looking at several more concrete instances of online learning.

- spam detection (online classification/regression): At each time $t = 1, 2, \dots$
 - receive an email $x_t \in \mathbb{R}^d$;
 - predict whether it is a spam $\hat{y}_t \in \{-1, +1\}$;
 - see its true label $y_t \in \{-1, +1\}$ (spam or not).
- sequential investment (universal portfolio): Start with capital W_1 . At each day $t = 1, 2, \dots$
 - decide $p_t \in \Delta(N) \stackrel{\text{def}}{=} \{p \in \mathbb{R}_+^N : \sum_{i=1}^N p(i) = 1\}$ and invest $W_t p_t(i)$ on asset i ;
 - at the end of the day observe relative prices $r_t \in \mathbb{R}_+^N$ such that the capital on asset i becomes $W_t p_t(i) r_t(i)$;
 - arrive at total capital $W_{t+1} = W_t \langle p_t, r_t \rangle$.
- aggregating weather prediction (the expert problem): At each day $t = 1, 2, \dots$
 - obtain temperature predictions from N experts/models for today;
 - make the final prediction by randomly following an expert according to $p_t \in \Delta(N)$;
 - at the end of the day observe the loss of each model $\ell_t \in [0, 1]^N$.
- product recommendation (multi-armed bandits): At each time $t = 1, 2, \dots$

- randomly recommend one of the K products a to a customer visiting the website;
- observe the loss of this product $\ell_t(a)$ (e.g. 0 if clicked, 1 otherwise), but not the losses for the other products.
- multiple-product recommendation (combinatorial bandits): At each time $t = 1, 2, \dots$
 - randomly recommend k of the K products to a customer visiting the website;
 - observe the losses of the k recommended products but not the other ones.
- personalized product recommendation (contextual bandits): Given N policies $\pi^1, \dots, \pi^N$, each of which is a mapping from $\mathcal{X}$ to $[K] \stackrel{\text{def}}{=} \{1, \dots, K\}$. At each time $t = 1, 2, \dots$
 - observe the contextual information $x_t \in \mathcal{X}$ of a customer (e.g. gender, IP address, purchase history, etc);
 - randomly select one of the N policies π_t and recommend product $\pi_t(x_t)$;
 - observe the loss of this product $\ell_t(\pi_t(x_t))$ but not the other ones.

2 A Unified Model

All of the examples in the last section can be (essentially) captured by a learning model called *Online Convex Optimization (OCO)*, first proposed by [Zinkevich \[2003\]](#). OCO can be viewed as a game between a learner/player and an environment/adversary. Before the game starts, a fixed compact convex set Ω is given to the learner as the action space. The game then proceeds for T rounds for some integer T . At each round $t = 1, \dots, T$,

1. the learner first picks a point $w_t \in \Omega$;
2. the environment then picks a convex loss function $f_t : \Omega \rightarrow \mathbb{R}$;
3. the learner suffers loss $f_t(w_t)$, and observes some information about f_t .

The table below summarizes how OCO captures all the aforementioned examples.

Problems	Ω	f_t	Observations
linear classification linear regression	e.g. $\{w : \ w\ _2 \leq 1\}$	$f_t(w) = \ell(\langle w, x_t \rangle, y_t)$, e.g. logistic loss: $\ell(\hat{y}, y) = \ln(1 + e^{-\hat{y}y})$ or square loss: $\ell(\hat{y}, y) = (\hat{y} - y)^2$	x_t and y_t thus entire f_t
universal portfolio	$\Delta(N)$	$f_t(p) = -\ln(\langle p, r_t \rangle)$ (note the unboundedness)	r_t , thus entire f_t
the expert problem	$\Delta(N)$	$f_t(p) = \langle p, \ell_t \rangle$	ℓ_t , thus entire f_t
multi-armed bandits	$\Delta(K)$	$f_t(p) = \langle p, \ell_t \rangle$	only one entry of ℓ_t
combinatorial bandits	e.g. $\{w \in [0, 1]^K : \sum_{i=1}^K w(i) = k\}$	$f_t(w) = \langle w, \ell_t \rangle$	k entries of ℓ_t
contextual bandits	$\Delta(N)$	$f_t(w) = \sum_{j=1}^N w(j) \ell_t(\pi^j(x_t))$ (note the loss structure among policies)	only one entry of ℓ_t

A couple of remarks are in order.

Why convexity? This is mainly because a) many problems are indeed convex and b) convexity allows efficient and provable algorithms. Of course, neural nets, the currently dominating machine learning models in many areas, are non-convex. Extending the theory of online learning to the non-convex regime is an important research direction, but note that even in the traditional “offline” setting, little is known currently.

Also note that for some problems, the action set is naturally discrete (e.g. all the bandit examples), but it can still be easily captured by OCO via convexifying the discrete action set (i.e., taking the convex hull).

Feedback model. As illustrated by these examples, there are several possible feedback models in OCO, roughly categorized into the following three:

- full-information setting: learner observes f_t (or sometimes just (sub)gradient $\nabla f_t(w_t)$);
- bandit setting: learner observes only $f_t(w_t)$;
- other partial information settings (sometimes not even the loss of the selected action).

The first half of this course will focus on the full full-information setting, while the second half focuses on the bandit setting.

Types of Environment. So far, we have not discussed how the environment decides the loss functions. Depending on its power, there are several possible settings (all of which allow the environment to have knowledge about the learner’s algorithm, but not her randomness, ahead of time):

- stochastic setting: $f_1, \dots, f_T$ are i.i.d. samples of a fixed and unknown distribution;
- oblivious adversary: $f_1, \dots, f_T$ are arbitrary, but decided before the game starts (i.e. independent of the learner’s actions);
- non-oblivious/adaptive adversary: for each t , f_t depends on $w_1, \dots, w_t$.

In this course we will mostly focus on the more general adversarial settings.

2.1 Goal of the Learner: Regret Minimization

The classical goal of OCO is to minimize the learner’s (static) regret against the best fixed action in hindsight:

$$\mathcal{R}_T \stackrel{\text{def}}{=} \sum_{t=1}^T f_t(w_t) - \min_{w \in \Omega} \sum_{t=1}^T f_t(w).$$

Note that this could be a random variable where the randomness might come from the learner and/or the environment. Intuitively, this is measuring how much we “regret” for not always picking the best fixed action in retrospect. If the regret is sublinear in T , that is, $\mathcal{R}_T = o(T)$ (in which case the learner is often called “no-regret”), then $\lim_{T \rightarrow \infty} \mathcal{R}_T/T = 0$ and thus on average the learner is doing almost as well as the best fixed action in hindsight, which is a pretty strong guarantee. However, two very natural questions arise upon closer examination of this performance measure:

1. *Why is it fair to compete with a fixed action while the learner can vary her actions over time? And why is it reasonable anyway given that the loss functions are changing over time and probably no single action is reasonably good overall?*

For the first part of the question, note that competing with a fixed action is already highly nontrivial, since this fixed action is chosen with perfect knowledge of the entire loss functions, while the learner receives information (sometimes only partial information) about them on the fly. That said, also note that the regret can indeed be negative exactly because of this reason if the environment is benign.

The second part of the question is a valid concern, and there are many stronger regret measures proposed to exactly address this issue, including interval/switching/dynamic regret and internal/swap regret, some of which will be covered later in this course.

2. *Why do we assume that the environment still chooses the same sequence of loss functions in retrospect while the learner has changed her behavior to always picking a fixed action?*

First note that at least for a stochastic environment or an oblivious adversary, this assumption indeed holds (since the loss functions are chosen independent of the learner’s decisions). However, this is indeed a valid concern for adaptive adversary, and a stronger regret measure called policy regret was proposed to exactly address this issue.

However, despite all these issues, **studying static regret is still extremely meaningful** for two reasons. First, static regret is the foundation of all other stronger regret measures. Algorithms for these stronger measures are most often designed by extending techniques from the static regret literature or

sometimes even by reducing the problem to minimizing static regret. Second, later in this course we will see examples in solving games via online learning, where the environment is definitely changing in an adaptive way, yet minimizing static regret still has strong implications for convergence to equilibria and optimal social welfare. Therefore, designing (static) regret minimization algorithm and proving their regret bounds has always been the central theme of online learning.

3 Connection to Statistical Learning

Before diving further into online learning, let's take a step back and discuss the connections and differences between online learning and the classical machine learning paradigm: *statistical learning*.

In statistical learning, a set of training examples $z_1, \dots, z_T \in \mathcal{Z}$ is given to the learner where each example z_t is an i.i.d. sample of some unknown distribution $\mathcal{D}$. Based on these training examples, the learner outputs a predictor $\bar{w} \in \Omega$ for some compact convex set Ω . For some loss function $\ell : \Omega \times \mathcal{Z} \rightarrow [0, 1]$, the *training error* of the learner is defined as $\frac{1}{T} \sum_{t=1}^T \ell(\bar{w}, z_t)$ while the *generalization error* is defined as $L(\bar{w}) \stackrel{\text{def}}{=} \mathbb{E}_{z \sim \mathcal{D}} \ell(\bar{w}, z)$. Note that this quantity is itself a random variable since $\bar{w}$ is random due to the randomness of the training set. The goal of the learner is usually to have small generalization error with high probability.

As one can see, distributional assumptions are fundamental in the definition of statistical learning. On the other hand, online learning does not necessarily assume any distributional distributions, which makes it a much more robust model. In fact, even if the data are entirely adversarial, which is indeed the case for applications such as spam detection or playing games, meaningful and strong guarantees can still be derived.

Another key advantage is that online learning algorithms are usually more memory-efficient, in the sense that they usually do not need to store data from the past. That is, at each round, a new data point is used to update the current state of the algorithm and then discarded. On the other hand, most statistical learning algorithms require storing the entire training set and touching each data point multiple times.

Finally and perhaps most importantly, one can in fact use an online learning algorithm to solve a statistical learning problem, meaning that online learning is only more general. This can be done via the following online-to-batch reduction [Cesa-Bianchi et al., 2004], which essentially lets an online learning algorithm make exactly one pass to the training set.

Algorithm 1: Online-to-Batch Reduction

Input: training set $\{z_1, \dots, z_T\}$, an online learning algorithm $\mathcal{A}$ with action space Ω

for $t = 1, \dots, T$ **do**

let w_t be the output of $\mathcal{A}$ for this round
 feed $\mathcal{A}$ with loss function $f_t(w) = \ell(w, z_t)$

Output (Option I): uniformly at random sample $\bar{t}$ from $\{1, \dots, T\}$, then $\bar{w} = w_{\bar{t}}$

Output (Option II): $\bar{w} = \frac{1}{T} \sum_{t=1}^T w_t$ if $L(w)$ is convex in w

The reduction enjoys the following guarantee.

Theorem 1. Suppose that $\mathcal{A}$ enjoys the following regret bound $\mathbb{E}[\mathcal{R}_T] \leq B(T)$ for some function B . Then the generalization error of the output $\bar{w}$ of Algorithm 1 satisfies

$$\mathbb{E}[L(\bar{w})] \leq L(w^*) + \frac{B(T)}{T}$$

where $w^* \in \arg\min_{w \in \Omega} L(w)$ is the optimal predictor.

Proof of Theorem 1. First focus on Option I. By definitions, we have (pay close attention to what randomness each expectation $\mathbb{E}[\cdot]$ is with respect to)

$$\mathbb{E}[L(\bar{w})] = \frac{1}{T} \sum_{t=1}^T \mathbb{E}[L(w_t)] = \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\mathbb{E}_{z \sim \mathcal{D}} \ell(w_t, z)]$$

$$\begin{aligned}
&= \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\ell(w_t, z_t)] \leq \mathbb{E} \left[\min_{w \in \Omega} \frac{1}{T} \sum_{t=1}^T \ell(w, z_t) \right] + \frac{B(T)}{T} \\
&\leq \frac{1}{T} \sum_{t=1}^T \mathbb{E}[\ell(w^*, z_t)] + \frac{B(T)}{T} \\
&= \frac{1}{T} \sum_{t=1}^T L(w^*) + \frac{B(T)}{T} = L(w^*) + \frac{B(T)}{T}.
\end{aligned}$$

For Option II, simply notice that by the convexity of L and Jensen's inequality:

$$L(\bar{w}) = L\left(\frac{1}{T} \sum_{t=1}^T w_t\right) \leq \frac{1}{T} \sum_{t=1}^T L(w_t).$$

The rest of the proof is identical to Option I. $\square$

Therefore, as long as $\mathcal{A}$ is no-regret (i.e. $B(T) = o(T)$), then the generalization error of $\bar{w}$ is arbitrarily close to the optimal error when the sample size T is large enough. For example, for many problems we can show $B(T) = \mathcal{O}(\sqrt{T})$, which implies a convergence rate of $1/\sqrt{T}$ to the optimal generalization error and is often known to be optimal. We note that although we present an expectation guarantee here, the same holds up to an additional $\mathcal{O}(\sqrt{\ln(1/\delta)/T})$ term with probability at least $1 - \delta$ by directly applying certain concentration inequalities (see e.g. [Luo, 2017]).

This illustrates that online learning algorithms can be used for traditional statistical learning problems as well, which is also the reason why algorithms such as ADAGRAD and ADAM are all derived from the OCO setting, but mostly used in training models with a fixed training set in practice.

Question. *Why making only one pass of the training set is important to Algorithm 1? Can you identify the step in the proof that will break as long as one training example is passed to $\mathcal{A}$ more than once?*

4 The Expert Problem and the Hedge Algorithm

Now we start discussing how to solve online learning problems. We defer the general OCO case to the next lecture, and focus on one special case, the expert problem, in the rest of this lecture. It turns out that this problem plays some fundamental role in online learning (and other related topics), as we will see soon in this course. The expert problem is originated from [Freund and Schapire, 1997]. Recall that in this problem, at each round t the learner decides a distribution $p_t \in \Delta(N)$ over N experts, and then suffers loss $\langle p_t, \ell_t \rangle$ for some loss vector $\ell_t \in [0, 1]^N$ decided by the environment. How do we ensure sublinear regret in this problem?

The first naive algorithm that comes to one's mind is probably the *follow the leader* (FTL) approach, which puts all the weights to the current best expert $\arg\max_{i \in [N]} - \sum_{s=1}^{t-1} \ell_s(i)$. However, it is not difficult to see that such an approach in fact could suffer linear regret in the worst case: simply consider the case where $N = 2$, $\ell_1 = (0.5, 0)$, and for $t \geq 2$, ℓ_t alternates between $(0, 1)$ and $(1, 0)$.

It turns out that, however, simply replacing the “max” in this naive algorithm by some “softmax” would change the situation greatly. In fact, this leads to the classical algorithm called Hedge [Littlestone and Warmuth, 1994, Freund and Schapire, 1997]. The same algorithm is also known as multiplicative weights update, exponential weights, and many others, and was discovered independently in many different areas. We present the pseudocode below.

Algorithm 2: Hedge

Input: learning rate $\eta > 0$ (also called step size, temperature, etc.)

Initialization: let $L_0 \in \mathbb{R}^N$ be the all-zero vector

for $t = 1, \dots, T$ **do**

compute $p_t \in \Delta(N)$ such that $p_t(i) \propto \exp(-\eta L_{t-1}(i))$
play p_t and observe loss vector $\ell_t \in [0, 1]^N$
update $L_t = L_{t-1} + \ell_t$

We will discuss more intuition of this algorithm in the next lecture when putting it into a more general framework, but it is clear that this algorithm is at least doing something sensible: the better an expert looks based on the past cumulative loss, the more weight we put on this expert, but at the same time we do not act as extremely as the FTL approach and always make sure that every expert gets some non-zero weight. We also point out that this is definitely not the only algorithm for this problem, and there are many other reasonable ways to spread the weights and ensure low regret.

The regret for this algorithm can be written as:

$$\mathcal{R}_T = \sum_{t=1}^T \langle p_t, \ell_t \rangle - \min_{p \in \Delta(N)} \sum_{t=1}^T \langle p, \ell_t \rangle = \sum_{t=1}^T \langle p_t, \ell_t \rangle - \sum_{t=1}^T \ell_t(i^*)$$

where $i^* \in \operatorname{argmin}_i \sum_{t=1}^T \ell_t(i)$ is the best expert in hindsight. Below we will prove a bound for any loss sequence $\ell_1, \dots, \ell_T$.

Theorem 2. *Hedge with learning rate η guarantees for any loss sequence $\ell_1, \dots, \ell_T \in [0, 1]^N$:*

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) \ell_t^2(i) \quad (1)$$

$$\leq \frac{\ln N}{\eta} + T\eta, \quad (2)$$

which is of order $\mathcal{O}(\sqrt{T \ln N})$ if η is optimally set to $\sqrt{(\ln N)/T}$.

There are many different proofs that lead to bound (2), but we will present a “potential-based” proof that uses bound (1) as an intermediate step, which will turn out to be very useful later on.

Proof. Define potential $\Phi_t = \frac{1}{\eta} \ln \left(\sum_{i=1}^N \exp(-\eta L_t(i)) \right)$. First consider how much the potential can increase between two consecutive rounds:

$$\begin{aligned} \Phi_t - \Phi_{t-1} &= \frac{1}{\eta} \ln \left(\frac{\sum_{i=1}^N \exp(-\eta L_t(i))}{\sum_{i=1}^N \exp(-\eta L_{t-1}(i))} \right) = \frac{1}{\eta} \ln \left(\frac{\sum_{i=1}^N \exp(-\eta L_{t-1}(i)) \exp(-\eta \ell_t(i))}{\sum_{i=1}^N \exp(-\eta L_{t-1}(i))} \right) \\ &= \frac{1}{\eta} \ln \left(\sum_{i=1}^N p_{t-1}(i) \exp(-\eta \ell_t(i)) \right) \\ &\leq \frac{1}{\eta} \ln \left(\sum_{i=1}^N p_{t-1}(i) (1 - \eta \ell_t(i) + \eta^2 \ell_t^2(i)) \right) \quad (e^{-y} \leq 1 - y + y^2 \text{ for all } y \geq -1) \\ &= \frac{1}{\eta} \ln \left(1 - \eta \langle p_{t-1}, \ell_t \rangle + \eta^2 \sum_{i=1}^N p_{t-1}(i) \ell_t^2(i) \right) \\ &\leq -\langle p_{t-1}, \ell_t \rangle + \eta \sum_{i=1}^N p_{t-1}(i) \ell_t^2(i). \quad (\ln(1+y) \leq y) \end{aligned}$$

Summing over t , telescoping, and rearranging gives

$$\begin{aligned} \sum_{t=1}^T \langle p_t, \ell_t \rangle &\leq \Phi_0 - \Phi_T + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) \ell_t^2(i) \\ &\leq \frac{\ln N}{\eta} - \frac{1}{\eta} \ln (\exp(-\eta L_T(i^*))) + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) \ell_t^2(i) \\ &\leq \frac{\ln N}{\eta} + L_T(i^*) + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) \ell_t^2(i), \end{aligned}$$

which proves Eq. (1). Eq. (2) follows immediately by the boundedness of losses. $\square$

One might wonder how to come up with such a proof and the specific form of “potential” that seems to come from nowhere. We will not go deep into this question, but only point out that a) the potential does come from somewhere and in fact can be “derived” in a sense from a principled framework, and b) in the next lecture we will discuss a different and more general and intuitive proof.

Question. Does Theorem 2 hold for an oblivious adversary or an adaptive adversary? If the learner randomly selects an expert i_t according to p_t at each time t , and we measure the regret by $\sum_{t=1}^T \ell_t(i_t) - \sum_{t=1}^T \ell_t(i^*)$, can we still obtain a similar regret bound (in expectation or with high probability)? If so, does it hold for an oblivious adversary or an adaptive adversary?

5 Lower bound for the Expert Problem

One important thing to note here is that the regret of Hedge has only logarithmic dependence on N , which is extremely important for problems with a huge number of experts as we will discuss in future lectures. But even if this regret bound already looks very good, how do we know whether we can do even better or not? In general, how can we tell whether a regret upper bound is satisfactory or not? The notion of minimax regret can be used to answer this question exactly. Intuitively, minimax regret is the smallest possible worst-case regret one can achieve. For example, the minimax regret of the expert problem with an oblivious adversary can be defined as

$$\min_{\mathcal{A}} \max_{\ell_1, \dots, \ell_T} \mathbb{E}_{\mathcal{A}}[\mathcal{R}_T]$$

where $\mathcal{A}$ is any legitimate expert algorithm and $\mathbb{E}_{\mathcal{A}}$ highlights the randomness with respect to the algorithm itself. Note that $\mathcal{R}_T$ depends on both $\mathcal{A}$ and all the losses even if the dependence is not explicitly spelled out. The existence of the Hedge algorithm already shows that

$$\min_{\mathcal{A}} \max_{\ell_1, \dots, \ell_T} \mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] \leq 2\sqrt{T \ln N}.$$

If we can further show a $\Omega(\sqrt{T \ln N})$ lower bound, that is, no algorithm can do better than $\mathcal{O}(\sqrt{T \ln N})$ in the worst case, then we call this bound the minimax optimal regret. The following theorem essentially shows that this is indeed the case (and thus Hedge is minimax optimal).

Theorem 3. For any algorithm, we have

$$\sup_{T, N} \max_{\ell_1, \dots, \ell_T} \frac{\mathbb{E}_{\mathcal{A}}[\mathcal{R}_T]}{\sqrt{T \ln N}} \geq \frac{1}{\sqrt{2}}.$$

How do we prove such a lower bound? It seemingly requires us to construct a worst-case loss sequence for any given algorithm, which is quite difficult given that an algorithm is a very complex object. Instead, a common technique is to construct some randomized environment, and then argue that all algorithms have to suffer certain regret in expectation, which in turn proves that for every algorithm, there exists a bad sequence leading to such regret, without even explicitly constructing it.

Proof. Let $\mathcal{D}$ be the uniform distribution over $\{0, 1\}$. We have

$$\begin{aligned} \max_{\ell_1, \dots, \ell_T} \mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] &\geq \mathbb{E}_{\ell_1, \dots, \ell_T \stackrel{iid}{\sim} \mathcal{D}^N} \mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] \\ &= \sum_{t=1}^T \mathbb{E}_{\mathcal{A}, \ell_1, \dots, \ell_t} [\langle p_t, \ell_t \rangle] - \mathbb{E}_{\ell_1, \dots, \ell_T} \left[\min_{i \in [N]} \sum_{t=1}^T \ell_t(i) \right] \\ &= \sum_{t=1}^T \mathbb{E}_{\mathcal{A}, \ell_1, \dots, \ell_{t-1}} \langle p_t, \mathbb{E}_{\ell_t}[\ell_t] \rangle - \mathbb{E}_{\ell_1, \dots, \ell_T} \left[\min_{i \in [N]} \sum_{t=1}^T \ell_t(i) \right] \\ &= T/2 - \mathbb{E}_{\ell_1, \dots, \ell_T} \left[\min_{i \in [N]} \sum_{t=1}^T \ell_t(i) \right] \\ &= \mathbb{E}_{\ell_1, \dots, \ell_T} \left[\max_{i \in [N]} \sum_{t=1}^T \left(\frac{1}{2} - \ell_t(i) \right) \right] \end{aligned}$$

$$= \frac{1}{2} \mathbb{E}_{\sigma_1, \dots, \sigma_T} \left[\max_{i \in [N]} \sum_{t=1}^T \sigma_t(i) \right],$$

where $\sigma_t(i)$ for $i \in [N], t \in [T]$ are i.i.d. Rademacher random variables (i.e. -1 with probability 0.5 and 1 with probability 0.5). Note that at this point we have converted the problem to calculating a simple quantity that is independent of the algorithm and also has no sequential structure. Using the following standard result from probability theory (see for example [Cesa-Bianchi and Lugosi, 2006, Lemmas A.11 and A.12]) completes the proof.

$$\lim_{T \rightarrow \infty} \lim_{N \rightarrow \infty} \frac{\mathbb{E}_{\sigma_1, \dots, \sigma_T} \left[\max_{i \in [N]} \sum_{t=1}^T \sigma_t(i) \right]}{\sqrt{T \ln N}} = \sqrt{2}.$$

□

Note that this lower bound is asymptotic — it does not tell us for a specific N and T , what the optimal regret is. For a non-asymptotic lower bound, see for example [Orabona and Pál, 2015].

References

- A. Agarwal, S. Bird, M. Cozowicz, L. Hoang, J. Langford, S. Lee, J. Li, D. Melamed, G. Oshri, O. Ribas, S. Sen, and A. Slivkins. A multiworld testing decision service. *arXiv:1606.03966*, 2016.
- Noga Alon, Mark Bun, Roi Livni, Maryanthe Malliaris, and Shay Moran. Private and online learnability are equivalent. *ACM Journal of the ACM (JACM)*, 2022.
- Michael Bowling, Neil Burch, Michael Johanson, and Oskari Tammelin. Heads-up limit hold'em poker is solved. *Science*, 347(6218):145–149, 2015.
- Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- Nicolò Cesa-Bianchi, Alex Conconi, and Claudio Gentile. On the generalization ability of on-line learning algorithms. *IEEE Transactions on Information Theory*, 50(9):2050–2057, 2004.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research*, 12(7), 2011.
- Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, August 1997.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *Proceedings of the 3rd International Conference on Learning Representations*, 2015.
- Nick Littlestone and Manfred K. Warmuth. The weighted majority algorithm. *Information and Computation*, 108:212–261, 1994.
- Haipeng Luo. Lecture notes 1, introduction to online learning, 2017. URL <https://haipeng-luo.net/courses/CSCI699/lecture1.pdf>.
- Francesco Orabona and Dávid Pál. Optimal non-asymptotic lower bound on the minimax regret of learning with expert advice. *arXiv preprint arXiv:1511.02176*, 2015.
- Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning*, 2003.

CSCI 659 Lecture 2

Fall 2022

Instructor: Haipeng Luo

1 A General Algorithmic Framework

In the last lecture, we discussed a special case of OCO, that is, the expert problem, and the classical Hedge algorithm for this problem. For a general OCO problem, how should we design a no-regret algorithm? (Recall the OCO setting: each round the learner decides $w_t \in \Omega$, while the environment decides a convex loss function $f_t : \Omega \rightarrow \mathbb{R}$).

Before any further discussion, we first make the following observation: it is sufficient to solve the case where each f_t is a *linear* function. To see this, notice that by convexity, we have $f_t(w_t) - f_t(w) \leq \langle \nabla f_t(w_t), w_t - w \rangle$ (draw a picture to convince yourself) and thus

$$\max_{w \in \Omega} \sum_{t=1}^T (f_t(w_t) - f_t(w)) \leq \max_{w \in \Omega} \sum_{t=1}^T \langle \nabla f_t(w_t), w_t - w \rangle.$$

Therefore, we only need to solve a different OCO instance where the loss function $f'_t(w) = \langle \ell_t, w \rangle$ is linear with $\ell_t = \nabla f_t(w_t)$. There are a couple of things worth noticing in this argument:

- First, this shows that knowing the gradient of f_t at the played action w_t is already enough, and we do not really need full information about f_t . More importantly, many algorithms (such as those to be discussed in this lecture) only need to store the cumulative gradients, leading to T -independent space complexity.
- Second, in this reduction, the linear loss function generally becomes adaptively chosen (that is, dependent on w_t), even if the original adversary is oblivious (unless f_t itself is a linear function). Therefore, we do generally need an algorithm that is able to deal with an adaptive adversary.
- Third, while this reduction is sufficient to derive some regret bounds, it might not be the best way to do so, especially if f_t has some curvature making the linear approximation too loose.

In light of this reduction, for the rest of this lecture, we assume that the loss functions are linear and parameterized by $\ell_1, \dots, \ell_T$.

1.1 The Importance of Stability

Once again, the natural first attempt is the FTL approach: $w_t = \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s=1}^{t-1} \ell_s \right\rangle$, which we already know suffers linear regret in the worst case. If we take a closer look at the worst case instance discussed in Lecture 1, we see that FTL exhibits highly unstable behavior — it alternates between two very different decisions. Motivated by this, we explore the idea of stabilizing the algorithm. This might not sound very intuitive at first glance: the loss functions can be changing in some arbitrary way, so why is it a good thing to have a stable learner? One answer is that, recall that the goal of the learner is only to compete with a *fixed* action, so there is really no point in “chasing” the loss functions all the time. Instead, one should stick around and not move too far away from the current best fixed action.

To make this intuition more formal, in this lecture we consider stabilizing the FTL approach by adding an auxiliary loss function $\Psi : \Omega \rightarrow \mathbb{R}$ that is differentiable and convex:

$$w_t \in \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s=1}^{t-1} \ell_s \right\rangle + \Psi(w), \quad (1)$$

and we will discuss two different types of Ψ , leading to two different classical approaches. Before that, we first derive an intermediate regret bound for this general form, which will further illustrate the importance of stability and the role of Ψ .

First, recall that the Bregman divergence $D_\Psi : \Omega \times \Omega \rightarrow \mathbb{R}_+$ with respect to Ψ is defined as

$$D_\Psi(w, u) = \Psi(w) - \Psi(u) - \langle \nabla \Psi(u), w - u \rangle,$$

which is simply the gap between Ψ and its first order approximation at u . Note that this is always nonnegative due to the convexity of Ψ and zero when $w = u$, but in general asymmetric between w and u (thus not a metric). Taylor theorem also tells us that there exists ξ between w and u such that $D_\Psi(w, u) = \frac{1}{2} \|w - u\|_{\nabla^2 \Psi(\xi)}^2$,¹ meaning that the Bregman divergence is roughly measuring some squared quadratic norm of $w - u$.

Question 1. What is the Bregman divergence with respect to a linear function? What about a quadratic function?

We will be using the following simple fact.

Lemma 1. Suppose that $F : \Omega \rightarrow \mathbb{R}$ is convex and differentiable, and $w^* \in \Omega$ minimizes F . Then $F(w^*) \leq F(w) - D_F(w, w^*)$ holds for any $w \in \Omega$.

Proof. By definition, this is equivalent to $\langle \nabla F(w^*), w - w^* \rangle \geq 0$, which is exactly the first-order optimality condition for w^* being a minimizer of F . $\square$

What this lemma says is that w^* being a minimizer of F actually tells us a bit more than the trivial fact $F(w^*) \leq F(w)$. With this, we prove the following useful lemma.

Lemma 2. Let $w^* \in \Omega$ be a minimizer of the function $\langle w, L \rangle + \Psi(w)$ for some arbitrary L and Φ be its minimum value; similarly, let $\bar{w}^*$ and $\bar{\Phi}$ be a minimizer and the minimum value respectively of the function $\langle w, \bar{L} \rangle + \Psi(w)$ for some arbitrary $\bar{L}$. Then we have

$$\bar{\Phi} - \Phi \leq \langle w^*, \bar{L} - L \rangle - D_\Psi(w^*, \bar{w}^*).$$

Proof. Simply apply Lemma 1 with $F(w) = \langle w, \bar{L} \rangle + \Psi(w)$ and note that $D_F = D_\Psi$:

$$\begin{aligned} \bar{\Phi} - \Phi &= F(\bar{w}^*) - \Phi && \text{(by definition)} \\ &\leq F(w^*) - \Phi - D_\Psi(w^*, \bar{w}^*) && \text{(Lemma 1)} \\ &= \langle w^*, \bar{L} - L \rangle - D_\Psi(w^*, \bar{w}^*). && \text{(by definition)} \end{aligned}$$

$\square$

Now we are ready to show the following intermediate regret bound.

Lemma 3. Algorithm described by Eq. (1) ensures for any $u \in \Omega$:

$$\begin{aligned} \sum_{t=1}^T \langle w_t - u, \ell_t \rangle &\leq \underbrace{\Psi(u) - \min_{w \in \Omega} \Psi(w)}_{\text{penalty term}} + \underbrace{\sum_{t=1}^T \langle w_t - w_{t+1}, \ell_t \rangle - \sum_{t=1}^T D_\Psi(w_{t+1}, w_t)}_{\text{stability term}} \\ &\leq B_\Psi + \sum_{t=1}^T \langle w_t - w_{t+1}, \ell_t \rangle - \sum_{t=1}^T D_\Psi(w_{t+1}, w_t), \end{aligned}$$

where $B_\Psi = \max_{w \in \Omega} \Psi(w) - \min_{w \in \Omega} \Psi(w)$ is the range of Ψ .

¹For a PSD matrix M , the notation $\|w\|_M$ denotes $\sqrt{w^\top M w}$, which is equivalent to $\|M^{\frac{1}{2}} w\|_2$.

In this bound, the first term is the penalty or error term, caused by the fact that Eq. (1) is not directly minimizing the cumulative loss, but instead that plus the auxiliary term Ψ . The second term, called stability term, exactly demonstrates why having a stable algorithm is important: the closer w_t and w_{t+1} are (in terms of their loss for the loss function at time t precisely), the smaller the regret. Finally, for this lecture, we can simply ignore the third nonpositive term, but in the future we will see that this is critical in some cases. In fact, after ignoring this term and rearranging, Lemma 3 is simply saying (try to verify yourself)

$$\Psi(w_1) + \sum_{t=1}^T \langle w_{t+1}, \ell_t \rangle \leq \Psi(u) + \sum_{t=1}^T \langle u, \ell_t \rangle$$

for any $u \in \Omega$, that is, the one-step lookahead strategy w_{t+1} , has negative regret (when the auxiliary loss is considered as well). This is often known as the Be-the-Leader lemma (where the leader refers to the one-step lookahead strategy).

Proof of Lemma 3. Let $\Phi_t = \min_{w \in \Omega} \langle w, \sum_{s \leq t} \ell_s \rangle + \Psi(w) = \langle w_{t+1}, \sum_{s \leq t} \ell_s \rangle + \Psi(w_{t+1})$. Then Lemma 2 tells us

$$\Phi_{t-1} - \Phi_t \leq -\langle w_{t+1}, \ell_t \rangle - D_\Psi(w_{t+1}, w_t).$$

Adding $\langle w_t, \ell_t \rangle$ to both sides, summing over t , telescoping, and rearranging leads to

$$\sum_{t=1}^T \langle w_t, \ell_t \rangle - \Phi_T \leq -\Phi_0 + \sum_{t=1}^T \langle w_t - w_{t+1}, \ell_t \rangle - \sum_{t=1}^T D_\Psi(w_{t+1}, w_t).$$

It thus remains to point out that $\Phi_0 = \Psi(w_1) = \min_{w \in \Omega} \Psi(w)$ and $\Phi_T \leq \langle u, \sum_{s \leq T} \ell_s \rangle + \Psi(u)$ for any u by definition. $\square$

Therefore, our goal will be to find Ψ that stabilizes the algorithm while having a relatively small penalty. In the rest of this lecture, we discuss two general approaches to doing so.

2 Follow the Regularized Leader

The first approach is by taking Ψ to be some *strongly convex* function, often called a *regularizer*, to stabilize the algorithm. This approach is called Follow-the-Regularized-Leader, or FTRL for short. Note that regularization is also a widely-used technique in statistical learning (in fact, being able to stabilize the learning algorithm is one explanation of why it helps reduce generalization error).

Formally, let $\Psi = \frac{1}{\eta} \psi$ for some learning rate $\eta > 0$ and a strongly convex function $\psi : \Omega \rightarrow \mathbb{R}$. Strong convexity means: for any $w, u \in \Omega$, the following holds:²

$$\psi(u) - \psi(w) \leq \langle \nabla \psi(u), u - w \rangle - \frac{1}{2} \|u - w\|^2 \quad (2)$$

for some norm $\|\cdot\|$. The following observations might help you understand this concept better:

- Compared to the convexity property $\psi(u) - \psi(w) \leq \langle \nabla \psi(u), u - w \rangle$, there is an extra nonpositive term in the right-hand side of Eq. (2) (thus indeed “stronger”).
- Rearranging Eq. (2) gives $\frac{1}{2} \|w - u\|^2 + \langle \nabla \psi(u), w - u \rangle + \psi(u) \leq \psi(w)$, which, for quadratic norm $\|\cdot\|$, says that at any point w , the function $\psi(w)$ is completely above a certain quadratic (the left-hand side) with the same function value and gradient at u , thus illustrating a certain curvature of ψ .
- By definition, Eq. (2) is the same as $\frac{1}{2} \|w - u\|^2 \leq D_\psi(w, u) = \frac{1}{2} \|w - u\|_{\nabla^2 \psi(\xi)}^2$ for some ξ between u and w , so strong convexity provides a nice lower bound for the Bregman divergence.

²More precisely, this is the definition of ψ being 1-strongly convex. Due to the scaling parameter of $1/\eta$, it does not matter whether ψ is 1-strongly convex or α -strongly convex for some other parameter α .

Due to the curvature, the minimizer of a strongly convex function is unique (try to verify this yourself), and more importantly, does not move significantly when the function is slightly changed, that is, the minimizer is stable. Formally, we prove the following.

Lemma 4. Let $w^* = \operatorname{argmin}_{w \in \Omega} \langle w, L \rangle + \frac{1}{\eta} \psi(w)$ and $\bar{w}^* = \operatorname{argmin}_{w \in \Omega} \langle w, \bar{L} \rangle + \frac{1}{\eta} \psi(w)$ for some arbitrary L and $\bar{L}$, $\eta > 0$, and strongly convex ψ (with respect to some norm $\|\cdot\|$). We have

$$\|w^* - \bar{w}^*\|^2 \leq \eta \langle w^* - \bar{w}^*, \bar{L} - L \rangle, \quad (3)$$

and consequently

$$\|w^* - \bar{w}^*\| \leq \eta \|L - \bar{L}\|_*, \quad (4)$$

where $\|\cdot\|_*$ is the dual norm.³

Proof. Similar to Lemma 2, define $\Phi = \langle w^*, L \rangle + \frac{1}{\eta} \psi(w^*)$ and $\bar{\Phi} = \langle \bar{w}^*, \bar{L} \rangle + \frac{1}{\eta} \psi(\bar{w}^*)$. Then we have

$$\bar{\Phi} - \Phi \leq \langle w^*, \bar{L} - L \rangle - \frac{1}{\eta} D_\psi(w^*, \bar{w}^*) \quad (\text{Lemma 2})$$

$$\leq \langle w^*, \bar{L} - L \rangle - \frac{1}{2\eta} \|w^* - \bar{w}^*\|^2. \quad (\text{definition of strong convexity})$$

On the other hand, noticing the symmetry between $\bar{\Phi}$ and Φ , we in fact also have

$$\Phi - \bar{\Phi} \leq \langle \bar{w}^*, L - \bar{L} \rangle - \frac{1}{\eta} D_\psi(\bar{w}^*, w^*) \quad (\text{Lemma 2})$$

$$\leq \langle \bar{w}^*, L - \bar{L} \rangle - \frac{1}{2\eta} \|w^* - \bar{w}^*\|^2. \quad (\text{definition of strong convexity})$$

Summing up the two inequalities and rearranging proves Eq. (3). To prove Eq. (4), it suffices to further apply Hölder's inequality:

$$\langle w^* - \bar{w}^*, L - \bar{L} \rangle \leq \|w^* - \bar{w}^*\| \|L - \bar{L}\|_*$$

and divide both sides by $\|w^* - \bar{w}^*\|$. □

Note that the stability level is naturally controlled by the parameter η . Combining this stability lemma and the intermediate regret bound Lemma 3, we obtain the following regret bound for FTRL, where one see that the tradeoff between the penalty term and stability term is exactly governed by η .

Theorem 1. The FTRL strategy: $w_t = \operatorname{argmin}_{w \in \Omega} \langle w, \sum_{s < t} \ell_s \rangle + \frac{1}{\eta} \psi(w)$ ensures for any loss sequence $\ell_1, \dots, \ell_T$:

$$\mathcal{R}_T = \max_{w \in \Omega} \sum_{t=1}^T \langle w_t - w, \ell_t \rangle \leq \frac{B_\psi}{\eta} + \eta \sum_{t=1}^T \|\ell_t\|_*^2.$$

Therefore, if we further have $\max_t \|\ell_t\|_* \leq G$ for some $G > 0$, then setting $\eta = \sqrt{\frac{B_\psi}{TG^2}}$ leads to regret bound $\mathcal{R}_T = \mathcal{O}(G\sqrt{TB_\psi})$.

Proof. Note that by the definition of FTRL, Lemma 4 exactly tells us $\|w_t - w_{t+1}\| \leq \eta \|\ell_t\|_*$. Therefore, the regret can be bounded as

$$\mathcal{R}_T \leq \frac{B_\psi}{\eta} + \sum_{t=1}^T \langle w_t - w_{t+1}, \ell_t \rangle \quad (\text{Lemma 3})$$

$$\leq \frac{B_\psi}{\eta} + \sum_{t=1}^T \|w_t - w_{t+1}\| \|\ell_t\|_* \quad (\text{Hölder's inequality})$$

$$\leq \frac{B_\psi}{\eta} + \eta \sum_{t=1}^T \|\ell_t\|_*^2, \quad (\text{Lemma 4})$$

which finishes the proof. □

We conclude by pointing out that having a uniform bound G on $\|\ell_t\|_*$, which we recall is just $\|\nabla f_t(w_t)\|_*$, is simply saying that the loss function f_t is G -Lipschitz with respect to the norm $\|\cdot\|_*$.

³Given a norm $\|\cdot\|$ (called primal norm), its dual norm is defined as $\|u\|_* = \max_{\|w\| \leq 1} \langle u, w \rangle$. The most important examples of primal-dual norm pair for this course are $(\|\cdot\|_2, \|\cdot\|_2)$ and $(\|\cdot\|_1, \|\cdot\|_\infty)$.

3 Instances of FTRL

We now discuss concrete instantiations of FTRL for different problems.

3.1 Online Gradient Descent

For an arbitrary action space Ω , we can pick $\psi(w) = \frac{1}{2} \|w\|_2^2$, a very standard regularizer in machine learning. The induced FTRL is thus

$$w_t = \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s < t} \ell_s \right\rangle + \frac{1}{2\eta} \|w\|_2^2 = \operatorname{argmin}_{w \in \Omega} \left\| w + \eta \sum_{s < t} \ell_s \right\|_2^2,$$

which means w_t is the L_2 projection of $u_t = -\eta \sum_{s < t} \ell_s$ onto Ω . This algorithm is called (lazy) *Online Gradient Descent* (OGD) [Zinkevich, 2003], which can be equivalently written as (after plugging in the original meaning of ℓ_t , that is, $\nabla f_t(w_t)$)

$$u_{t+1} = \textcolor{blue}{u}_t - \eta \nabla f_t(w_t); \quad w_{t+1} = \operatorname{argmin}_{w \in \Omega} \|w - u_{t+1}\|_2.$$

A closely related variant (which enjoys similar regret bounds and is also referred to as OGD) is the following strategy that actively projects:

$$u_{t+1} = \textcolor{red}{w}_t - \eta \nabla f_t(w_t); \quad w_{t+1} = \operatorname{argmin}_{w \in \Omega} \|w - u_{t+1}\|_2.$$

If f_t stays the same over time, then the strategy above is apparently the standard gradient descent for optimizing this function. OGD shows that even if the function is changing over time, making a gradient descent step at each round is still a good strategy.

To apply the general regret bound from Theorem 1, we point out that $\psi(w) = \frac{1}{2} \|w\|_2^2$ is strongly convex with respect to the L_2 norm (verify it yourself). The dual norm of the L_2 norm is itself, so if G is such that $\max_t \|\nabla f_t(w_t)\|_2 \leq G$, then the regret of OGD is bounded by

$$\mathcal{R}_T \leq \frac{\max_{w \in \Omega} \|w\|_2^2}{2\eta} + \eta T G^2 = \mathcal{O} \left(\max_{w \in \Omega} \|w\|_2 G \sqrt{T} \right),$$

where the last step is by picking the optimal η . It can be shown that this bound is minimax optimal.

Examples Consider the online regression problem where $\Omega = \{w \in \mathbb{R}^d : \|w\|_2 \leq 1\}$ is a set of linear predictors with bounded norm, and $f_t(w) = \frac{1}{2} (\langle w, x_t \rangle - y_t)^2$ is the square loss for an example $x_t \in \{x \in \mathbb{R}^d : \|x\|_2 \leq 1\}$ and its label $y_t \in [-1, 1]$. Then because $\nabla f_t(w) = (\langle w, x_t \rangle - y_t)x_t$, we have $G = 2$ and $\max_{w \in \Omega} \|w\|_2 = 1$, and therefore OGD has regret $\mathcal{O}(\sqrt{T})$, *independent* of the dimension of the problem d .

Next consider using OGD for the expert problem (where $\Omega = \Delta(N)$ and $\ell_t \in [0, 1]^N$). In this case we have $\max_{p \in \Delta(N)} \|p\|_2 \leq \max_{p \in \Delta(N)} \|p\|_1 = 1$, and $\|\ell_t\|_2 \leq \sqrt{N}$. Thus OGD's regret is $\mathcal{O}(\sqrt{TN})$ in this case, which has *exponentially worse* dependence on N compared to Hedge.

3.2 Recovering Hedge for the Expert Problem

OGD fails to achieve the optimal regret bound for the expert problem because the squared L_2 norm regularizer does not fully exploit the particular structure of the problem. Instead, for the simplex, a classical regularizer is the (negative) Shannon entropy function (we switch the notation from w to p for convention): $\psi(p) = \sum_{i=1}^N p(i) \ln p(i)$. Then one can verify that the induced FTRL strategy

$$p_t = \operatorname{argmin}_{p \in \Delta(N)} \left\langle p, \sum_{s < t} \ell_s \right\rangle + \frac{1}{\eta} \sum_{i=1}^N p(i) \ln p(i)$$

is exactly the Hedge algorithm, that is, $p_t(i) \propto \exp(-\eta \sum_{s < t} \ell_s(i))$ (verify this yourself). In other words, Hedge is just one special case of FTRL. This also gives us more intuition about Hedge: the negative entropy is minimized when p is the uniform distribution, so Hedge is trying to minimize the

cumulative loss while not being too far away from the uniform distribution (in case some currently bad experts become good in the future).

To apply the FTRL regret bound, we use the fact that the entropy function is strongly convex with respect to the L_1 norm. To see this, note that it suffices to prove $\|p - q\|_1^2 \leq \|p - q\|_{\nabla^2 \psi(\xi)}^2$ for any $\xi \in \Delta(N)$ between two distributions p and q , which is indeed true via a direct application of the Cauchy-Schwarz inequality:

$$\begin{aligned} \|p - q\|_1^2 &= \left(\sum_{i=1}^N |p(i) - q(i)| \right)^2 = \left(\sum_{i=1}^N \sqrt{\xi(i)} \cdot \frac{|p(i) - q(i)|}{\sqrt{\xi(i)}} \right)^2 \\ &\leq \left(\sum_{i=1}^N \xi(i) \right) \left(\sum_{i=1}^N \frac{(p(i) - q(i))^2}{\xi(i)} \right) = \|p - q\|_{\nabla^2 \psi(\xi)}^2. \end{aligned} \quad (5)$$

Therefore, to apply the general regret bound in Theorem 1, it suffices to do the following two simple calculations: first, the range of the entropy function is $B_\psi = \ln N$; second, the dual norm of the L_1 norm is the L_∞ norm, and by the definition of the problem (that ℓ_t is a vector in $[0, 1]^N$), we have $\|\ell_t\|_\infty \leq 1$ (note that this step is the key improvement compared to using OGD in this setting). Combining these facts, Theorem 1 implies the following regret bound for Hedge: $\mathcal{R}_T \leq \frac{\ln N}{\eta} + T\eta$, the same bound we proved last time using a different potential-based argument.

Question 2. In this case, can you recognize the connection between the Φ_t defined in the proof of Lemma 3, and the potential function (using the same notation Φ_t) we used in Lecture 1?

3.3 Combinatorial Problems

Next, we consider a generalization of the expert problem that has a certain combinatorial structure. Let $S = \{v_1, \dots, v_M\}$ be a set of *combinatorial actions* such that $v_j \in \{0, 1\}^N$ and $\max_j \|v_j\|_1 \leq m$ for some integer $m \leq N$. Roughly speaking, the learner needs to select one of these combinatorial actions at each time. For randomized strategy, this means that the decision space for the learner is the convex hull of S , that is, $\Omega = \left\{ \sum_{j=1}^M p(j) v_j : p \in \Delta(M) \right\} \subseteq [0, 1]^N$, so that each point in Ω specifies a distribution over these combinatorial actions. We consider linear loss functions $f_t(w) = \langle w, \ell_t \rangle$ for some $\ell_t \in [0, 1]^N$. Note that if $w = \sum_{j=1}^M p(j) v_j$, then $\langle w, \ell_t \rangle = \sum_{j=1}^M p(j) \langle v_j, \ell_t \rangle$ is clearly the expected loss if one selects a random combinatorial action according to p .

The expert problem is clearly a special case where S consists of all the standard basis vectors in $\mathbb{R}^N$ and $m = 1$. Another example is the so-called m -set problem, where each combinatorial action corresponds to a set of m experts (that is, instead of picking one expert each time, we now need to pick m of them). Formally, in this case we have $S = \{v \in \{0, 1\}^N : \|v\|_1 = m\}$ and $\Omega = \{w \in [0, 1]^N : \|w\|_1 = m\}$ (recall the multiple-product recommendation example in Lecture 1).

Yet another important example is the online shortest path problem (useful for online routing for example). In this problem, a directed acyclic graph with N edges, a source vertex, and a destination vertex is given. Each round the player first randomly picks a path to send a package, then the loss (e.g. delay) for each edge is revealed, and the player suffers the total loss of all the edges on the selected path. This can be formulated as a special case of the above combinatorial problem by setting S to be the set of all paths starting from the source and ending at the destination (that is, a path is represented by a vector in $\{0, 1\}^N$ so that each coordinate indicates whether the corresponding edge is on the path or not). Ω is now the set of all unit flows for this graph. Also note that m is the length of the longest path in S .

How do we solve such problems? The first natural approach is to reduce it to another standard expert problem. Earlier we point out that the latter is a special case of the former, but in fact the reverse is also true: we can simply treat each of the M combinatorial actions v_j as an expert with loss $\langle v_j, \ell_t \rangle$ at time t . Noticing that now the loss of each expert can be at most m , we see that applying Hedge to this problem leads to a regret bound of $\mathcal{O}(m\sqrt{T \ln M}) = \mathcal{O}\left(m\sqrt{T \ln \binom{N}{m}}\right) = \mathcal{O}\left(m\sqrt{mT \ln \frac{N}{m}}\right)$.

From this discussion we see again the importance of having only logarithmic dependence on the total number of experts: M is potentially exponential in m , so we can only afford $\ln M$ in the

regret. However, this does not address the computational issue since running Hedge (naively) for this problem does still require $\mathcal{O}(M)$ time complexity.

To address this, we consider another approach: run FTRL directly over Ω with a suitable regularizer. The regularizer we use is the generalized entropy $\psi(w) = m \sum_{i=1}^N w(i) \ln w(i)$, which looks identical to the standard entropy except for being scaled up by m and extended from the simplex to the space of Ω . The induced FTRL strategy:

$$w_t = \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s < t} \ell_s \right\rangle + \frac{m}{\eta} \sum_{i=1}^N w(i) \ln w(i)$$

is an N -dimensional convex problem with a polytope constraint set Ω . Therefore, as long as Ω is not too complicated, that is, described by $\operatorname{poly}(N, m)$ number of linear constraints (which is indeed the case for m -set and online shortest path), then one can apply any standard convex optimization method (such as the Interior Point Method) to find w_t efficiently.

After addressing the computational issue, we now consider the regret of this method. Via an argument similar to Eq. (5), one can show that the generalized entropy is also strongly convex with respect to the L_1 norm (verify it yourself). In addition, the range of ψ is now $B_\psi \leq m^2 \ln \frac{N}{m}$, and we still have $\|\ell_t\|_\infty \leq 1$. Applying Theorem 1 thus shows a regret bound of $\frac{m^2 \ln \frac{N}{m}}{\eta} + \eta T$, which is $\mathcal{O}(m \sqrt{T \ln \frac{N}{m}})$ with the optimal η and is $\sqrt{m}$ times better than what Hedge achieves. To conclude, by directly applying FTRL with a suitable regularizer over Ω , we not only resolve the computational issue, but also achieve an even better regret bound (optimal in fact).

4 Follow the Perturbed Leader

Even though the FTRL strategy for the combinatorial problems amounts to solving a convex problem that often admits polynomial time complexity, the actual running time might still be quite large. Is there a more efficient approach? In particular, is it possible to solve this problem based on only an offline linear optimization oracle that solves problems of the form $\operatorname{argmin}_{w \in \Omega} \langle w, L \rangle$? Notice that for m -set, this simply corresponds to finding the m smallest coordinates, while for online shortest path, this corresponds to finding the shortest path of a given graph, which are all “simple” problems.

Motivated by these questions, researchers developed another type of no-regret algorithms called Follow-the-Perturbed-Leader (FTPL) [Kalai and Vempala, 2005]. The idea is to introduce stability via *random perturbation/noise*. Specifically, we let $\Psi(w)$ be a (random) linear function $\langle w, \ell_0 \rangle$ where ℓ_0 is drawn from some distribution, and the strategy at time t is simply

$$w_t \in \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s=0}^{t-1} \ell_s \right\rangle,$$

which can clearly be solved by one call to the aforementioned linear optimization oracle, making the algorithm highly efficient.

Why is FTPL stable? The key is really in the randomness of ℓ_0 , since we already know that linear functions do not admit stable minimizers. However, if the distribution of ℓ_0 is dispersed enough, then in expectation the minimizer should not change significantly between two rounds. To illustrate this idea, we focus on the combinatorial setup and one particular distribution of the noise, and prove the following stability lemma (even though the same idea is applicable more generally). For simplicity, we assume w_t is always selected as one of the combinatorial actions $v \in S$ (since the minimum value of a linear function over a polytope can always be achieved by one of its vertices), and the tie is broken in some deterministic way. Also, we restrict our attention to an oblivious adversary who decides the loss sequence ahead of time, so that the only randomness in the following discussion comes from ℓ_0 . (To deal with adaptive adversary, it turns out that it suffices to resample ℓ_0 at every time to avoid leaking the randomness to the adversary; see [Hutter and Poland, 2005, Lemma 12].)

Lemma 5. *For the combinatorial problem described in Section 3.3, consider running the FTPL strategy $w_t \in \operatorname{argmin}_{w \in \Omega} \langle w, \sum_{s=0}^{t-1} \ell_s \rangle$ where each coordinate of ℓ_0 is an i.i.d. sample of the Laplace distribution with density function $h(x) = \frac{\eta}{2} e^{-\eta|x|}$ for some parameter $\eta > 0$. Then we have $\mathbb{E}[\langle w_t - w_{t+1}, \ell_t \rangle] \leq \eta N m$.*

Proof. Slightly abusing the notation, let $h(\ell_0) = \prod_{i=1}^N h(\ell_0(i)) = \frac{\eta}{2} e^{-\eta \|\ell_0\|_1}$ be the density function of the noise vector. For any combinatorial action $v_j \in S$, define $p_t(j)$ to be the probability of the event $w_t = v_j$ (with respect to the randomness of ℓ_0), which can be written as

$$\begin{aligned} p_t(j) &= \int_{\ell_0 \in \mathbb{R}^N} \mathbf{1} \left[v_j = \underset{w \in \Omega}{\operatorname{argmin}} \left\langle w, \sum_{s=0}^{t-1} \ell_s \right\rangle \right] h(\ell_0) d\ell_0 \\ &= \int_{\ell_0 \in \mathbb{R}^N} \mathbf{1} \left[v_j = \underset{w \in \Omega}{\operatorname{argmin}} \left\langle w, \sum_{s=0}^t \ell_s \right\rangle \right] h(\ell_0 + \ell_t) d\ell_0. \end{aligned}$$

(change of variable: from ℓ_0 to $\ell_0 + \ell_t$)

Since $h(\ell_0 + \ell_t) = \frac{\eta}{2} e^{-\eta \|\ell_0 + \ell_t\|_1} \leq \frac{\eta}{2} e^{-\eta \|\ell_0\|_1 + \eta \|\ell_t\|_1} = h(\ell_0) e^{\eta \|\ell_t\|_1}$, we continue with

$$p_t(j) \leq e^{\eta \|\ell_t\|_1} \int_{\ell_0 \in \mathbb{R}^N} \mathbf{1} \left[v_j = \underset{w \in \Omega}{\operatorname{argmin}} \left\langle w, \sum_{s=0}^t \ell_s \right\rangle \right] h(\ell_0) d\ell_0 = e^{\eta \|\ell_t\|_1} p_{t+1}(j).$$

Finally, this means

$$\begin{aligned} \mathbb{E}[\langle w_t - w_{t+1}, \ell_t \rangle] &= \sum_{j=1}^M (p_t(j) - p_{t+1}(j)) \langle v_j, \ell_t \rangle \leq \sum_{j=1}^M (1 - e^{-\eta \|\ell_t\|_1}) p_t(j) \langle v_j, \ell_t \rangle \\ &\leq \sum_{j=1}^M \eta \|\ell_t\|_1 \cdot p_t(j) \langle v_j, \ell_t \rangle \leq \eta N m \sum_{j=1}^M p_t(j) = \eta N m, \end{aligned}$$

where the second inequality uses the fact $1 - e^{-z} \leq z$ for all z . □

With this stability lemma, we prove the following regret bound.

Theorem 2. *The FTPL strategy described in Lemma 5 ensures $\mathbb{E}[\mathcal{R}_T] \leq \frac{2m}{\eta} (1 + \ln N) + \eta T N m$, which is $\mathcal{O}(m\sqrt{TN \ln N})$ with the optimally tuned η .*

Proof. It suffices to apply Lemma 3 and figure out the expected value of $\max_w \langle w, \ell_0 \rangle - \min_w \langle w, \ell_0 \rangle$, which by symmetry of the Laplace distribution, is $2\mathbb{E}[\max_{w \in \Omega} \langle w, \ell_0 \rangle]$, and bounded by $2m\mathbb{E}[\|\ell_0\|_\infty]$. For any value of $b > 0$, we further bound $\mathbb{E}[\|\ell_0\|_\infty]$ as

$$\begin{aligned} \mathbb{E}[\|\ell_0\|_\infty] &= \int_0^\infty \Pr[\|\ell_0\|_\infty \geq x] dx \leq b + \int_b^\infty \Pr[\|\ell_0\|_\infty \geq x] dx \\ &\leq b + \sum_{i=1}^N \int_b^\infty \Pr[|\ell_0(i)| \geq x] dx = b + N \int_b^\infty e^{-\eta x} dx = b + \frac{N}{\eta} e^{-\eta b}, \end{aligned}$$

where the first equality uses the standard fact: for any nonnegative random variable X with density function g , $\mathbb{E}[X] = \int_0^\infty xg(x)dx = \int_0^\infty \int_0^x g(x)dydx = \int_0^\infty \int_y^\infty g(x)dx dy = \int_0^\infty \Pr[X \geq y]dy$; and the second inequality is by a union bound. Picking $b = \frac{1}{\eta} \ln N$ leads to the minimum upper bound $\frac{1}{\eta} (1 + \ln N)$. Combining this fact, Lemma 5, and Lemma 3 thus finishes the proof. □

We conclude with two remarks. First, the role of η is exactly the same as the learning rate in FTRL, that is, to balance between the penalty term and the stability term. Second, the regret bound we prove above has the undesirable $\sqrt{N}$ dependence. However, this is merely an artifact of the loose analysis of the stability term. In HW1, we will improve it and replace the $\sqrt{N}$ dependence with $\sqrt{m}$, which then basically matches the regret bound of running Hedge inefficiently over all combinatorial actions, while enjoying the favorable time complexity of one linear optimization per round.

References

Marcus Hutter and Jan Poland. Adaptive online prediction by following the perturbed leader. *Journal of Machine Learning Research*, 6(Apr):639–660, 2005.

Adam Kalai and Santosh Vempala. Efficient algorithms for online decision problems. *Journal of Computer and System Sciences*, 71(3):291–307, 2005.

Martin Zinkevich. Online convex programming and generalized infinitesimal gradient ascent. In *Proceedings of the 20th International Conference on Machine Learning*, 2003.

CSCI 659 Lecture 3

Fall 2022

Instructor: Haipeng Luo

1 Adaptive Algorithms and Optimistic FTRL

While being able to deal with adversarial environments is one key advantage of online learning algorithms, they might be overly conservative exactly for the same reason. Indeed, even if the algorithm is minimax optimal, all we know is that *in the worst case*, it behaves optimally, but it is totally possible that in some benign cases, it performs worse than what one should hope for. Ideally, we want an algorithm that is robust in the worst case, but at the same time able to adapt to easier instances automatically with better than worst-case regret.

There are many different ways to define “easy” instances, each of which might lead to quite different algorithm design. In this lecture, we focus on one particular setup where at the beginning of each round, the learner has access to some prediction about what might happen for this round before making her decision. Formally, we consider the following variant of the OCO setup (recall that assuming linear loss functions is without loss of generality): at each round $t = 1, \dots, T$,

1. the learner first obtains $m_t \in \mathbb{R}^d$, a predictor for the loss vector ℓ_t of this round,
2. the learner then decides her action $w_t \in \Omega$;
3. the environment decides the loss vector ℓ_t ;
4. the learner suffers loss $\langle w_t, \ell_t \rangle$ and observes ℓ_t .

For now, you should think of m_t as some external information. Instead of assuming that it is a good predictor, our goal is to design an algorithm whose regret naturally depends on how accurate these predictors are: the more accurate they are, the smaller the regret. Later, we will discuss how to come up with these predictors on our own (so no extra external information is needed).

It turns out that a small change to the FTRL framework is enough to incorporate this extra predictor and lead to strong guarantees.¹ The resulting algorithm, called *Optimistic or Predictive FTRL* [Rakhlin and Sridharan, 2013], simply adds m_t to the current cumulative losses:

$$w_t = \operatorname{argmin}_{w \in \Omega} \left\langle w, m_t + \sum_{s < t} \ell_s \right\rangle + \frac{1}{\eta} \psi(w)$$

where as before ψ is a regularizer and η is a learning rate. The algorithm is “optimistic” in the sense that it believes m_t is close to ℓ_t and thus directly includes it when calculating the cumulative loss. Note that if m_t is indeed ℓ_t , then the above strategy is exactly the $(t+1)$ -th action of vanilla FTRL, which has constant regret based on the Be-the-Leader lemma discussed last week. In fact, these iterates from the vanilla FTRL will play a crucial role in the analysis, and we now denote them by $w'_1, w'_2, \dots$, so that

$$w'_{t+1} = \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s \leq t} \ell_s \right\rangle + \frac{1}{\eta} \psi(w).$$

¹Similar modifications apply to FTPL as well. For simplicity, we focus on FTRL in this lecture.

Extending the techniques from last lecture, we prove the following intermediate regret bound (you should compare this proof with that of Lemma 3 in Lecture 2).

Lemma 1. *The optimistic FTRL strategy ensures for any $u \in \Omega$:*

$$\sum_{t=1}^T \langle w_t - u, \ell_t \rangle \leq \frac{\psi(u) - \psi(w'_1)}{\eta} + \sum_{t=1}^T \langle w_t - w'_{t+1}, \ell_t - m_t \rangle - \sum_{t=1}^T \frac{D_\psi(w_t, w'_t) + D_\psi(w'_{t+1}, w_t)}{\eta}.$$

Proof. Define

$$\begin{aligned} \Phi_{t-1} &= \min_{w \in \Omega} \left\langle w, m_t + \sum_{s < t} \ell_s \right\rangle + \frac{1}{\eta} \psi(w) = \left\langle w_t, m_t + \sum_{s < t} \ell_s \right\rangle + \frac{\psi(w_t)}{\eta} \quad \text{and} \\ \Phi'_t &= \min_{w \in \Omega} \left\langle w, \sum_{s \leq t} \ell_s \right\rangle + \frac{1}{\eta} \psi(w) = \left\langle w'_{t+1}, \sum_{s \leq t} \ell_s \right\rangle + \frac{\psi(w'_{t+1})}{\eta}. \end{aligned}$$

Then by Lemma 2 of Lecture 2, we have

$$\Phi'_{t-1} - \Phi_{t-1} \leq -\langle w_t, m_t \rangle - \frac{1}{\eta} D_\psi(w_t, w'_t)$$

and

$$\Phi_{t-1} - \Phi'_t \leq -\langle w'_{t+1}, \ell_t - m_t \rangle - \frac{1}{\eta} D_\psi(w'_{t+1}, w_t).$$

Adding up both and rearranging shows

$$\langle w_t, \ell_t \rangle \leq \Phi'_t - \Phi'_{t-1} + \langle w_t - w'_{t+1}, \ell_t - m_t \rangle - \frac{D_\psi(w_t, w'_t) + D_\psi(w'_{t+1}, w_t)}{\eta}.$$

Finally, summing over t , telescoping, and realizing $\Phi'_T \leq \langle u, \sum_{t \leq T} \ell_t \rangle + \frac{\psi(u)}{\eta}$ and $\Phi'_0 = \frac{\psi(w'_1)}{\eta}$ finishes the proof. $\square$

An important distinction between this regret bound and that of vanilla FTRL is the improvement in the stability term, which now depends on the difference between ℓ_t and its predicted value m_t . Indeed, this is even better illustrated when we use a strongly convex regularizer and obtain the following final regret bound.

Theorem 1. *Suppose that the regularizer ψ is strongly convex with respect to some norm $\|\cdot\|$ and has range $B_\psi = \max_{u \in \Omega} \psi(u) - \min_{u \in \Omega} \psi(u)$. Then Optimistic FTRL ensures*

$$\mathcal{R}_T \leq \frac{B_\psi}{\eta} + \eta \sum_{t=1}^T \|\ell_t - m_t\|_*^2 - \frac{1}{4\eta} \sum_{t=2}^T \|w_t - w_{t-1}\|^2.$$

Proof. It suffices to handle the stability term and the negative terms from the bound of Lemma 1. Using Lemma 4 of Lecture 2, we immediately obtain $\|w_t - w'_{t+1}\| \leq \eta \|\ell_t - m_t\|_*$, and thus the stability term $\langle w_t - w'_{t+1}, \ell_t - m_t \rangle$ is further bounded by $\eta \|\ell_t - m_t\|_* \|\ell_t - m_t\|_* \leq \eta \|\ell_t - m_t\|_*^2$. Next, for the term $\sum_{t=1}^T D_\psi(w_t, w'_t) + D_\psi(w'_{t+1}, w_t)$, we first drop $D_\psi(w_1, w'_1)$ and $D_\psi(w'_{T+1}, w_T)$ (both nonnegative), and then shift the index to arrive at a lower bound:

$$\sum_{t=2}^T D_\psi(w_t, w'_t) + D_\psi(w'_t, w_{t-1}),$$

which can be further lower bounded as

$$\begin{aligned} & \frac{1}{2} \sum_{t=2}^T \|w_t - w'_t\|^2 + \|w'_t - w_{t-1}\|^2 && \text{(by strong convexity)} \\ & \geq \frac{1}{4} \sum_{t=2}^T (\|w_t - w'_t\| + \|w'_t - w_{t-1}\|)^2 && (a^2 + b^2 \geq \frac{(a+b)^2}{2}) \\ & \geq \frac{1}{4} \sum_{t=2}^T \|w_t - w_{t-1}\|^2. && \text{(triangle inequality)} \end{aligned}$$

Combining these with Lemma 1 finishes the proof. $\square$

For a moment, assume that we can set the learning rate as $\eta = \sqrt{B_\psi / \sum_{t=1}^T \|\ell_t - m_t\|_*^2}$. Then ignoring the last negative term, we obtain a regret bound of order $\mathcal{O}\left(\sqrt{B_\psi \sum_{t=1}^T \|\ell_t - m_t\|_*^2}\right)$, showing that the performance of the algorithm indeed depends on how accurate the predictors m_t are. The prediction error $\sum_{t=1}^T \|\ell_t - m_t\|_*^2$ thus serves as a measure of difficulty for this problem. For example, if the predictors are perfect with zero error (that is, $m_t = \ell_t$), then we simply have zero regret. As a less extreme example, if the predictors are somewhat accurate such that $\sum_{t=1}^T \|\ell_t - m_t\|_*^2 = o(T)$, then the regret bound becomes $o(\sqrt{T})$ (ignoring the dependence on B_ψ). On the other hand, even if the predictors are all inaccurate with $\Theta(T)$ error (this is under a mild condition that $\|\ell_t\|_*$ is bounded, so $\|m_t\|_*$ is naturally bounded as well), the regret bound simply recovers the $\mathcal{O}(\sqrt{T})$ worst-case regret of vanilla FTRL. In other words, optimistic FTRL is never worse than vanilla FTRL, but could perform much better as long as we are not in the worst-case scenario.

The aforementioned choice of the learning rate η requires knowing the prediction error ahead of time, which is rather unrealistic. However, this can be easily resolved. The simplest approach is the doubling trick (which you have seen in HW1): make a guess on the value of the prediction error (starting from 1 for example), optimally tune the algorithm based on this guess, and once the actual error exceeds the guess, double the guess and restart the algorithm. This leads to the same regret bound up to a constant factor for the exact same reason as the case you need to solve in HW1.

While this doubling trick is a very convenient and general way to resolve such tuning issues, it might not be a practical one. Indeed, restarting the algorithm means completely discarding the previously collected data, which often sounds unreasonable (if not absurd) to practitioners. Fortunately, most of the time such tuning issues can also be resolved by a *time-varying* learning rate that follows the form of the optimal tuning but with only the observed data. For example, in our case, using learning rate $\eta_t = \sqrt{B_\psi / \sum_{s < t} \|\ell_s - m_s\|_*^2}$ at time t would also lead to the same regret bound. We omit the details and leave this as an exercise for interested readers.

Finally, the negative term $-\frac{1}{4\eta} \sum_{t=2}^T \|w_t - w_{t-1}\|^2$ in the regret bound, which will be used to derive something important in the next lecture, might seem a bit mysterious — it seemingly indicates that the less stable the algorithm, the smaller the regret, exactly the opposite of what we showed before. One explanation is: keep in mind that what Theorem 1 shows is an *upper bound* of the regret, and it might not truly characterizes the behavior of the actual regret itself. In fact, if one takes a step back and focuses on the term $\langle w_t - w'_{t+1}, \ell_t - m_t \rangle - \frac{1}{\eta} D_\psi(w'_{t+1}, w_t)$ in Lemma 1, which is bounded by $\|w_t - w'_{t+1}\| \|\ell_t - m_t\|_* - \frac{1}{2\eta} \|w_t - w'_{t+1}\|^2$, then by seeing this as a quadratic of $\|w_t - w'_{t+1}\|$ whose range is in $[0, \eta \|\ell_t - m_t\|_*]$ (based on our earlier analysis), we conclude that this upper bound is again strictly increasing in $\|w_t - w'_{t+1}\|$, thus in favor of stable algorithms and consistent with our earlier intuition.

1.1 Constructing the Predictors

So far we have treated m_t as some external information. Can we instead construct m_t ourselves without using any extra information? Intuitively, if we believe that the past loss vectors $\ell_1, \dots, \ell_{t-1}$ are predictive for the current loss vector ℓ_t , then we should be able to come up with a reasonable predictor m_t based on $\ell_1, \dots, \ell_{t-1}$. Indeed, we can treat this as another machine learning problem such as time series forecasting (an example would be predicting the next word based on the first few words of a sentence), which is often solved by autoregressive models (that is, predicting the next value using a linear combination of past values). In fact, we can also treat this as *another OCO problem*, where the learner is required to predict m_t at each round, with loss function $\|\ell_t - m_t\|_*^2$.

Here, we only spell out perhaps the simplest method: use the most recent loss vector as the predictor, that is, $m_t = \ell_{t-1}$ (with arbitrary ℓ_0). This leads to a bound of order $\mathcal{O}\left(\sqrt{B_\psi \sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_*^2}\right)$, which depends on the variation of the loss sequence (often called the *path length*). Thus, as long as the environment is not changing rapidly, the algorithm is able to exploit this fact and achieves better performance. In the next lecture, we will see a concrete example of such slowly changing environments.

Another remark is that the path length $\sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_*^2$ is always bounded by

$$\sum_{t=1}^T \|\ell_t - \mu + \mu - \ell_{t-1}\|_*^2 \leq 2 \sum_{t=1}^T \left(\|\ell_t - \mu\|_*^2 + \|\mu - \ell_{t-1}\|_*^2 \right) = \mathcal{O} \left(\sum_{t=1}^T \|\ell_t - \mu\|_*^2 \right)$$

for any $\mu \in \mathbb{R}^d$. In particular, we can pick the μ that minimizes $\sum_{t=1}^T \|\ell_t - \mu\|_*^2$, showing that our time-varying predictors are overall never worse than a fixed predictor. For example, when $\|\cdot\|$ is the L_2 norm, then the optimal μ is simply the average loss $\frac{1}{T} \sum_{t=1}^T \ell_t$, and the bound becomes the variance of the loss vectors. Note that there are clearly cases where the variance is much worse than the path length though (try to come up with one yourself).

2 Refinement for the Expert Problem

For the rest of the lecture, we focus on the expert problem (switching the notation from $w \in \Omega$ to $p \in \Delta(N)$ again). Applying Optimistic FTRL with the negative entropy regularizer, which corresponds to the following strategy called *Optimistic Hedge*:

$$p_t(i) \propto \exp \left(-\eta \left(m_t(i) + \sum_{s < t} \ell_s(i) \right) \right),$$

we obtain a regret bound of order $\mathcal{O} \left(\sqrt{\sum_{t=1}^T \|\ell_t - m_t\|_\infty \ln N} \right)$ (with the optimal tuning). This bound depends on the worst prediction error among all experts at each time (due to the infinity norm). Instead, is there a way to depend on, for example, only the prediction error for the optimal expert?

To address this question, we first point out that the same algorithm above in fact enjoys a more refined regret bound that depends on some weighted average of the prediction errors (weighted by p_t precisely), instead of the worst case one. This is done by analyzing the stability term more carefully, without directly invoking strong convexity. The result is summarized in the following theorem, and is in fact a generalization of the bound we proved in Theorem 2 of Lecture 1.

Theorem 2. *As long as $\eta|\ell_t(i) - m_t(i)| \leq 1$ holds for all t and i , Optimistic Hedge ensures*

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) (\ell_t(i) - m_t(i))^2.$$

Proof. Using Lemma 1, we have

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \sum_{t=1}^T \left(\langle p_t - p'_{t+1}, \ell_t - m_t \rangle - \frac{1}{\eta} D_\psi(p'_{t+1}, p_t) \right).$$

Note that we have kept one Bregman divergence term here (but dropped the other one $D_\psi(p_t, p'_t)$), which is crucial for the proof. Specifically, we will show $\langle p_t - p'_{t+1}, \ell_t - m_t \rangle - \frac{1}{\eta} D_\psi(p'_{t+1}, p_t) \leq \eta \sum_{i=1}^N p_t(i) (\ell_t(i) - m_t(i))^2$, which then finishes the proof. Indeed, by definitions we have

$$p'_{t+1}(i) \propto \exp \left(-\eta \sum_{s \leq t} \ell_s(i) \right) \propto p_t(i) \exp(-\eta(\ell_t(i) - m_t(i))).$$

Then, noticing that the Bregman divergence $D_\psi(p, q)$ with respect to the negative entropy is simply the KL-divergence $\text{KL}(p, q) = \sum_{i=1}^N p(i) \ln \frac{p(i)}{q(i)}$, we have

$$\begin{aligned} -\frac{1}{\eta} D_\psi(p'_{t+1}, p_t) &= \frac{1}{\eta} \sum_{i=1}^N p'_{t+1}(i) \ln \frac{p_t(i)}{p'_{t+1}(i)} \\ &= \frac{1}{\eta} \sum_{i=1}^N p'_{t+1}(i) \ln \frac{\sum_{j=1}^N p_t(j) \exp(-\eta(\ell_t(j) - m_t(j)))}{\exp(-\eta(\ell_t(i) - m_t(i)))} \\ &= \langle p'_{t+1}, \ell_t - m_t \rangle + \frac{1}{\eta} \ln \left(\sum_{j=1}^N p_t(j) \exp(-\eta(\ell_t(j) - m_t(j))) \right). \end{aligned}$$

The rest of the proof simply repeats what we did in Lecture 1:

$$\begin{aligned}
& \frac{1}{\eta} \ln \left(\sum_{j=1}^N p_t(j) \exp(-\eta(\ell_t(j) - m_t(j))) \right) \\
& \leq \frac{1}{\eta} \ln \left(\sum_{j=1}^N p_t(j) (1 - \eta(\ell_t(j) - m_t(j)) + \eta^2(\ell_t(j) - m_t(j))^2) \right) \\
& \quad (e^{-y} \leq 1 - y + y^2, \forall y \geq -1) \\
& = \frac{1}{\eta} \ln \left(1 - \eta \langle p_t, \ell_t - m_t \rangle + \eta^2 \sum_{j=1}^N p_t(j) (\ell_t(j) - m_t(j))^2 \right) \\
& \leq -\langle p_t, \ell_t - m_t \rangle + \eta \sum_{j=1}^N p_t(j) (\ell_t(j) - m_t(j))^2, \quad (\ln(1 + y) \leq y)
\end{aligned}$$

where in the first inequality we use the condition $\eta|\ell_t(j) - m_t(j)| \leq 1$ of the theorem. $\square$

With this refined regret bound, we make the following modification to the algorithm, which further enables us to derive a bound only in terms of the prediction error of the best expert:

$$p_t(i) \propto \exp \left(-\eta \left(m_t(i) + \sum_{s < t} (\ell_s(i) + c_s(i)) \right) \right), \quad (1)$$

where $c_s(i) = 4\eta(\ell_s(i) - m_s(i))^2$ is sometimes referred to as a *loss correction term*. Compared to Optimistic Hedge, this new algorithm adds the correction term to the loss of each expert at each time, further penalizing those experts with a large prediction error. The regret of this algorithm against any expert thus naturally depends on the prediction error for this expert, as shown below.

Theorem 3. *As long as $\eta|\ell_t(i) - m_t(i)| \leq \frac{1}{4}$ holds for all t and i , Algorithm (1) ensures that for any expert $j \in [N]$, the regret against this expert is bounded as*

$$\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle \leq \frac{\ln N}{\eta} + 4\eta \sum_{t=1}^T (\ell_t(j) - m_t(j))^2$$

(where e_i is the standard basis vector with the i -th coordinate being 1).

Proof. First observe that Algorithm (1) is exactly feeding Optimistic Hedge with the loss vector $\ell_t + c_t$ (instead of just ℓ_t). This motivates us to apply Theorem 1 with ℓ_t replaced by $\ell_t + c_t$. To do so, we first verify that the condition of Theorem 1 is indeed satisfied:

$$\begin{aligned}
\eta|\ell_t(i) - m_t(i) + c_t(i)| &= \eta|\ell_t(i) - m_t(i) + 4\eta(\ell_t(i) - m_t(i))^2| \quad (\text{definition of } c_t(i)) \\
&\leq \eta|\ell_t(i) - m_t(i)|(1 + 4\eta|\ell_t(i) - m_t(i)|) \\
&\leq 2\eta|\ell_t(i) - m_t(i)| \\
&\leq \frac{1}{2},
\end{aligned} \quad (2)$$

where both of the last two inequalities use the condition $\eta|\ell_t(i) - m_t(i)| \leq \frac{1}{4}$. Thus, we can indeed apply Theorem 1 and obtain for any expert $j \in [N]$:

$$\begin{aligned}
\sum_{t=1}^T \langle p_t - e_j, \ell_t + c_t \rangle &\leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) (\ell_t(i) - m_t(i) + c_t(i))^2 \\
&\leq \frac{\ln N}{\eta} + 4\eta \sum_{t=1}^T \sum_{i=1}^N p_t(i) (\ell_t(i) - m_t(i))^2 \quad (\text{by Eq. (2)}) \\
&= \frac{\ln N}{\eta} + \sum_{t=1}^T \langle p_t, c_t \rangle, \quad (\text{definition of } c_t)
\end{aligned}$$

which then completes the proof after moving the term $\sum_{t=1}^T \langle p_t - e_j, c_t \rangle$ to the right. $\square$

Let $j^* \in \operatorname{argmin}_j \sum_{t=1}^T \ell_t(j)$ be an optimal expert. Then with the learning rate η set to $\min \left\{ 1/4, \sqrt{\ln N / \sum_{t=1}^T (\ell_t(j^*) - m_t(j^*))^2} \right\}$ (we prevent this from going above $\frac{1}{4}$ to make sure that the condition $\eta |\ell_t(i) - m_t(i)| \leq \frac{1}{4}$ is always satisfied for losses/predictors in $[0, 1]$), we obtain the bound $\mathcal{R}_T = \mathcal{O} \left(\sqrt{\sum_{t=1}^T (\ell_t(j^*) - m_t(j^*))^2 \ln N} + \ln N \right)$, which only depends on the prediction error of the optimal expert. However, an obvious issue is that this tuning of the learning rate depends on the knowledge of the best expert j^* , which we clearly do not have ahead of time. Note that this issue is very different from the tuning issue discussed earlier in Section 1, which as mentioned can be easily solved via a time-varying learning rate. Here, the difficulty is that each expert requires a different tuning. A natural attempt would be to let the learning rate vary not only over time, but also across the experts, leading to the following algorithm

$$p_t(i) \propto \exp \left(-\eta_t(i) \left(m_t(i) + \sum_{s < t} (\ell_s(i) + c_s(i)) \right) \right), \quad (3)$$

where we replace the fixed learning rate η in Eq. (1) with

$$\eta_t(i) = \min \left\{ \frac{1}{4}, \sqrt{\frac{\ln N}{\sum_{s < t} (\ell_s(i) - m_t(i))^2}} \right\}. \quad (4)$$

Unfortunately, no one has been able to prove (or disprove) that this works, and this tuning issue has been referred to as the *impossible tuning issue*.

Somewhat surprisingly, a recent work by [Chen et al. \[2021\]](#) makes this “impossible tuning” possible, and the solution is very close to the natural idea above. Recall that Algorithm (1) can be written as the following Optimistic FTRL form:

$$p_t = \operatorname{argmin}_{p \in \Delta(N)} \left\langle p, m_t + \sum_{s < t} (\ell_s + c_s) \right\rangle + \Psi(p), \quad (5)$$

where $\Psi(p) = \frac{1}{\eta} \sum_{i=1}^N p(i) \ln p(i)$ is the entropy regularizer with a fixed learning rate. The solution of [\[Chen et al., 2021\]](#) is simply to replace Ψ with

$$\Psi_t(p) = \sum_{i=1}^N \frac{1}{\eta_t(i)} p(i) \ln p(i)$$

where $\eta_t(i)$ is defined in the same way as Eq. (4).² The first important thing to notice here is that this is *not* the same algorithm as Eq. (3), even though they look very closely related. Indeed, by writing down the Lagrangian and setting the gradient to zero, we see that the algorithm of [\[Chen et al., 2021\]](#) is in fact the following:

$$p_t(i) = \exp \left(-\eta_t(i) \left(\lambda + m_t(i) + \sum_{s < t} (\ell_s(i) + c_s(i)) \right) - 1 \right), \quad (6)$$

for some Lagrangian multiplier λ that can be efficiently found via binary search based on the fact $\sum_i p_t(i) = 1$. Note that if $\eta_t(i)$ does not vary over different i ’s, then the above is indeed the same as Eq. (3) since the factor $\exp(-\eta_t(i)\lambda - 1)$ simply becomes the constant previously hidden in the $\propto$ sign. But for general $\eta_t(i)$ that varies over i , such as Eq. (4), there is no closed-form for λ , and the two algorithms are thus indeed not equivalent.

[Chen et al. \[2021\]](#) prove the following guarantee of this algorithm: for *any* expert j ,

$$\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle = \mathcal{O} \left(\sqrt{\sum_{t=1}^T (\ell_t(j) - m_t(j))^2 \ln(NT)} + \ln(NT) \right). \quad (7)$$

Other than the small extra $\ln T$ factor (which we will hide using the $\tilde{\mathcal{O}}(\cdot)$ notation in the remaining discussion), this guarantee is even stronger than our earlier goal $\mathcal{R}_T = \mathcal{O} \left(\sqrt{\sum_{t=1}^T (\ell_t(j^*) - m_t(j^*))^2 \ln N} + \ln N \right)$ (think about why the former could be *strictly* stronger).

²There are other minor modifications of the algorithm that we omit here; see [\[Chen et al., 2021\]](#) if interested.

2.1 Implications

Why do we bother to spend so much effort to obtain regret bound (7), which in fact might not even be better than the version stated in Theorem 2 with the weighted error? It turns out that there are indeed many nice applications of Eq. (7). We explore some of these in the rest of this lecture and will also see more in the future.

Application 1 First, consider not using any predictors at all, that is, $m_t = \mathbf{0}$ (the all-zero vector). Then Eq. (7) implies $\mathcal{R}_T = \tilde{O}\left(\sqrt{\sum_{t=1}^T \ell_t(j^*)^2 \ln N} + \ln N\right) = \tilde{O}(\sqrt{L^* \ln N} + \ln N)$ where $L^* = \sum_{t=1}^T \ell_t(j^*)$ is the loss of the best expert. Therefore, the better the optimal expert, the faster our algorithm converges to such a good expert. This is often called a “small-loss” regret bound, and we will see its application for learning in games in the future.

In the next two applications, we make use of a useful trick: for each t , if $m_t(i)$ is the same for all expert i , then by examining either Eq. (5) or Eq. (6), one sees that the distribution p_t is in fact *independent* of m_t (but any future p_s with $s > t$ depends on m_t through c_t of course). This small observation implies that m_t can even “cheat” and depend on ℓ_t , *unknown* at the beginning of round t , as long as it has identical coordinates and becomes available at the end of round t .

Application 2 The first such “cheating” predictor we consider is $m_t(i) = \ell_t(1)$ for all i . Applying Eq. (7), we know that the regret of the algorithm against Expert 1 is only $\tilde{O}(\ln N)$, while the regret against all other experts is still at most $\tilde{O}(\sqrt{T \ln N})$. This is useful in for example the following scenario: a company has been using a decent online learning algorithm $\mathcal{A}$ for a while, and one day as a researcher of the company you propose a new algorithm $\mathcal{B}$ that you think might perform even better. What the company can do now is to treat these two algorithms as experts, and combine them using an expert algorithm. However, since this is related to the profit of the company and we have no prior knowledge about how $\mathcal{B}$ might actually perform in practice, to be conservative we want our overall performance to be at least not too far away from the baseline $\mathcal{A}$. Then the guarantee mentioned above exactly fits our need here: let $\mathcal{A}$ be Expert 1 so that the loss of the final algorithm is no worse than $\mathcal{A}$ by only $\tilde{O}(1)$, but at the same time is also comparable to $\mathcal{B}$ if it is indeed a much better algorithm.

Question 1. *How is this approach compared to the common practice of doing $\mathcal{A}/\mathcal{B}$ testing?*

Question 2. *Can we go one step further and use the following cheating predictor: $m_t(i) = \ell_t(j^*)$ for all i , where j^* is the optimal expert, so that our regret against j^* is always $\tilde{O}(\ln N)$?*

Application 3 The next “cheating” predictor we consider is $m_t(i) = \langle p_t, \ell_t \rangle$ for all i . Eq. (7) implies that the regret against any expert j is bounded as

$$\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle = \tilde{O} \left(\sqrt{\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle^2 \ln N} + \ln N \right), \quad (8)$$

scaling with the cumulative squared instantaneous regret against the same expert j . While the meaning of this bound is less intuitive, it in fact has many further strong implications. The first one being that it recovers the aforementioned “small-loss” bounds.

Theorem 4. *If an algorithm ensures Eq. (8), then it also ensures*

$$\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle = \tilde{O} \left(\sqrt{\sum_{t=1}^T \ell_t(j) \ln N} + \ln N \right),$$

Proof. For conciseness, define $R_T(j) = \sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle$ and $L_T(j) = \sum_{t=1}^T \ell_t(j)$. With the fact $\langle p_t - e_j, \ell_t \rangle^2 \leq |\langle p_t - e_j, \ell_t \rangle| \leq \langle p_t + e_j, \ell_t \rangle = \langle p_t - e_j + 2e_j, \ell_t \rangle$, Eq. (8) implies:

$$R_T(j) = \tilde{O} \left(\sqrt{(R_T(j) + 2L_T(j)) \ln N} + \ln N \right) = \tilde{O} \left(\sqrt{R_T(j) \ln N} + \sqrt{L_T(j) \ln N} + \ln N \right),$$

a recursive guarantee on $R_T(j)$. Further using the fact that $x \leq b\sqrt{x} + c$ implies $x \leq b^2 + 2c$ (verify it yourself) then proves the theorem. $\square$

Even more surprisingly, the same bound also implies almost constant regret in a stochastic setting.

Theorem 5. *Suppose that there exists a good expert j^* and a constant gap $\Delta \in (0, 1]$ such that $\mathbb{E}_t[\ell_t(i) - \ell_t(i^*)] \geq \Delta$ for all t and $i \neq i^*$, where $\mathbb{E}_t$ denotes the conditional expectation given all randomness before round t . If an algorithm ensures Eq. (8), then its regret against j^* is at most $\tilde{\mathcal{O}}\left(\frac{\ln N}{\Delta}\right)$.*

Proof. Let $R_T(j^*) = \sum_{t=1}^T \langle p_t - e_{j^*}, \ell_t \rangle$ be the regret against expert j^* . By the condition we have

$$\mathbb{E}_t[\langle p_t - e_{j^*}, \ell_t \rangle] = \mathbb{E}_t \left[\sum_{i \neq j^*} p_t(i) (\ell_t(i) - \ell_t(j^*)) \right] \geq \Delta \sum_{i \neq j^*} p_t(i) = \Delta(1 - p_t(j^*)),$$

and thus $\mathbb{E}[R_T(j^*)] \geq \Delta B$ where we define $B = \mathbb{E} \left[\sum_{t=1}^T (1 - p_t(j^*)) \right]$. On the other hand,

$$\begin{aligned} \mathbb{E} \left[\sum_{t=1}^T \langle p_t - e_{j^*}, \ell_t \rangle^2 \right] &\leq \mathbb{E} \left[\sum_{t=1}^T \left| \sum_{i=1}^N p_t(i) (\ell_t(i) - \ell_t(j^*)) \right|^2 \right] \\ &\leq \mathbb{E} \left[\sum_{t=1}^T \sum_{i=1}^N p_t(i) |\ell_t(i) - \ell_t(j^*)|^2 \right] \leq \mathbb{E} \left[\sum_{t=1}^T \sum_{i \neq j^*} p_t(i) \right] = B. \end{aligned}$$

Combining these with Eq. (8) implies

$$\begin{aligned} \Delta B &\leq \mathbb{E}[R_T(j^*)] = \tilde{\mathcal{O}} \left(\mathbb{E} \left[\sqrt{\sum_{t=1}^T \langle p_t - e_{j^*}, \ell_t \rangle^2 \ln N} \right] + \ln N \right) \\ &= \tilde{\mathcal{O}} \left(\sqrt{\mathbb{E} \left[\sum_{t=1}^T \langle p_t - e_{j^*}, \ell_t \rangle^2 \right] \ln N + \ln N} \right) \quad (\text{Jensen's inequality}) \\ &\leq \tilde{\mathcal{O}} \left(\sqrt{B \ln N} + \ln N \right). \end{aligned} \tag{9}$$

Applying $x \leq b\sqrt{x} + c \Rightarrow x \leq b^2 + 2c$ again shows $B = \tilde{\mathcal{O}}(\ln N / \Delta^2)$. Plugging this back to Eq. (9) finishes the proof. $\square$

We emphasize that the last two theorems hold simultaneously as long as an algorithm satisfies Eq. (8) (the detail of the algorithm is irrelevant), illustrating strong adaptivity of the algorithm. In the future, we will see one more important consequence of having regret bound (8).

Application 4 In the last application, we take $m_t(i) = m'_t(i) + \langle p_t, \ell_t - m'_t \rangle$ for another arbitrary predictor $m'_t \in [0, 1]^N$ (that is available at the beginning of round t). For a similar reason as the last two applications, even though the term $\langle p_t, \ell_t - m'_t \rangle$ is unknown before the end of round t , the algorithm is still valid since this part is the same across all i . In fact, Application 3 is just a special case with $m'_t = \mathbf{0}$. Eq. (7) now implies that the regret against any expert j is bounded as

$$\sum_{t=1}^T \langle p_t - e_j, \ell_t \rangle = \tilde{\mathcal{O}} \left(\sqrt{\sum_{t=1}^T \langle p_t - e_j, \ell_t - m'_t \rangle^2 \ln N + \ln N} \right), \tag{10}$$

which is further bounded by $\tilde{\mathcal{O}} \left(\sqrt{\sum_{t=1}^T \|\ell_t - m'_t\|_\infty^2 \ln N + \ln N} \right)$, a bound similar to the one we saw earlier for Optimistic Hedge (without using correction terms). However, the benefit here is that simultaneously, this bound (10) also enjoys the same fast rate consequence stated in Theorem 5.

Theorem 6. *In the same environment as stated in Theorem 5, if an algorithm ensures Eq. (10), then its regret against j^* is at most $\tilde{\mathcal{O}}\left(\frac{\ln N}{\Delta}\right)$.*

The proof is almost identical to that for Theorem 5, and we leave it as an exercise.

We conclude by pointing out that there are in fact many more applications of Eq. (7), including those for the general OCO problem (obtained by combining different OCO algorithms via an expert algorithm satisfying Eq. (7)). See [Chen et al., 2021] for details.

References

- Liyu Chen, Haipeng Luo, and Chen-Yu Wei. Impossible tuning made possible: A new expert algorithm and its applications. In *Conference on Learning Theory*, pages 1216–1259. PMLR, 2021.
- Sasha Rakhlin and Karthik Sridharan. Optimization, learning, and games with predictable sequences. In *Advances in Neural Information Processing Systems 26*, 2013.

CSCI 659 Lecture 4

Fall 2022

Instructor: Haipeng Luo

1 Two-Player Zero-Sum Games

In this lecture, we explore the connection between game theory and online learning, and explain why online learning is the cornerstone for solving large-scale games. We start by considering the simplest two-player zero-sum normal-form games and some fundamental concepts of game theory, before discussing how it is connected to online learning in the next section.

A two-player zero-sum normal-form game can be represented using a payoff matrix $G \in [0, 1]^{N \times M}$. Here, one player (called the row or min player, with pronoun “he”) has N possible actions and another player (called the column or max player, with pronoun “she”) has M possible actions, and entry $G(i, j)$ represents the loss of the row player if he selects action i while the column player selects action j , which is also the reward for the column player (hence zero-sum).

A classical example is the Rock-Paper-Scissors game. If we assign loss 1 for losing the game, 0 for winning, and $1/2$ for a tie, then G is

$$\begin{array}{cc} & \begin{array}{ccc} \text{Rock} & \text{Paper} & \text{Scissors} \end{array} \\ \begin{array}{c} \text{Rock} \\ \text{Paper} \\ \text{Scissors} \end{array} & \left(\begin{array}{ccc} 1/2 & 1 & 0 \\ 0 & 1/2 & 1 \\ 1 & 0 & 1/2 \end{array} \right) . \end{array}$$

The exact same idea applies to much more complicated games, including those that involve sequential structures. For example, a poker game can be formulated in this way with a huge G where each action corresponds to a complete strategy of playing this poker game.

Instead of playing a fixed action (also called “pure strategy”), it often makes more sense to play an action randomly according to a distribution (called “mixed strategy”). For some mixed strategy $p \in \Delta(N)$ for the row player and some mixed strategy $q \in \Delta(M)$ for the column player, the expected loss for the row player, which is also the expected reward of the column player, is denoted by $G(p, q) = p^\top G q = \sum_{i=1}^N \sum_{j=1}^M p(i)q(j)G(i, j)$. We will also use the notation $G(i, q)$ and $G(p, j)$ to denote $\sum_{j=1}^M q(j)G(i, j)$ and $\sum_{i=1}^N p(i)G(i, j)$ respectively.

The most fundamental solution concept in game theory is the *Nash equilibrium*. A pair of mixed strategy (p, q) is called a Nash equilibrium if neither player has the incentive to change his/her strategy given that the opponent is keeping his/hers. In other words, both players are best-responding to each other and thus happy about the current situation. Formally, this means that

$$G(p, j) \leq G(p, q) \leq G(i, q), \quad \forall i \in [N], j \in [M].$$

One can easily verify that for the Rock-Paper-Scissors game, playing uniformly at random for both players is a Nash Equilibrium (in fact the only one).

On the other hand, minimax solution is also a natural concept for a two-player zero-sum game. Specifically, in the worst case, playing p leads to a loss of at most $\max_q G(p, q)$ for the row player if the column player sees p before making her decision, and therefore in this sense the worst-case

optimal strategy for the row player is $p^* \in \operatorname{argmin}_p \max_q G(p, q)$, which is called the minimax strategy. Similarly, the maximin strategy for the column player is $q^* \in \operatorname{argmax}_q \min_p G(p, q)$. Together, we call (p^*, q^*) a minimax solution of the game.

Therefore, $\min_p \max_q G(p, q)$ and $\max_q \min_p G(p, q)$ are respectively the smallest loss and the largest reward that the respective player can hope for when against an optimal opponent who plays second. How are these two values related? Intuitively, both players are playing optimally in the two expressions, but there should be no disadvantage in playing second. Therefore we should have $\min_p \max_q G(p, q) \geq \max_q \min_p G(p, q)$ (row player playing first on the left and second on the right). Indeed, this is true by a simple argument:

$$\min_p \max_q G(p, q) = \max_q G(p^*, q) \geq G(p^*, q^*) \geq \min_p G(p, q^*) = \max_q \min_p G(p, q).$$

While one may imagine that this inequality should be strict at least for some cases, the surprising fact is that the reverse inequality is also true and thus the two values are exactly the same! In other words, if both players are playing optimally, there is no difference in playing first or second. This is the celebrated von Neumann's minimax theorem.

Theorem 1 (von Neumann's minimax theorem). *For any two-player zero-sum game $G \in [0, 1]^{N \times M}$, we have*

$$\min_{p \in \Delta(N)} \max_{q \in \Delta(M)} G(p, q) = \max_{q \in \Delta(M)} \min_{p \in \Delta(N)} G(p, q).$$

This single value is called the value of the game, denoted by $v(G)$. The original proof relies on a fixed-point theorem, but we will prove it in a different way *by running online learning algorithms* in the next section. For now, we discuss the connection between these different notions we have discussed so far: Nash equilibrium, minimax solution, and the value of the game.

Theorem 2. *A pair of mixed strategy (p, q) is a Nash equilibrium if and only if it is also a minimax solution. Moreover, $G(p, q)$ is the value of the game.*

Proof. Suppose that (p, q) is a Nash equilibrium. By definition and optimality, we have

$$\min_{p'} \max_{q'} G(p', q') \leq \max_{q'} G(p, q') = G(p, q) = \min_{p'} G(p', q) \leq \max_{q'} \min_{p'} G(p', q').$$

Now by the minimax theorem, the above inequalities are actually all equalities, which implies $G(p, q) = v(G)$ and also that (p, q) is a minimax solution.

For the other direction, if (p, q) is a minimax solution, then again by optimality and definition

$$\min_{p'} \max_{q'} G(p', q') = \max_{q'} G(p, q') \geq G(p, q) \geq \min_{p'} G(p', q) = \max_{q'} \min_{p'} G(p', q').$$

By the minimax theorem, the above is again a sequence of equalities, which implies $G(p, q) = v(G)$ and that (p, q) is a Nash equilibrium. $\square$

By this theorem and the fact that minimax solutions always exist (due to the compactness of the simplex), Nash equilibria also always exist.

Question 1. *Theorem 2 asserts that a minimax solution (p^*, q^*) is a Nash equilibrium and also $q^* \in \operatorname{argmax}_q G(p^*, q)$. Is it true that (p^*, q) for any $q \in \operatorname{argmax}_q G(p^*, q)$ is a Nash equilibrium?*

2 Repeated Play and Connections to Online Learning

How should we play a game? If we know the matrix G , then playing with the minimax solutions seems to be a good strategy. However, what if G is unknown? Moreover, minimax solutions might also be overly conservative. For example, if we play Rock-Paper-Scissors with a friend who we know prefers to play Paper, then should we still play uniformly at random? In general, how do we exploit the fact that the opponent might not be optimal?

If the game is only played once, then there is little we can do. However, it is often the case that a game is repeatedly played for many times. In this case, there is hope to apply learning algorithms to learn to play well against a specific opponent. We take the row player as an example and formulate the learning model as follows: at round $t = 1, \dots, T$,

1. the row player chooses a mixed strategy $p_t \in \Delta(N)$;
2. the column player chooses a mixed strategy $q_t \in \Delta(M)$ (which may or may not depend on p_t);
3. the row player observes $G(i, q_t)$ for all $i \in [N]$.

The feedback model might look unrealistic in this case, but all discussions in this section extend trivially to the weaker model where the row player only observes $G(i, j_t)$ for all i , where j_t is sampled from q_t . In other words, for the realized action j_t of the opponent, the player knows the loss of each of his possible action. Of course, the most challenging case is when the player only observes $G(i_t, j_t)$ where $i_t \sim p_t$ and $j_t \sim q_t$ are the realized action of the respective player. This is essentially a bandit feedback setting, which we will discuss in the second half of the course.

A very natural idea for the player is simply to run an expert algorithm such as Hedge, treating each available action i as an expert. Specifically, given an expert algorithm as a blackbox, p_t will be the output of this algorithm at round t , and the loss vector to be fed to the algorithm is ℓ_t such that $\ell_t(i) = G(i, q_t)$, $\forall i$. Let $\mathcal{R}_T$ be the regret of this algorithm and $\bar{q} = \frac{1}{T} \sum_{t=1}^T q_t$. Then we have

$$\begin{aligned} \frac{1}{T} \sum_{t=1}^T G(p_t, q_t) &= \min_p \frac{1}{T} \sum_{t=1}^T G(p, q_t) + \frac{\mathcal{R}_T}{T} \\ &= \min_p G(p, \bar{q}) + \frac{\mathcal{R}_T}{T} \\ &\leq \max_q \min_p G(p, q) + \frac{\mathcal{R}_T}{T}. \end{aligned} \tag{1}$$

Therefore, if $\mathcal{R}_T = o(T)$ and T is large enough, the average loss of the row player is very close to the value of the game, which again is the smallest possible loss if against an optimal opponent. However, by using a learning algorithm instead of a minimax solution directly (if it is available), the average loss can also be much smaller in the case when the opponent is not exactly optimal (that is, when $\bar{q}$ is not close to the maximin strategy and the last inequality is loose).

When both players learn. Even more interesting thing happens if the column player also uses an expert algorithm to come up with q_t (by treating the negative rewards as losses). To see this, let $\mathcal{R}'_T = o(T)$ be the regret of the column player:

$$\mathcal{R}'_T = \sum_{t=1}^T -G(p_t, q_t) - \min_q \sum_{t=1}^T -G(p_t, q) = \max_q \sum_{t=1}^T G(p_t, q) - \sum_{t=1}^T G(p_t, q_t).$$

Then we can repeat the earlier calculation, but now for the column player with $\bar{p} = \frac{1}{T} \sum_{t=1}^T p_t$:

$$\begin{aligned} \frac{1}{T} \sum_{t=1}^T G(p_t, q_t) &= \max_q \frac{1}{T} \sum_{t=1}^T G(p_t, q) - \frac{\mathcal{R}'_T}{T} \\ &= \max_q G(\bar{p}, q) - \frac{\mathcal{R}'_T}{T} \\ &\geq \min_p \max_q G(p, q) - \frac{\mathcal{R}'_T}{T}. \end{aligned} \tag{2}$$

Combining the two derivations, we have

$$\min_p \max_q G(p, q) \leq \frac{1}{T} \sum_{t=1}^T G(p_t, q_t) + \frac{\mathcal{R}_T}{T} \leq \max_q \min_p G(p, q) + \frac{\mathcal{R}_T}{T} + \frac{\mathcal{R}'_T}{T}.$$

Since the term $\frac{\mathcal{R}_T}{T} + \frac{\mathcal{R}'_T}{T}$ can be arbitrarily close to 0 as T goes to infinity, we must have

$$\min_p \max_q G(p, q) \leq \max_q \min_p G(p, q),$$

which means that we just proved the minimax theorem (recall that the other direction is trivial)! This is a classical result taken from [Freund and Schapire, 1999]. It is quite intriguing because it says that the existence of a no-regret algorithm implies that the minimax theorem must hold, without using

heavier tools such as a fixed-point theorem. In fact, a closer look at equalities (1) and (2) also tells us $\max_q G(\bar{p}, q) - \min_p G(p, \bar{q}) = \frac{\mathcal{R}_T + \mathcal{R}'_T}{T}$, and thus:

$$\max_q G(\bar{p}, q) - \epsilon = \min_p G(p, \bar{q}) \leq G(\bar{p}, \bar{q}) \leq \max_q G(\bar{p}, q) = \min_p G(p, \bar{q}) + \epsilon$$

for $\epsilon = \frac{\mathcal{R}_T + \mathcal{R}'_T}{T}$ (called social average regret), showing that the empirical average strategies $\bar{p}$ and $\bar{q}$ are *approximately minimax solutions* or *approximate Nash equilibrium* with error ϵ . This provides a highly efficient way to approximately find a Nash equilibrium, making regret minimization algorithms (together with other tricks) one of the most practical ways to solve large-scale games.¹

3 Faster Convergence via Adaptivity

We know that the worst-case optimal regret for the expert problem is of order $\mathcal{O}(\sqrt{T})$ (ignoring dependence on the number of actions), which means the convergence rate of the above approach is of order $\mathcal{O}(1/\sqrt{T})$. However, since each player is not really dealing with a worst-case environment, a natural question is whether we can achieve even faster convergence in this specific context. The answer turns out to be yes, and the key is the optimistic algorithms we discussed last time. Indeed, by setting $m_t = \ell_{t-1}$, we know that optimistic Hedge's regret depends on the path length of the loss sequence $\sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_\infty$, which in this game setting should intuitively depend on the variation of the opponent's strategy and thus should be small due to the stability of these algorithms. In fact, utilizing the negative regret term that we ignored in the past lectures, we can make this argument even stronger, as shown in the following theorem.

Theorem 3. *Suppose that both players apply optimistic Hedge with the predictor being the loss vector of the last round, that is:*

$$p_t(i) \propto \exp \left(-\eta \left(\ell_{t-1}(i) + \sum_{s < t} \ell_s(i) \right) \right), \quad \text{where } \ell_s(i) = G(i, q_s),$$

$$q_t(j) \propto \exp \left(-\eta \left(g_{t-1}(j) + \sum_{s < t} g_s(j) \right) \right), \quad \text{where } g_s(j) = -G(p_s, j).$$

Then with $\eta = \frac{1}{4}$, the total regret of the players is bounded as $\mathcal{R}_T + \mathcal{R}'_T = \mathcal{O}(\ln(NM))$, and thus $(\frac{1}{T} \sum_t p_t, \frac{1}{T} \sum_t q_t)$ is an approximate Nash equilibrium with approximation error $\mathcal{O}(\ln(NM)/T)$.

Proof. Directly applying Theorem 1 of Lecture 3, we have

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_\infty^2 - \frac{1}{4\eta} \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2. \quad (3)$$

Further using the definition of ℓ_t , we bound the path-length term in terms of the stability of the opponent: for any $t > 1$,

$$\|\ell_t - \ell_{t-1}\|_\infty = \max_i |G(i, q_t) - G(i, q_{t-1})| = \max_i |\langle G(i, \cdot), q_t - q_{t-1} \rangle| \leq \|q_t - q_{t-1}\|_1,$$

where we use $G(i, \cdot)$ to denote the i -th row of G . This implies:

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \eta + \eta \sum_{t=2}^T \|q_t - q_{t-1}\|_1^2 - \frac{1}{4\eta} \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2. \quad (4)$$

The exact same argument applies to the column player as well, meaning

$$\mathcal{R}'_T \leq \frac{\ln N}{\eta} + \eta + \eta \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2 - \frac{1}{4\eta} \sum_{t=2}^T \|q_t - q_{t-1}\|_1^2,$$

¹At this point, you should revisit one statement we made in Lecture 1: even though the definition of regret seemingly does not make sense at all for an adaptive adversary, it can still be very meaningful.

and thus the sum is bounded as

$$\begin{aligned}
\mathcal{R}_T + \mathcal{R}'_T &\leq \frac{\ln(NM)}{\eta} + 2\eta + \left(\eta - \frac{1}{4\eta}\right) \sum_{t=2}^T \left(\|p_t - p_{t-1}\|_1^2 + \|q_t - q_{t-1}\|_1^2\right) \\
&= 4\ln(NM) + \frac{1}{2} - \frac{3}{4} \sum_{t=2}^T \left(\|p_t - p_{t-1}\|_1^2 + \|q_t - q_{t-1}\|_1^2\right) \\
&\leq 4\ln(NM) + \frac{1}{2}
\end{aligned} \tag{5}$$

where the equality is by plugging in the value of η (which is $1/4$). This completes the proof. $\square$

Hence, using optimistic Hedge significantly speeds up the convergence rate from $\mathcal{O}(1/\sqrt{T})$ to $\mathcal{O}(1/T)$. In other words, to get an ϵ -approximate Nash equilibrium, we only need $\mathcal{O}(1/\epsilon)$ iterations instead of $\mathcal{O}(1/\epsilon^2)$; for example, for $\epsilon = 0.001$, this is an improvement from one million iterations to just one thousand.

We point out that this result is also quite robust. For example, the learning rate does not have to be exactly $1/4$ nor identical for the two players — it is easy to see from the proof that if the row player uses η and the column player uses η' , then as long as $4\eta\eta' \leq 1$, the regret is at most $\mathcal{O}(\frac{\ln N}{\eta} + \frac{\ln M}{\eta'})$. Moreover, the key of the proof clearly only relies on having an adaptive regret bound of the form (3) and is independent of the details of the algorithm. There are indeed many other algorithms that enjoy a similar bound and thus the same fast convergence rate; see [Syrgkanis et al., 2015] for details.

Another interesting phenomenon is the behavior of the strategy sequence $(p_1, q_1), (p_2, q_2), \dots$. While any no-regret algorithm ensures that the average of these strategies converges, this sequence itself might not converge. For example, the sequence generated by vanilla Hedge could circle around the equilibrium and never gets close to it. On the other hand, the sequence generated by Optimistic Hedge has been proven to converge to the equilibrium in recent years. Such property is often called *last-iterate convergence*, and is another reason why Optimistic Hedge is much more favorable for such problems. Notice that the original goal of the players is not to find the equilibrium, but instead simply to selfishly minimize their own loss by exploiting the weakness of the opponent. However, Nash equilibrium happens to be the natural long-term outcome of this selfish process.

A closer look at the stability. Recall our earlier intuition: in the game setting, the loss path-length $\|\ell_t - \ell_{t-1}\|_\infty^2$ of the row player is bounded by the stability $\|q_t - q_{t-1}\|_1^2$ of the column player, which should be of order η and thus “small”. However, in the end, we are in fact choosing a *constant* learning rate $\eta = 1/4$, which seemingly contradicts with our earlier intuition and shows that the both players could be highly unstable instead. Is this truly how these algorithms behave?

It turns out that our earlier intuition is still correct: the algorithms are highly stable, leading to a slowly changing environment for the opponent. To see this, first notice that the social regret is never negative:

$$\frac{\mathcal{R}_T + \mathcal{R}'_T}{T} = \max_q G(\bar{p}, q) - \min_p G(p, \bar{q}) \geq G(\bar{p}, \bar{q}) - G(\bar{p}, \bar{q}) = 0.$$

Therefore, rearranging Eq. (5) shows

$$\frac{3}{4} \sum_{t=2}^T \left(\|p_t - p_{t-1}\|_1^2 + \|q_t - q_{t-1}\|_1^2\right) \leq 4\ln(NM) + \frac{1}{2},$$

and thus $\sum_{t=2}^T \|p_t - p_{t-1}\|_1^2 = \mathcal{O}(\ln(NM))$ (same for the column player). This shows that the cumulative movement of the player’s strategy, a quantity that could be of order T in the worst case, is in fact completely independent of T . This is also consistent with the earlier point that the sequence $p_1, p_2, \dots$ is converging (though not a formal proof yet).

This small observation has two important consequences. First, the individual regret of each player ($\mathcal{R}_T$ or $\mathcal{R}'_T$) is in fact also of order $\mathcal{O}(\ln(NM))$. Note that this cannot be simply concluded from the fact $\mathcal{R}_T + \mathcal{R}'_T = \mathcal{O}(\ln(NM))$, since regret can be negative. Instead, one can see this by starting from Eq. (4), ignoring the negative term, and using the stability $\sum_{t=2}^T \|q_t - q_{t-1}\|_1^2 = \mathcal{O}(\ln(NM))$ we

just proved. Such an individual constant regret bound incentivizes both players to deploy Optimistic Hedge.

Second, this also provides a way to robustify the algorithm. Indeed, the aforementioned good individual regret bound only holds when both players exactly follow Optimistic Hedge, so what if my opponent deviates to something else to exploit my weakness? To address this, ideally we want our algorithm to also enjoy the worst-case $\mathcal{O}(\sqrt{T})$ bound no matter what the opponent actually ends up doing. With a fixed learning rate $\eta = 1/4$, it is not hard to see that Optimistic Hedge does not enjoy such a robustness guarantee. However, this can be easily addressed by the following modification: start with Optimistic Hedge with $\eta = 1/4$, keep track of the path length $\sum_{s \leq t} \|\ell_s - \ell_{s-1}\|_\infty^2$, and whenever it exceeds $\mathcal{O}(\ln(NM))$, switch to vanilla Hedge (or any other minimax optimal algorithm). This does not ruin our earlier nice $\mathcal{O}(\ln(NM))$ regret bound if both players follow the same algorithm (since the path length cannot exceed $\mathcal{O}(\ln(NM))$ in this case); on the other hand, no matter what the opponent does, our regret is never worse than $\mathcal{O}(\ln(NM) + \sqrt{T \ln N})$, because before our algorithm switches to vanilla Hedge, its regret is at most $\mathcal{O}(\ln(NM))$ based on Eq. (3). This robustness guarantee further incentivizes both parties to deploy this algorithm.

4 General-Sum Games

Next, we consider general-sum games, a much broader type of games where the players might not have exact opposite interest any more. For simplicity, we still focus on the two-player setting, where Player 1 has N actions and Player 2 has M actions. The loss matrices $G_1 \in [0, 1]^{N \times M}$ and $G_2 \in [0, 1]^{N \times M}$ are such that $G_1(i, j)$ and $G_2(i, j)$ are the loss for Player 1 and Player 2 respectively if Player 1 picks action i while Player 2 picks action j . The zero-sum setting is clearly a special case with $G_2 = -G_1$.

A classical example is the “game of chicken”, where two players are driving toward each other and both can either “Dare” (D) or “Chicken” (C): both dare leads to car crash and the worst case loss of 1 for both; if one dares and other other chickens, the former has no loss and the latter has loss 0.5; finally if both chicken, they both get a small loss 0.1. This simple game captures numerous real-life situations (e.g. nuclear arms race between counties). The loss matrices G_1 and G_2 in this game are:

$$G_1 = \begin{matrix} & \begin{matrix} D & C \end{matrix} \\ \begin{matrix} D \\ C \end{matrix} & \begin{pmatrix} 1 & 0 \\ 0.5 & 0.1 \end{pmatrix} \end{matrix}, \quad G_2 = \begin{matrix} & \begin{matrix} D & C \end{matrix} \\ \begin{matrix} D \\ C \end{matrix} & \begin{pmatrix} 1 & 0.5 \\ 0 & 0.1 \end{pmatrix} \end{matrix}.$$

Nash equilibria can be defined in the same way for general-sum games: a pair of mixed strategy p and q is a Nash equilibrium if neither player has incentive to deviate:

$$G_1(p, q) \leq G_1(i, q), \quad \forall i \in [N] \quad \text{and} \quad G_2(p, q) \leq G_2(p, j), \quad \forall j \in [M].$$

However, in this case there is no “minimax” interpretation of Nash equilibria or the corresponding unique game value any more. For example, the game of chicken has three Nash equilibria: two deterministic ones where one player always dares and the other always chickens, and a mixed one where both players dare with probability $1/6$ (verify it yourself). Each of these equilibria leads to different losses for the two players, illustrating very different characteristics of Nash equilibria for general-sum games. In fact, finding Nash equilibria for general-sum games has also been shown to belong to a complexity class called PPAD (believed to be computationally hard), so there is basically no hope that our earlier discussions on the connection between no-regret learning and finding Nash equilibria can be extended to this case.

Because of this hardness result, weaker notions of equilibria have been studied. Here, we only discuss one of them: *coarse correlated equilibria* (CCE), which is a direct generalization of Nash equilibria from product distributions over action pairs to joint distributions. Formally, a joint distribution $\sigma \in \Delta(NM)$ is a CCE if neither player has incentive to deviate assuming that the opponent sticks with σ :

$$\mathbb{E}_{(i,j) \sim \sigma}[G_1(i, j)] \leq G_1(i', \sigma_2), \quad \forall i' \in [N] \quad \text{and} \quad \mathbb{E}_{(i,j) \sim \sigma}[G_2(i, j)] \leq G_2(\sigma_1, j'), \quad \forall j' \in [M],$$

where σ_1 and σ_2 are the marginal distributions over Player 1’s actions and Players 2’s actions respectively. If a CCE σ happens to be a product distribution, that is $\sigma(i, j) = \sigma_1(i)\sigma_2(j)$, then clearly

σ is also a Nash equilibrium by definition. For a general CCE σ that is not a product distribution, one way to interpret it is that if a mediator recommends to both players an action pair drawn from σ , then both players feel rational about accepting the recommendation even without looking at it.

For example, in Rock-Paper-Scissors, a uniform distribution over all the six non-tie action pairs is a CCE. For the game of chicken, picking (C, C) with $5/7$ probability, and (D, C) or (C, D) with probability $1/7$ is a CCE. In fact, this CCE leads to even smaller total losses of the two players compared to all the Nash equilibria. (Verify all these yourself.)

4.1 Finding CCEs via No-regret Learning

While Nash equilibria are hard to find, CCEs can be computed efficiently, and in fact can be done via no-regret learning in an *uncoupled* way again, even though CCE, as a joint distribution, is by definition coupled. The learning setup is the same as before: each time Player 1 (respectively Player 2) independently uses an expert algorithm to come up with an action distributions $p_t \in \Delta(N)$ (respectively $q_t \in \Delta(M)$), and then sees the loss vector $G_1(\cdot, q_t)$ (respectively $G_2(p_t, \cdot)$) to be fed to the expert algorithm. This is uncoupled in the sense that both players are simply maintaining a distribution over their own actions, and more importantly, they only need to see their own losses (for example, Player 1 is completely oblivious about G_2). Nevertheless, it can be shown that their average joint behavior converges to a CCE in the following sense.

Theorem 4. *In the learning setup above, if $\mathcal{R}_T$ and $\mathcal{R}'_T$ are the regret of Player 1 and Player 2 respectively, then the joint distribution $\sigma \in \Delta(NM)$ with $\sigma(i, j) = \frac{1}{T} \sum_{t=1}^T p_t(i)q_t(j)$ (that is, uniform over the empirical mixed strategy pairs) is a $\max\{\mathcal{R}_T/T, \mathcal{R}'_T/T\}$ -approximate CCE.*

Proof. The proof is simply by definition. First observe that the marginal distribution σ_1 and σ_2 are simply $\frac{1}{T} \sum_{t=1}^T p_t$ and $\frac{1}{T} \sum_{t=1}^T q_t$. For Player 1, by definition we have:

$$\begin{aligned} \mathbb{E}_{(i,j) \sim \sigma} [G_1(i, j)] &= \sum_{i,j} \sigma(i, j) G_1(i, j) = \frac{1}{T} \sum_{t=1}^T \sum_{i,j} p_t(i) q_t(j) G_1(i, j) = \frac{1}{T} \sum_{t=1}^T G_1(p_t, q_t) \\ &= \min_p \frac{1}{T} \sum_{t=1}^T G_1(p, q_t) + \frac{\mathcal{R}_T}{T} = \min_p G(p, \sigma_2) + \frac{\mathcal{R}_T}{T} \leq \min_p G(p, \sigma_2) + \max \left\{ \frac{\mathcal{R}_T}{T}, \frac{\mathcal{R}'_T}{T} \right\}, \end{aligned}$$

and similarly for Player 2, we have

$$\mathbb{E}_{(i,j) \sim \sigma} [G_2(i, j)] = \min_q G_2(\sigma_1, q) + \frac{\mathcal{R}'_T}{T} \leq \min_q G_2(\sigma_1, q) + \max \left\{ \frac{\mathcal{R}_T}{T}, \frac{\mathcal{R}'_T}{T} \right\}.$$

Together, this is exactly the definition of σ being a $\max\{\mathcal{R}_T/T, \mathcal{R}'_T/T\}$ -approximate CCE. $\square$

This result again provides a highly efficient and uncoupled way to find an approximate CCE, as a long-term outcome of a selfish process where each player's motivation is simply to minimize their own regret. One can easily verify that the constant social regret bound of Theorem 3 still holds in this case. However, unlike the zero-sum case, the CCE approximation error is now in terms of $\max\{\mathcal{R}_T, \mathcal{R}'_T\}$, not $\mathcal{R}_T + \mathcal{R}'_T$, so once again we care about the individual regret but not the regret sum. This is where things can get much more complicated — indeed, the earlier argument on getting $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = \mathcal{O}(\ln(NM))$ in the zero-sum case crucially relies on the fact $\mathcal{R}_T + \mathcal{R}'_T \geq 0$, which no longer holds for general-sum games!

In fact, getting $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = \mathcal{O}(\ln(NM))$ in this case is still an open problem, but recent research has made significant progress, with the latest breakthrough by [Daskalakis et al., 2021] showing that Optimistic Hedge (when deployed by both players) achieves $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = \mathcal{O}(\ln(NM) \ln^4 T)$. This result again makes use of some adaptivity property of Optimistic Hedge similar to our discussion, but is too involved to be covered here. Instead, in the rest of this lecture, we discuss two simpler results that achieve $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = o(\sqrt{T})$, taken from [Syrkkanis et al., 2015] and [Chen and Peng, 2020] respectively.

The first result again uses the fact that the environment is stable due to the stability of learning algorithms.

Theorem 5. *If both players use Optimistic Hedge with the predictor being the last loss vector:*

$$p_t(i) \propto \exp \left(-\eta \left(\ell_{t-1}(i) + \sum_{s < t} \ell_s(i) \right) \right), \quad \text{where } \ell_s(i) = G_1(i, q_s),$$

$$q_t(j) \propto \exp \left(-\eta \left(g_{t-1}(j) + \sum_{s < t} g_s(j) \right) \right), \quad \text{where } g_s(j) = G_2(p_s, j).$$

then with $\eta = (\ln(NM)/T)^{1/4}$, $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = \mathcal{O}(T^{1/4} \ln^{3/4}(NM))$.

Proof. By Lemma 4 of Lecture 2, we have $\|q_t - q_{t-1}\|_1 \leq \eta \|2g_{t-1} - g_{t-2}\|_\infty \leq 2\eta$, and thus based on Eq (3) and our earlier observation $\|\ell_t - \ell_{t-1}\|_\infty \leq \|q_t - q_{t-1}\|_1$, we have

$$\mathcal{R}_T \leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_\infty^2 \leq \frac{\ln N}{\eta} + \eta + \eta \sum_{t=2}^T \|q_t - q_{t-1}\|_1^2 \leq \frac{\ln N}{\eta} + \eta + 4T\eta^3.$$

The same bound holds for Player 2 (with N replaced by M). Plugging in the (optimal) learning rate value thus proves the theorem. $\square$

With a more careful treatment of the stability, the result above can be improved to the following.

Theorem 6. *For the same algorithm described in Theorem 5, using $\eta = (\ln(NM)/T)^{1/6}$ ensures $\max\{\mathcal{R}_T, \mathcal{R}'_T\} = \mathcal{O}(T^{1/6} \ln^{5/6}(NM))$.*

Proof. Instead of using the final statement of Lemma 4 of Lecture 2, we use the intermediate step (Eq. (3) of Lecture 2): $\|q_t - q_{t-1}\|_1^2 \leq \eta \langle q_t - q_{t-1}, g_{t-2} - 2g_{t-1} \rangle$, which further implies

$$\begin{aligned} \sum_{t=2}^T \|q_t - q_{t-1}\|_1^2 &\leq \eta \sum_{t=2}^T \langle q_t - q_{t-1}, g_{t-2} - 2g_{t-1} \rangle \\ &= \eta \langle q_T, g_{T-2} - 2g_{T-1} \rangle - \eta \langle q_1, g_0 - 2g_1 \rangle + \eta \sum_{t=2}^{T-1} \langle q_t, 2g_t - 3g_{t-1} + g_{t-2} \rangle \quad (\text{rearranging}) \\ &\leq \eta \|g_{T-2} - 2g_{T-1}\|_\infty + \eta \|g_0 - 2g_1\|_\infty + \eta \sum_{t=2}^{T-1} \|2g_t - 3g_{t-1} + g_{t-2}\|_\infty \\ &\leq 4\eta + \eta \sum_{t=2}^{T-1} (2\|g_t - g_{t-1}\|_\infty + \|g_{t-1} - g_{t-2}\|_\infty) \\ &\leq 5\eta + 3\eta \sum_{t=2}^{T-1} \|g_t - g_{t-1}\|_\infty \leq 5\eta + 3\eta \sum_{t=2}^{T-1} \|p_t - p_{t-1}\|_1. \end{aligned}$$

Note that we have moved from the (squared) stability of q_t , to the stability of g_t , and finally back to the stability of p_t . Therefore, starting from the regret bound (3), we now have

$$\begin{aligned} \mathcal{R}_T &\leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \|\ell_t - \ell_{t-1}\|_\infty^2 - \frac{1}{4\eta} \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2 \\ &\leq \frac{\ln N}{\eta} + \eta + \eta \sum_{t=2}^T \|q_t - q_{t-1}\|_1^2 - \frac{1}{4\eta} \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2 \\ &\leq \frac{\ln N}{\eta} + \eta + 5\eta^2 + 3\eta^2 \sum_{t=2}^T \|p_t - p_{t-1}\|_1 - \frac{1}{4\eta} \sum_{t=2}^T \|p_t - p_{t-1}\|_1^2 \\ &\leq \frac{\ln N}{\eta} + \eta + 5\eta^2 + 9T\eta^5, \end{aligned}$$

where the last step is due to $3\eta^2 \|p_t - p_{t-1}\|_1 \leq \frac{1}{4\eta} \|p_t - p_{t-1}\|_1^2 + 9\eta^5$ since $2\sqrt{ab} \leq a + b$. Plugging in the learning rate then finishes the proof (the reasoning for $\mathcal{R}'_T$ is symmetric). $\square$

References

- Xi Chen and Binghui Peng. Hedging in games: Faster convergence of external and swap regrets. *Advances in Neural Information Processing Systems*, 33:18990–18999, 2020.
- Constantinos Daskalakis, Maxwell Fishelson, and Noah Golowich. Near-optimal no-regret learning in general games. *Advances in Neural Information Processing Systems*, 34:27604–27616, 2021.
- Yoav Freund and Robert E Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29(1-2):79–103, 1999.
- Vasilis Syrgkanis, Alekh Agarwal, Haipeng Luo, and Robert E Schapire. Fast convergence of regularized learning in games. In *Advances in Neural Information Processing Systems* 28, 2015.

CSCI 659 Lecture 5

Fall 2022

Instructor: Haipeng Luo

1 Stronger Regret Measures for Non-Stationary Environments

When introducing the classical static regret definition in Lecture 1, we mentioned that competing with a fixed optimal action in hindsight might not always make sense, especially when the environment is non-stationary such that there is no single fixed action that performs well overall. To see this, consider a simple expert problem with two experts where the first one always suffers loss 0 for the first $T/2$ rounds and loss 1 for the remaining $T/2$ rounds, and the situation for the second expert is exactly reversed. Then, either expert is the overall best one, but with a huge total loss of $T/2$. In this case, even if we have zero regret against the best fixed expert, all we can say is that the loss of our algorithm is bounded by $T/2$, a very disappointing guarantee.

Moreover, this might not be just due to loose regret analysis, and one can show that some algorithms with sublinear regret guarantees indeed suffer linear loss $\Omega(T)$ in this instance. Take Hedge as an example. Observe that by the algorithm's definition, the weight for the first expert is always not smaller than the second one (since its cumulative loss is always not larger). This means that for the second half of the rounds, the loss of the algorithm is at least $1/2$ each round, and thus the cumulative loss of Hedge is at least $T/4$.

How do we address such issues? From an algorithmic perspective, intuitively we need algorithms that can realize the changes in the environment quickly and gradually forget about the outdated data in the past. To guide such algorithm design, we consider the following two stronger regret measures.

Interval regret. For two integers s (starting point) and e (end point) such that $1 \leq s \leq e \leq T$, we use the notation $\mathcal{I} = [s, e]$ to denote the rounds $\{s, s+1, \dots, e-1, e\}$ and call it an interval. The interval regret with respect to an interval $\mathcal{I}$ is then simply defined as the standard regret restricted on this interval:

$$\mathcal{R}_{\mathcal{I}} = \sum_{t \in \mathcal{I}} f_t(w_t) - \min_{w \in \Omega} \sum_{t \in \mathcal{I}} f_t(w).$$

In other words, $\mathcal{R}_{\mathcal{I}}$ compares the loss of the algorithm on interval $\mathcal{I}$ to the loss of the best fixed point on the same interval. If we know where the starting point s of the interval $\mathcal{I}$ is, then we would simply run a standard algorithm starting from round s and obtain $\mathcal{R}_{\mathcal{I}} = \mathcal{O}(\sqrt{|\mathcal{I}|})$ (omitting other dependence), where we use $|\mathcal{I}|$ to denote the length of interval $\mathcal{I}$ (that is, $e - s + 1$). Of course, the challenge is that we do not know what $\mathcal{I}$ is beforehand, or in other words, we want to design an algorithm with interval regret $\mathcal{R}_{\mathcal{I}} = \mathcal{O}(\sqrt{|\mathcal{I}|})$ *simultaneously* for all $\mathcal{I}$. In the literature, such an algorithm is also sometimes called a *strongly adaptive* algorithm. This is clearly a stronger measure compared to the standard static regret where we only care about $\mathcal{I} = [1, T]$.

Going back to the example discussed earlier. If we have a strongly adaptive algorithm, then we can conclude that the interval regret for the first $T/2$ rounds and the remaining $T/2$ rounds are both of order $\mathcal{O}(\sqrt{T})$. Since the best experts on these two intervals (expert 1 and 2 respectively) both have zero cumulative loss on their respective interval, the cumulative loss of such a strongly adaptive algorithm over T rounds is only $\mathcal{O}(\sqrt{T})$, much better than the $\Omega(T)$ loss suffered by Hedge.

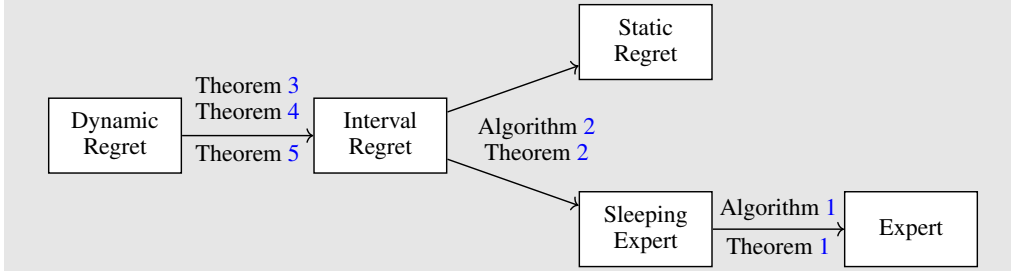
Dynamic regret. While interval regret considers the performance of the algorithm in a local region, dynamic regret measures the global performance of the algorithm over T rounds when compared to a sequence of changing actions. Specifically, the dynamic regret with respect to a comparator sequence $u_1, \dots, u_T \in \Omega$ is defined as

$$\mathcal{R}_T(u_1, \dots, u_T) = \sum_{t=1}^T (f_t(w_t) - f_t(u_t)).$$

When $u_1 = \dots = u_T = u$, this clearly recovers the standard static regret. On the other hand, the most ambitious goal would be to compare with the optimal comparator sequence $w_1^*, \dots, w_T^*$ where $w_t^* = \operatorname{argmin}_{w \in \Omega} f_t(w)$ is the optimal action at time t . However, it is not difficult to see that sublinear dynamic regret against such an optimal comparator sequence is impossible in general. To see this, simply consider a 2-expert problem where the environment always assigns loss 0 to the expert that has the larger weight from the algorithm and loss 1 to the other expert. Then at each round, clearly the best expert has loss 0, while the algorithm suffers loss at least $1/2$, leading to $\Omega(T)$ dynamic regret.

Therefore, the objective here is to express the dynamic regret in terms of a certain “complexity” measure of either the comparator sequence or the loss function sequence, such that the algorithm enjoys sublinear dynamic regret whenever such complexity is not too large. We defer the concrete discussion to Section 4.

A reduction roadmap. We will take a reduction approach to derive algorithms with interval regret or dynamic regret guarantees. Specifically, we will reduce the problem of getting dynamic regret bounds to the problem of getting interval regret bounds, and then further reduce the latter to getting static regret bounds plus solving a so-called *sleeping experts* problem, which is finally reduced to solving the standard expert problem. This reduction chain is illustrated in the following picture. It again demonstrates the fundamental role of the expert problem as well as the static regret (even if its definition seems questionable at first glance).



2 Sleeping Experts

We start with a detour and discuss the sleeping expert problem, a variant of the expert problem introduced by Freund et al. [1997] that is itself interesting and useful. In this variant, each expert comes with different expertise and thus can choose to abstain from providing any advice for a given round if they do not feel confident enough. Formally, at round $t = 1, \dots, T$,

1. the environment first decides (possibly adversarially) which of the N experts are *awake* and which are *asleep*: $a_t(i) = 1$ means expert i is awake and $a_t(i) = 0$ means it is asleep;
2. $a_t \in \{0, 1\}^N$ is then revealed to the learner who needs to decide a distribution $p_t \in \Delta(N)$ with the restriction that no weights are put on asleep experts, that is, $p_t(i) = 0$ if $a_t(i) = 0$;
3. the environment decides and reveals the loss for the awake experts, that is, $\ell_t(i) \in [0, 1]$ for all i such that $a_t(i) = 1$.

The regular expert problem is clearly a special case with $a_t(i) = 1$ for all t and i . Because an expert i is not necessarily involved in every round of the game, the regret against this expert, denoted by $R_T(i)$, is now naturally defined only in terms of those rounds when the expert is awake:

$$R_T(i) = \sum_{t \leq T: a_t(i)=1} \langle p_t - e_i, \ell_t \rangle.$$

Note that while we use the notation $\ell_t \in [0, 1]^N$, some of its coordinates might not be defined since the corresponding experts are asleep for this round. However, this is not really an issue because p_t is required to put zero weight on those coordinates anyway and thus they make no difference to the inner product $\langle p_t, \ell_t \rangle$.

It is natural to ask for a sleeping expert algorithm with $R_T(i) = \mathcal{O}(\sqrt{|\{t : a_t(i) = 1\}| \ln N})$ for all i . Indeed, this is achievable by reducing the sleeping expert problem to the regular expert problem. Specifically, suppose we are given a regular expert algorithm $\mathcal{E}$ with prediction $\hat{p}_t$ on round t . To come up with a prediction p_t for the sleeping expert problem so that $p_t(i) = 0$ for all i with $a_t(i) = 0$, a natural idea is to simply ignore the weights in $\hat{p}_t$ for these asleep experts and renormalize the others, that is, $p_t(i) \propto a_t(i)\hat{p}_t(i)$.

Next, we need to come up with a loss vector as the feedback to $\mathcal{E}$. For the awake experts, it is natural to just use the same loss from the sleeping expert problem. What about the asleep experts? Suppose that we assign the same value x to all these asleep experts. Then with the goal of forcing the loss of $\mathcal{E}$ for this round to be the same as the loss of the sleeping expert algorithm, we arrive at an equation

$$\sum_{i:a_t(i)=1} \hat{p}_t(i)\ell_t(i) + \left(\sum_{i:a_t(i)=0} \hat{p}_t(i) \right) x = \sum_{i:a_t(i)=1} p_t(i)\ell_t(i).$$

Using the definition of p_t and solving for x gives

$$\begin{aligned} x &= \frac{\sum_{i:a_t(i)=1} (p_t(i) - \hat{p}_t(i)) \ell_t(i)}{\sum_{i:a_t(i)=0} \hat{p}_t(i)} = \frac{\left(\frac{1}{\sum_{i:a_t(i)=1} \hat{p}_t(i)} - 1 \right) \sum_{i:a_t(i)=1} \hat{p}_t(i) \ell_t(i)}{1 - \sum_{i:a_t(i)=1} \hat{p}_t(i)} \\ &= \frac{1}{\sum_{i:a_t(i)=1} \hat{p}_t(i)} \sum_{i:a_t(i)=1} \hat{p}_t(i) \ell_t(i) = \sum_{i:a_t(i)=1} p_t(i) \ell_t(i), \end{aligned}$$

showing that the “fake” loss of asleep experts should be exactly the loss of the sleeping expert algorithm! As discussed earlier, the value of $\ell_t(i)$ for $a_t(i) = 0$ does not matter in the sleeping expert problem. We will therefore overload the notation ℓ_t to denote the loss vector for both the regular expert problem and the sleeping expert problem, where $\ell_t(i) = \sum_{j:a_t(j)=1} p_t(j)\ell_t(j) = \langle p_t, \ell_t \rangle$ if $a_t(i) = 0$, and otherwise is the loss revealed by the environment. The complete reduction is shown in the following algorithm, followed by its formal guarantee.

Algorithm 1: Reduction from Sleeping Experts to Regular Experts

Input: a regular expert algorithm $\mathcal{E}$

for $t = 1, \dots, T$ **do**
 let $\hat{p}_t \in \Delta(N)$ be the decision of $\mathcal{E}$ at round t
 observe a_t from the environment
 play p_t such that $p_t(i) \propto a_t(i)\hat{p}_t(i)$
 observe $\ell_t(i)$ for all i such that $a_t(i) = 1$
 set $\ell_t(i) = \langle p_t, \ell_t \rangle$ for all i such that $a_t(i) = 0$
 feed ℓ_t to $\mathcal{E}$

Theorem 1. *If the regular expert algorithm $\mathcal{E}$ enjoys the optimal regret bound $\sum_{t=1}^T \langle \hat{p}_t - e_i, \ell_t \rangle = \mathcal{O}(\sqrt{T \ln N})$ for all i and T , then Algorithm 1 ensures $R_T(i) = \sum_{t:a_t(i)=1} \langle p_t - e_i, \ell_t \rangle = \mathcal{O}(\sqrt{T \ln N})$ for all i and T as well. On the other hand, if $\mathcal{E}$ satisfies an adaptive regret bound*

$$\sum_{t=1}^T \langle \hat{p}_t - e_i, \ell_t \rangle = \mathcal{O} \left(\sqrt{\sum_{t=1}^T \langle \hat{p}_t - e_i, \ell_t \rangle^2 \ln N} \right) \quad (1)$$

for all i and T , then Algorithm 1 ensures for all i and T :

$$R_T(i) = \mathcal{O}(\sqrt{|\{t : a_t(i) = 1\}| \ln N}). \quad (2)$$

Proof. The design of Algorithm 1 ensures $\langle p_t, \ell_t \rangle = \langle \hat{p}_t, \ell_t \rangle$ for all t and also $\langle p_t - e_i, \ell_t \rangle = 0$ for t with $a_t(i) = 0$. Therefore, we can connect the regret in these two problems as:

$$R_T(i) = \sum_{t: a_t(i)=1} \langle p_t - e_i, \ell_t \rangle = \sum_{t=1}^T \langle p_t - e_i, \ell_t \rangle = \sum_{t=1}^T \langle \hat{p}_t - e_i, \ell_t \rangle.$$

The first statement of the theorem thus follows immediately. The second statement is also clear after realizing

$$\sum_{t=1}^T \langle \hat{p}_t - e_i, \ell_t \rangle^2 = \sum_{t=1}^T \langle p_t - e_i, \ell_t \rangle^2 = \sum_{t: a_t(i)=1} \langle p_t - e_i, \ell_t \rangle^2 \leq |\{t : a_t(i) = 1\}|,$$

where the inequality uses the fact $\langle p_t - e_i, \ell_t \rangle \in [-1, 1]$. $\square$

This shows that to obtain the ideal regret guarantee for the sleeping expert problem, we need slightly more than a minimax optimal expert algorithm in the reduction, that is, we need an adaptive regret bound (1). Note that we indeed have such adaptive algorithms — in the last lecture, we discussed one already, and in fact, there are several more in the literature. Recall that we also discussed two interesting consequences of (1) last time: small-loss bound and almost constant regret for a stochastic setting. One can verify that the same consequences also hold for $R_T(i)$ in this sleeping expert problem.

3 From Sleeping Experts to Interval Regret

We are now ready to introduce a general mechanism to derive strongly adaptive algorithms (that is, $\mathcal{R}_{\mathcal{I}} = \mathcal{O}(\sqrt{|\mathcal{I}|})$ or at least $\mathcal{R}_{\mathcal{I}} = \tilde{\mathcal{O}}(\sqrt{|\mathcal{I}|})$ for any interval $\mathcal{I}$ in the OCO setting,¹ given any OCO algorithm $\mathcal{A}$ with static regret $\mathcal{O}(\sqrt{T})$ for all T . As mentioned earlier, if the interval $\mathcal{I} = [s, e]$ was known, or in fact only the starting point s was known, then one could simply run $\mathcal{A}$ starting from round s . Since we now want to consider all different starting points, a natural idea is to start a new instance of $\mathcal{A}$ at the beginning of every round, and to combine the actions selected by these different instances to come up with the final action.

This idea can be exactly captured by the sleeping expert problem: each instance of $\mathcal{A}$ is an expert, and the instance that starts at time t (denoted by $\mathcal{A}^t$) is asleep for the first $t - 1$ rounds and awake for the rest of the problem. At time t , all the t awake instances $\mathcal{A}^1, \dots, \mathcal{A}^t$ propose their action, denoted by $w_t^1, w_t^2, \dots, w_t^t$ respectively. Then, we use a sleeping expert algorithm to come up with a distribution p_t over these awake experts, and finally select the action that is the average of $w_t^1, w_t^2, \dots, w_t^t$ weighted by $p_t(1), \dots, p_t(t)$, that is, their convex combination with respect to p_t (one can also randomly follow one of these actions according to p_t). At the end of round t , the loss for expert $\mathcal{A}^i$ ($i \leq t$) is naturally $f_t(w_t^i)$, the loss of the action proposed by $\mathcal{A}^i$. The full reduction is summarized below, followed by its formal guarantee.

Algorithm 2: Strongly Adaptive Algorithm via Sleeping Expert

Input: an OCO algorithm $\mathcal{A}$ with a static regret guarantee, a sleeping expert algorithm $\mathcal{S}$

for $t = 1, \dots, T$ **do**

start a new instance of $\mathcal{A}$, called $\mathcal{A}^t$
 obtain actions $w_t^1, \dots, w_t^t \in \Omega$ proposed by $\mathcal{A}^1, \dots, \mathcal{A}^t$ respectively
 feed a_t to $\mathcal{S}$ where $a_t(i) = 1$ for $i \leq t$ and $a_t(i) = 0$ for $i > t$
 obtain distribution p_t from $\mathcal{S}$
 select final action $w_t = \sum_{i=1}^t p_t(i) w_t^i$
 observe loss function f_t and suffer loss $f_t(w_t)$
 feed f_t to $\mathcal{A}^1, \dots, \mathcal{A}^t$
 feed ℓ_t to $\mathcal{S}$ where $\ell_t(i) = f_t(w_t^i)$ for $i \leq t$.

¹Recall that the notation $\tilde{\mathcal{O}}(\cdot)$ hides all the factors that are logarithmic or polylogarithmic in T .

Theorem 2. *If the OCO algorithm $\mathcal{A}$ ensures $\mathcal{R}_T = \mathcal{O}(\sqrt{T})$ for all T and the sleeping expert algorithm $\mathcal{S}$ ensures Eq. (2), then Algorithm 2 ensures $\mathcal{R}_{\mathcal{I}} = \mathcal{O}(\sqrt{|\mathcal{I}| \ln T})$ for all interval $\mathcal{I}$.*

Proof. By the construction, we have for any $w \in \Omega$ and interval $\mathcal{I} = [s, e]$,

$$\begin{aligned} \sum_{t \in \mathcal{I}} (f_t(w_t) - f_t(w)) &\leq \sum_{t \in \mathcal{I}} \left(\sum_{i=1}^t p_t(i) f_t(w_t^i) - f_t(w) \right) && \text{(Jensen's inequality)} \\ &= \sum_{t \in \mathcal{I}} (\langle p_t, \ell_t \rangle - \ell_t(s)) + \sum_{t \in \mathcal{I}} (f_t(w_t^s) - f_t(w)) \\ &= R_e(s) + \mathcal{O}(\sqrt{|\mathcal{I}|}) && \text{(by the guarantee of } \mathcal{A}^s) \\ &= \mathcal{O}(\sqrt{|\mathcal{I}| \ln T}) + \mathcal{O}(\sqrt{|\mathcal{I}|}), && \text{(by the guarantee of } \mathcal{S}) \end{aligned}$$

completing the proof. $\square$

We have thus shown that a strongly adaptive algorithm can be constructed via a standard algorithm and another sleeping expert algorithm, with only a small price of an additional $\sqrt{\ln T}$ factor in the regret (that is known to be necessary). Note that the improved sleeping expert guarantee Eq. (2) plays a key role here; indeed, if one only has the weaker guarantee stated in the first part of Theorem 1, then Algorithm 2 only guarantees $\mathcal{R}_{\mathcal{I}} = \mathcal{O}(\sqrt{T \ln T})$ for all $\mathcal{I}$ (known as *weakly adaptive*). Also, the logarithmic dependence on the number of experts is clearly critical here as well, since the total number of experts is T in this case.

Computational efficiency. While for regret we only pay logarithmic dependence on the number of experts, the running time of $\mathcal{S}$ does still depend linearly on this number, an issue that is fundamental for the expert problem as we discussed before. Although the number of experts is only T (instead of something exponentially large as in the combinatorial example discussed in Lecture 2), this still makes the algorithm much less practical and also destroys the efficiency of most OCO algorithms whose update time per round is independent of T .

Fortunately, it turns out that we can improve the running time to $\mathcal{O}(\ln T)$ per round, without sacrificing anything for the regret. The idea is to kill some instances of $\mathcal{A}$ when they have lived long enough in some sense so that at each time there are only $\mathcal{O}(\ln T)$ instances alive. Killing an instance can be easily incorporated into the sleeping expert model by just putting the corresponding expert to sleep forever. The key is to do this in a careful way so that the regret is almost not affected, and there are in fact several different ways to do so; see for example [Hazan and Seshadhri, 2007].

4 From Interval Regret to Dynamic Regret

Finally, we discuss how an interval regret guarantee automatically implies several different dynamic regret guarantees. Recall that for dynamic regret $\mathcal{R}_T(u_1, \dots, u_T) = \sum_{t=1}^T (f_t(w_t) - f_t(u_t))$, our goal is to express it in terms of some complexity measure of either the comparator sequence $u_1, \dots, u_T$, or the loss function sequence $f_1, \dots, f_T$. We discuss three such examples below.

Switching/tracking regret. The simplest case is to measure the complexity of $u_1, \dots, u_T$ by how many times it switches from one action to another, in which case dynamic regret is also called switching regret or tracking regret. Formally, let $S \geq 1$ be such that $\sum_{t=2}^T \mathbf{1}\{u_t \neq u_{t-1}\} = S - 1$, that is, the number of switches plus one. In other words, we allow the benchmark to divide the total T rounds into S disjoint intervals, and to select any fixed action on each interval. Based on this interpretation, we immediately obtain the following result.

Theorem 3. *If an algorithm ensures $\mathcal{R}_{\mathcal{I}} = \tilde{\mathcal{O}}(\sqrt{|\mathcal{I}|})$ for all interval $\mathcal{I}$, then it also ensures $\mathcal{R}_T(u_1, \dots, u_T) = \tilde{\mathcal{O}}(\sqrt{ST})$ for any $S \in [T]$ and any comparator sequence with $S - 1$ switches.*

Proof. Such a comparator sequence naturally divides the T rounds into S intervals $\mathcal{I}_1, \dots, \mathcal{I}_S$ such that the comparator stays the same on each interval. Then, we simply apply the interval regret

guarantee on each interval:

$$\begin{aligned}\mathcal{R}_T(u_1, \dots, u_T) &= \sum_{m=1}^S \sum_{t \in \mathcal{I}_m} (f_t(w_t) - f_t(u_t)) = \tilde{\mathcal{O}} \left(\sum_{m=1}^S \sqrt{|\mathcal{I}_m|} \right) \\ &\leq \tilde{\mathcal{O}} \left(\sqrt{S \sum_{m=1}^S |\mathcal{I}_m|} \right) = \tilde{\mathcal{O}}(\sqrt{ST})\end{aligned}$$

where the inequality is by Cauchy-Schwarz inequality. $\square$

Therefore, as long as S is sublinear in T , the switching regret is also sublinear. Such a low-switching benchmark can be significantly better than a no-switching benchmark — just consider the 2-expert example discussed in Section 1 again, where one switch already leads to $\Omega(T)$ difference in the total loss of the benchmark. Generally, switching regret is suitable for environments that are close to being piecewise stationary, and there are indeed many such real-life examples. For instance, in the context of a recommendation system, users' preferences usually exhibit a certain stationary trend within a period (say a month or a season).

Question 1. *If the interval regret guarantee is instead $\mathcal{R}_{\mathcal{I}} = \tilde{\mathcal{O}}(\sqrt{T})$ for all interval $\mathcal{I}$, what would be the corresponding switching regret guarantee?*

Variation of the loss sequence. In the second example, we allow the competitor sequence to be arbitrary (so we might just as well pick $u_t = w_t^* = \min_w f_t(w)$, leading to the strongest benchmark), but measure the complexity using the variation of the loss function sequence $f_1, \dots, f_T$, defined as

$$V_T = \sum_{t=2}^T \max_{w \in \Omega} |f_t(w) - f_{t-1}(w)|,$$

that is, the cumulative differences between two consecutive loss functions (measured by their largest difference at the same point). A small variation implies that the environment is slowly drifting over time, making learning possible even if we want to compete with the best action at each round. This measure is quite similar to the path length of the loss sequence discussed in the last two lectures: previously, our goal was to derive a static regret bound in terms of the path length, with the hope of getting an $o(\sqrt{T})$ bound whenever the environment is slowly changing, and now our goal is to derive a dynamic regret bound in terms of the variation, with the goal of getting an $o(T)$ bound whenever the environment is slowly changing.

Interestingly, this problem is again already solved by using a strongly adaptive algorithm.

Theorem 4. *If an algorithm ensures $\mathcal{R}_{\mathcal{I}} = \tilde{\mathcal{O}}(\sqrt{|\mathcal{I}|})$ for all interval $\mathcal{I}$, then it also ensures*

$$\mathcal{R}_T(w_1^*, \dots, w_T^*) = \tilde{\mathcal{O}} \left(\sqrt{T} + T^{2/3} V_T^{1/3} \right)$$

where $w_t^* = \arg\min_{w \in \Omega} f_t(w)$.

Proof. Let $\mathcal{I}_1 = [s_1, e_1], \dots, \mathcal{I}_M = [s_M, e_M]$ be any partition of the entire T rounds, and

$$V_{\mathcal{I}_m} = \sum_{t=s_m+1}^{e_m} \max_{w \in \Omega} |f_t(w) - f_{t-1}(w)|$$

be the variation of interval $\mathcal{I}_m$. For any $m \in [M]$, we can bound the regret on this interval as:

$$\begin{aligned}\sum_{t \in \mathcal{I}_m} (f_t(w_t) - f_t(w_t^*)) &= \sum_{t \in \mathcal{I}_m} (f_t(w_t) - f_t(w_{s_m}^*)) + \sum_{t \in \mathcal{I}_m} (f_t(w_{s_m}^*) - f_t(w_t^*)) \\ &\leq \tilde{\mathcal{O}}(\sqrt{|\mathcal{I}_m|}) + 2|\mathcal{I}_m|V_{\mathcal{I}_m},\end{aligned}$$

where the last step is by the interval regret guarantee of the algorithm and the fact

$$f_t(w_{s_m}^*) - f_t(w_t^*) \leq f_t(w_{s_m}^*) - f_{s_m}(w_{s_m}^*) + f_{s_m}(w_{s_m}^*) - f_t(w_t^*) \quad (\text{by optimality of } w_{s_m}^*)$$

$$\begin{aligned}
&= \sum_{\tau=s_m+1}^t (f_\tau(w_{s_m}^*) - f_{\tau-1}(w_{s_m}^*)) + \sum_{\tau=s_m+1}^t (f_{\tau-1}(w_t^*) - f_\tau(w_t^*)) \\
&\leq 2 \sum_{\tau=s_m+1}^t \max_{w \in \Omega} |f_\tau(w) - f_{\tau-1}(w)| \leq 2V_{\mathcal{I}_m}.
\end{aligned}$$

Therefore, the overall dynamic regret is bounded by

$$\begin{aligned}
\mathcal{R}_T(w_1^*, \dots, w_T^*) &\leq \sum_{m=1}^M \tilde{\mathcal{O}}(\sqrt{|\mathcal{I}_m|}) + 2|\mathcal{I}_m|V_{\mathcal{I}_m} \\
&\leq \tilde{\mathcal{O}}(\sqrt{MT}) + 2 \max_m |\mathcal{I}_m| \sum_{m=1}^M V_{\mathcal{I}_m} \quad (\text{Cauchy-Schwarz inequality}) \\
&\leq \tilde{\mathcal{O}}(\sqrt{MT}) + 2 \max_m |\mathcal{I}_m| V_T.
\end{aligned}$$

Finally we just need to balance M and $\max_m |\mathcal{I}_m|$. For a fixed M , it is clear that if we want to minimize $\max_m |\mathcal{I}_m|$, we should divide the entire T rounds (almost) evenly into M intervals so that $\max_m |\mathcal{I}_m| = \mathcal{O}(T/M)$ and $\mathcal{R}_T(w_1^*, \dots, w_T^*) = \tilde{\mathcal{O}}(\sqrt{MT}) + TV_T/M$. Setting M (optimally) to $\max\{1, \lfloor T^{1/3} V_T^{2/3} \rfloor\}$ finishes the proof (note that M only appears in the analysis). $\square$

Question 2. Once again, if the interval regret guarantee is instead $\mathcal{R}_{\mathcal{I}} = \tilde{\mathcal{O}}(\sqrt{T})$ for all interval $\mathcal{I}$, what would be the corresponding dynamic regret guarantee?

Therefore, as long as V_T is sublinear, the dynamic regret is sublinear. When V_T is linear, the bound is vacuous, but this is consistent with our earlier discussion that it is impossible to always enjoy $o(T)$ dynamic regret. We make two more remarks on this dynamic regret bound:

First, notice that if f_t is completely revealed at the end of each round, then there is in fact a trivial algorithm with a better dynamic regret bound: simply pick w_t as w_{t-1}^* , that is, best respond to the most recently observed loss function. Its dynamic regret is:

$$\sum_{t=1}^T (f_t(w_{t-1}^*) - f_t(w_t^*)) \leq \sum_{t=1}^T (f_t(w_{t-1}^*) - f_{t-1}(w_{t-1}^*) + f_{t-1}(w_t^*) - f_t(w_t^*)) \leq 2V_T,$$

which is even better than $\tilde{\mathcal{O}}(T^{2/3}V_T^{1/3})$. However, this naive approach heavily relies on the fact that f_t is completely revealed, while the algorithms we have been discussing only need the gradient $\nabla f_t(w_t)$ (see Lecture 2). In fact, even if we only observe a noisy version of $\nabla f_t(w_t)$ with a bounded variance, these algorithms still work as well, while it is easy to see that a small amount of noise can completely destroy the naive approach. Indeed, with the presence of noise, the bound stated in Theorem 4 is worst-case optimal as shown in [Besbes et al., 2015]. Even ignoring this issue, the naive approach is also bad in the sense that it has no static regret guarantee at all.

Second, the switching regret bound from Theorem 3 and the dynamic regret bound from Theorem 4 are generally not comparable. However, the key is that a strongly adaptive algorithm *simultaneously* enjoys both bounds. In fact, a simple rewriting of these results shows that a strongly adaptive algorithm makes sure that its total loss $\sum_{t=1}^T f_t(w_t)$ is at most

$$\min \left\{ \sum_{t=1}^T \min_{w \in \Omega} f_t(w) + \tilde{\mathcal{O}}\left(\sqrt{T} + T^{2/3}V_T^{1/3}\right), \min_S \min_{\substack{u_1, \dots, u_T \\ \text{w. } S-1 \text{ switches}}} \sum_{t=1}^T f_t(u_t) + \tilde{\mathcal{O}}\left(\sqrt{ST}\right) \right\}.$$

Variation of the comparator sequence. Switching regret measures the complexity of the comparator sequence by counting the number of hard switches, so even a tiny change between u_t and u_{t-1} contributes the same amount of complexity as a bigger change, which does not sound very reasonable. Ideally, we want to measure the complexity in a smoother way, such as by $\sum_{t=1}^T \|u_t - u_{t-1}\|$, which clearly recovers and generalizes the switching regret. This is indeed possible via a strongly adaptive algorithm again. For simplicity, in what follows we only focus on the expert problem (a special of OCO) and we define $u_0 = \mathbf{0}$ (all-zero vector) for convenience.

Theorem 5. For the expert problem, if an algorithm ensures $\mathcal{R}_{\mathcal{I}} = \tilde{O}(\sqrt{|\mathcal{I}|})$ for all interval $\mathcal{I}$, then it also ensures $\mathcal{R}_T(u_1, \dots, u_T) = \tilde{O}(\sqrt{TA_T})$ for any $u_1, \dots, u_T \in \Delta(N)$ where $A_T = \sum_{t=1}^T \sum_{i=1}^N [u_t(i) - u_{t-1}(i)]_+ = \frac{1}{2} \sum_{t=1}^T \|u_t - u_{t-1}\|_1$ and $[\cdot]_+$ is a shorthand for $\max\{\cdot, 0\}$.

Proof. The dynamic regret $\mathcal{R}_T(u_1, \dots, u_T)$ can be written as $\sum_{t=1}^T \sum_{i=1}^N u_t(i) r_t(i)$ where $r_t(i) = \langle p_t - e_i, \ell_t \rangle$ is the instantaneous regret. In the rest of the proof, we will fix an expert i and prove $\sum_{t=1}^T u_t(i) r_t(i) = \tilde{O}\left(\sqrt{\sum_{t=1}^T [u_t(i) - u_{t-1}(i)]_+} \sqrt{\sum_{t=1}^T u_t(i)}\right)$, which then completes the proof after summing over i and applying Cauchy-Schwarz inequality.

The idea is to rewrite $\sum_{t=1}^T u_t(i) r_t(i)$ as a weighted sum of interval regret $\sum_{m=1}^M \alpha_m \mathcal{R}_{\mathcal{I}_m}$, for some intervals $\mathcal{I}_1, \dots, \mathcal{I}_M$ (not necessarily a partition of $[1, T]$) and some corresponding weights $\alpha_1, \dots, \alpha_M > 0$. This rewriting holds as long as $u_t(i) = \sum_{m=1}^M \mathbf{1}\{t \in \mathcal{I}_m\} \alpha_m$ for any t , that is, each $u_t(i)$ can be decomposed as the sum of the weights of the intervals that cover t , because

$$\sum_{t=1}^T u_t(i) r_t(i) = \sum_{t=1}^T \sum_{m=1}^M \mathbf{1}\{t \in \mathcal{I}_m\} \alpha_m r_t(i) = \sum_{m=1}^M \alpha_m \sum_{t=1}^T \mathbf{1}\{t \in \mathcal{I}_m\} r_t(i) \leq \sum_{m=1}^M \alpha_m \mathcal{R}_{\mathcal{I}_m}.$$

Applying the interval regret guarantee, we then have

$$\begin{aligned} \sum_{t=1}^T u_t(i) r_t(i) &= \tilde{O}\left(\sum_{m=1}^M \alpha_m \sqrt{|\mathcal{I}_m|}\right) \leq \tilde{O}\left(\sqrt{\sum_{m=1}^M \alpha_m} \sqrt{\sum_{m=1}^M \alpha_m |\mathcal{I}_m|}\right) \quad (\text{Cauchy-Schwarz}) \\ &= \tilde{O}\left(\sqrt{\sum_{m=1}^M \alpha_m} \sqrt{\sum_{m=1}^M \alpha_m \sum_{t=1}^T \mathbf{1}\{t \in \mathcal{I}_m\}}\right) = \tilde{O}\left(\sqrt{\sum_{m=1}^M \alpha_m} \sqrt{\sum_{t=1}^T u_t(i)}\right). \end{aligned}$$

It thus remains to find α_m and $\mathcal{I}_m$ to minimize $\sum_{m=1}^M \alpha_m$ subject to the constraint $u_t(i) = \sum_{m=1}^M \mathbf{1}\{t \in \mathcal{I}_m\} \alpha_m$. The optimal construction is described below. We will skip its optimality proof here (see [Luo and Schapire, 2015] if interested), but it suffices to prove that this construction indeeds leads to $\sum_{m=1}^M \alpha_m = \sum_{t=1}^T [u_t(i) - u_{t-1}(i)]_+$.

For conciseness, we drop the index i in what follows. The construction is recursive. First, we find $t^* \in \arg\min_t u_t$ and create an interval $\mathcal{I}_1 = [1, T]$ with weight $\alpha_1 = u_{t^*}$. Then we recursively perform the same construction for the inputs $u_1 - u_{t^*}, \dots, u_{t^*-1} - u_{t^*}$ and $u_{t^*+1} - u_{t^*}, \dots, u_T - u_{t^*}$ respectively until there are no non-zero inputs left. Let $h(u_1, \dots, u_T)$ denote the sum of the weights of this construction. We now use an induction (on the length of the input T) to prove $h(u_1, \dots, u_T) = \sum_{t=1}^T [u_t - u_{t-1}]_+ = u_1 + \sum_{t=2}^T [u_t - u_{t-1}]_+$. The base case $T = 1$ holds trivially. Suppose that the statement holds for any input length smaller than T . Then we have

$$\begin{aligned} h(u_1, \dots, u_T) &= u_{t^*} + h(u_1 - u_{t^*}, \dots, u_{t^*-1} - u_{t^*}) + h(u_{t^*+1} - u_{t^*}, \dots, u_T - u_{t^*}) \\ &= u_{t^*} + (u_1 - u_{t^*}) + \sum_{t=2}^{t^*-1} [u_t - u_{t-1}]_+ + (u_{t^*+1} - u_{t^*}) + \sum_{t=t^*+2}^T [u_t - u_{t-1}]_+ \\ &= u_1 + \sum_{t=2}^{t^*-1} [u_t - u_{t-1}]_+ + [u_{t^*+1} - u_{t^*}]_+ + \sum_{t=t^*+2}^T [u_t - u_{t-1}]_+ = \sum_{t=1}^T [u_t - u_{t-1}]_+. \end{aligned}$$

where the last step is by $[u_{t^*} - u_{t^*-1}]_+ = 0$. This finishes the proof. $\square$

Once again, as long as A_T is sublinear, the dynamic regret is sublinear. Note that this bound is also not directly comparable to the one in Theorem 4, and a strongly adaptive algorithm enjoys both of them simultaneously, leading to the following upper bound on the total loss of the algorithm:

$$\min \left\{ \sum_{t=1}^T \min_{i \in [N]} \ell_t(i) + \tilde{O}\left(\sqrt{T} + T^{2/3} V_T^{1/3}\right), \min_{u_1, \dots, u_T \in \Delta(N)} \sum_{t=1}^T \langle u_t, \ell_t \rangle + \tilde{O}\left(\sqrt{TA_T}\right) \right\},$$

where V_T in this case simplifies to $\sum_{t=2}^T \|\ell_t - \ell_{t-1}\|_\infty$.

References

- Omar Besbes, Yonatan Gur, and Assaf Zeevi. Non-stationary stochastic optimization. *Operations Research*, 63(5):1227–1244, 2015.
- Yoav Freund, Robert E Schapire, Yoram Singer, and Manfred K Warmuth. Using and combining predictors that specialize. In *29th Annual ACM Symposium on the Theory of Computing*, pages 334–343. ACM, 1997.
- Elad Hazan and C. Seshadhri. Adaptive algorithms for online decision problems. In *Electronic Colloquium on Computational Complexity (ECCC)*, volume 14, 2007.
- Haipeng Luo and Robert E. Schapire. Achieving All with No Parameters: AdaNormalHedge. In *28th Annual Conference on Learning Theory*, 2015.

CSCI 659 Lecture 6

Fall 2022

Instructor: Haipeng Luo

1 The Adversarial Multi-Armed Bandit Problem

All the topics we have discussed so far consider problems with full information feedback. Starting from this lecture, we will move on to the more challenging settings with partial information feedback. The classical example of such problems is the *Multi-Armed Bandit* (MAB) problem [Lai and Robbins, 1985], and in this lecture we start with the adversarial version of MAB introduced in [Auer et al., 2002], which can be seen as a variant of the expert problem.

More specifically, the problem models the situation where a gambler sequentially pulls the arm of one of the slot machines in a casino, with the hope of maximizing reward. A slot machine is sometimes called a “one-armed bandit”, and hence the name multi-armed bandit for this problem. Formally, there are K arms/actions available for a learner, and at each time $t = 1, \dots, T$,

1. the learner picks an action $a_t \in [K]$ while simultaneously the environment decides the loss vector $\ell_t \in [0, 1]^K$,
2. the learner then suffers and observes (only) the loss $\ell_t(a_t)$.

Clearly, this is simply a partial information version of the expert problem, with the difference being that the learner has to actually pick one action at each round and then observes only the loss for this action but not the whole loss vector ℓ_t . For convention, we move from the notation i and N to a and K to denote a specific action and the total number of actions respectively.

For simplicity, we focus on an oblivious adversary and only point out that dealing with an adaptive adversary only requires small modifications. An equivalent way to think about the oblivious case is that the adversary, knowing the learner’s algorithm, decides the sequence of loss vector $\ell_1, \dots, \ell_T$ ahead of time (possibly in a randomized way). For any such a loss sequence, we measure the algorithm’s performance by the expected regret

$$\mathbb{E}[\mathcal{R}_T] = \mathbb{E} \left[\sum_{t=1}^T \ell_t(a_t) \right] - \min_{a \in [K]} \sum_{t=1}^T \ell_t(a),$$

where the expectation is with respect to the randomness of the algorithm.

While simple, MAB captures the essence of many real-life applications such as clinical trials and recommendation systems, where arms correspond to available choices of medicines or products. MAB is also the foundational model to study the well-known *exploitation versus exploration* trade-off, one of the key challenges for all problems with partial information feedback. Indeed, on the one hand, it is tempting to select arms that have suffered small losses in the past (exploitation), but on the other hand, there is also an incentive to select other actions just to find out whether they can lead to even smaller losses (exploration). These two incentives are in conflict when one can only select one action at each round, and thus having a good balance between them is the key to design good algorithms.

1.1 Loss Estimators and Exp3

Since the only difference between MAB and the expert problem is the incomplete information about the loss vector ℓ_t , a natural idea to solve MAB is to construct a loss estimator $\hat{\ell}_t$ and then feed it to any expert algorithm in a blackbox manner. While it might seem impossible to accurately estimate ℓ_t , a vector that can be arbitrarily different from the past loss vectors, by only seeing one of its coordinates, this is indeed doable in a sense via the help of randomness. More specifically, suppose that at time t we pick a_t randomly according to a distribution $p_t \in \Delta(K)$ with a full support, then after seeing $\ell_t(a_t)$, we can construct an *inverse importance weighted* estimator $\hat{\ell}_t \in \mathbb{R}_+^K$ such that

$$\hat{\ell}_t(a) = \frac{\ell_t(a)}{p_t(a)} \mathbf{1}\{a = a_t\} = \begin{cases} \frac{\ell_t(a_t)}{p_t(a_t)} & \text{if } a = a_t, \\ 0 & \text{else.} \end{cases}$$

While written in terms of $\ell_t(a)$ for each a , the indicator $\mathbf{1}\{a = a_t\}$ makes sure that we indeed only use the information $\ell_t(a)$ when we pick this arm and observe its loss. In other words, the estimator is well defined and computable using the information available to the learner.

Unbiasedness The reason that inverse importance weighting makes sense is because it leads to an unbiased estimator.

Lemma 1. *Let $\mathbb{E}_t[\cdot]$ denote the conditional expectation with respect to the random draw of a_t given the past. Then we have for any $a \in [K]$, $\mathbb{E}_t[\hat{\ell}_t(a)] = \ell_t(a)$.*

Proof. This is by direct calculation: $\mathbb{E}_t[\hat{\ell}_t(a)] = (1 - p_t(a)) \times 0 + p_t(a) \frac{\ell_t(a)}{p_t(a)} = \ell_t(a)$. $\square$

As mentioned, with such an unbiased estimator, it is natural to feed it to any expert algorithm. Indeed, the very first MAB algorithm is obtained by feeding this estimator to Hedge, resulting to the following Exp3 algorithm [Auer et al., 2002] (short for Exponential-weight for Exploration and Exploitation): at time t , sample $a_t \sim p_t \in \Delta(K)$ where

$$\forall a \in [K], \quad p_t(a) \propto \exp \left(-\eta \sum_{s < t} \hat{\ell}_s(a) \right) \quad (\text{Exp3})$$

for some learning rate $\eta > 0$.

Before analyzing the regret of Exp3, we first address the following question: where is the aforementioned exploration versus exploitation trade-off in this algorithm? The exploitation part is basically executed by the Hedge algorithm: arms with smaller estimated losses are selected with higher probability. On the other hand, the exploration part is somewhat implicit. Indeed, whenever an arm a_t is selected (maybe due to exploitation), the probability of selecting this arm next time is always decreased (or at least not increased), which will then encourage the algorithm to explore other actions. This is due to the structure of the estimator $\hat{\ell}_t$ so that only the selected action a_t can have non-zero loss, while all the other actions have zero estimated loss.

To better understand the importance of this implicit exploration, consider the case where the losses are negative: $\ell_t \in [-1, 0]^K$ (or equivalently their magnitude corresponds to reward). One can verify that this does not make any difference to Hedge for the expert problem. However, for MAB, Exp3 does not work anymore, since whenever an arm a_t is selected, its probability of being selected next time gets even larger (again due to the structure of $\hat{\ell}_t$). This scheme clearly lacks sufficient exploration and will suffer linear regret in the worst case. Therefore, when using Exp3, it is important that losses are shifted so that they are all nonnegative.

1.2 Variance and Variance Cancellation

Given that the estimators are unbiased, does it mean that the expected regret of Exp3 is simply $\mathcal{O}(\sqrt{T \ln K})$, the same as Hedge for the expert problem? The answer is no — the $\mathcal{O}(\sqrt{T \ln K})$ bound only holds when the losses fed to Hedge is in $[0, 1]$, but in Exp3, the estimated losses fed to Hedge can be huge due to the inverse importance weighting (for example, $\hat{\ell}_t(a)$ can be as large as

$1/p_t(a)$). One may try to fix this by enforcing a lower bound on $p_t(a)$ such that the estimators are not too large, but such methods would lead to $\omega(\sqrt{T})$ regret (you should try it yourself).

One may also wonder why it makes sense to have such large estimated losses given that we know the true losses are always in $[0, 1]$. However, this is intuitively unavoidable since we are estimating an arbitrary vector by only seeing one of its coordinates. In fact, the importance weighted estimators not only have a large magnitude, but also have a large variance (or rather a large second moment):

Lemma 2. Let $\mathbb{E}_t[\cdot]$ denote the conditional expectation with respect to the random draw of a_t given the past. Then we have for any $a \in [K]$, $\mathbb{E}_t[\widehat{\ell}_t(a)^2] = \frac{\ell_t(a)^2}{p_t(a)} \leq \frac{1}{p_t(a)}$.

Proof. This is also by direct calculation: $\mathbb{E}_t[\widehat{\ell}_t(a)] = (1 - p_t(a)) \times 0 + p_t(a) \frac{\ell_t(a)^2}{p_t(a)^2} = \frac{\ell_t(a)^2}{p_t(a)}$. $\square$

So how do we deal with such a large variance/magnitude? Somewhat surprisingly, there is nothing we need to do algorithmically once we realize that Exp3 itself enjoys a certain *variance cancellation* effect, highlighted in the proof below.

Theorem 1. With $\eta = \sqrt{\frac{\ln K}{TK}}$, Exp3 ensures $\mathbb{E}[\mathcal{R}_T] = \mathcal{O}(\sqrt{TK \ln K})$.

Proof. We apply the following regret bound of Hedge proven in Lecture 1 (see Eq. (1) therein): for any $a^* \in [K]$,

$$\sum_{t=1}^T \langle p_t, \widehat{\ell}_t \rangle - \sum_{t=1}^T \widehat{\ell}_t(a^*) \leq \frac{\ln K}{\eta} + \eta \sum_{t=1}^T \sum_{a=1}^K p_t(a) \widehat{\ell}_t(a)^2, \quad (1)$$

which holds as long as $\widehat{\ell}_t(a) \geq 0$ (in fact, as long as $\eta \widehat{\ell}_t(a) \geq -1$). On the other hand, using Lemma 1 and Lemma 2, we have $\mathbb{E}_t[\widehat{\ell}_t(a^*)] = \ell_t(a^*)$,

$$\mathbb{E}_t[\langle p_t, \widehat{\ell}_t \rangle] = \langle p_t, \mathbb{E}_t[\widehat{\ell}_t] \rangle = \langle p_t, \ell_t \rangle = \mathbb{E}_t[\ell_t(a_t)],$$

and

$$\mathbb{E}_t \left[\sum_{a=1}^K p_t(a) \widehat{\ell}_t(a)^2 \right] = \sum_{a=1}^K p_t(a) \frac{\ell_t(a)^2}{p_t(a)} \leq K. \quad (\text{variance cancellation})$$

Therefore, taking expectation on both sides of Eq. (1) leads to $\mathbb{E}[\mathcal{R}_T] \leq \frac{\ln K}{\eta} + TK\eta$, and picking the optimal value of η finishes the proof. $\square$

From this proof, it is clear that the key is in the specific bound $\sum_{t=1}^T \sum_{a=1}^K p_t(a) \widehat{\ell}_t(a)^2$ on the stability term of Hedge, which quite remarkably cancels the potentially large variance of the estimators automatically. Note that this bound does not necessarily hold if the true losses are negative, which is consistent with our earlier intuition that Exp3 with negative losses should not work due to the lack of exploration. Also note that we have derived other types of bound on the stability term for Hedge and its variant, including $\sum_{t=1}^T \|\widehat{\ell}_t\|_\infty^2$ (Section 3.2 of Lecture 2) and $\sum_{t=1}^T \widehat{\ell}_t(a^*)^2$ when competing with a^* (Theorem 3 of Lecture 3), but none of these enjoys the same variance cancellation effect that is critical for MAB (verify this yourself).

Question 1. Can you figure out what breaks in this proof when the adversary is adaptive?

2 Lower Bounds

The regret bound of Exp3 shows that the price of having bandit feedback is an extra $\sqrt{K}$ factor compared to full information setting, which is quite intuitive since each round we only obtain $1/K$ fraction of information. So is this the optimal regret bound? To answer this question, we first show an $\Omega(\sqrt{TK})$ lower bound in this section.

The intuition of the lower bound is rather straightforward. For any fixed algorithm, first imagine running it in a simple world where losses for all arms are generated independently and uniformly from $\{0, 1\}$. There must exist an arm that is selected no more than T/K times by this algorithm.

Now suppose that the adversary secretly modifies the environment so that the loss of this arm follows a Bernoulli distribution with parameter $1/2 - \sqrt{K/T}$, which is not distinguishable from the uniform distribution with only T/K samples based on information theory. Thus, when run in this new environment, the same algorithm should not be aware of this change and will still pick this arm not often enough, say no more than $T/2$ rounds, leading to at least $\frac{T}{2}\sqrt{K/T} = \Omega(\sqrt{TK})$ regret.

The question is how to make this argument formal. In particular, how to formally argue that in the new environment the algorithm's behavior stays roughly the same. As we will see in the proof below, this can in fact be related to the KL divergence between two distributions corresponding to the two environments.

Theorem 2. *For any MAB algorithm $\mathcal{A}$, there exists a fixed sequence of loss vectors such that*

$$\mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] = \Omega(\sqrt{TK})$$

where we use $\mathbb{E}_{\mathcal{A}}[\cdot]$ to denote the expectation with respect to the randomness of $\mathcal{A}$.

Proof. According to the informal argument mentioned earlier, we create two randomized environments $\mathcal{E}$ and $\mathcal{E}'$ in the following way (and use $\mathbb{E}$ and $\mathbb{E}'$ to denote the expectation in these two environments respectively). In $\mathcal{E}$, every loss $\ell_t(a)$ follows independently a Bernoulli distribution with parameter $1/2$, denoted by $\text{Ber}(1/2)$. There must exist $a' \in [K]$ such that $\mathbb{E}[n(a')] \leq \frac{T}{K}$ where $n(a) = \sum_{t=1}^T \mathbf{1}\{a_t = a\}$ is the total number of times a is selected. Then $\mathcal{E}'$ is constructed such that the losses of arm a' follow $\text{Ber}(1/2 - \epsilon)$ independently for some small $\epsilon \leq 1/4$ to be specified later, and every other arm still follows $\text{Ber}(1/2)$ independently.

The rest of the proof argues that $\mathbb{E}'\mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] = \Omega(\sqrt{TK})$, which implies that there exists a *fixed* sequence of loss vectors such that $\mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] = \Omega(\sqrt{TK})$ and concludes the proof. Further note that $\mathbb{E}'\mathbb{E}_{\mathcal{A}}[\mathcal{R}_T] = \mathbb{E}_{\mathcal{A}}\mathbb{E}'[\mathcal{R}_T]$, so it is sufficient to prove that for any *deterministic* algorithm, $\mathbb{E}'[\mathcal{R}_T] = \Omega(\sqrt{TK})$. If we denote the observation of the learner at time t by $\tilde{\ell}_t = \ell_t(a_t)$, then a deterministic algorithm selects a_t via some fixed function of $\tilde{\ell}_{1:t-1}$, a shorthand for the sequence $\tilde{\ell}_1, \dots, \tilde{\ell}_{t-1}$ (note that the information of $a_{1:t-1}$ is redundant since it is determined by $\tilde{\ell}_{1:t-2}$ already).

Clearly, in expectation a' is the best arm in $\mathcal{E}'$ and

$$\mathbb{E}'[\mathcal{R}_T] = \mathbb{E}' \left[\sum_{t=1}^T \ell_t(a_t) - \min_{a \in [K]} \sum_{t=1}^T \ell_t(a) \right] \geq \mathbb{E}' \left[\sum_{t=1}^T \ell_t(a_t) - \sum_{t=1}^T \ell_t(a') \right] = (T - \mathbb{E}'[n(a')])\epsilon.$$

We next show that $\mathbb{E}'[n(a')]$ and $\mathbb{E}[n(a')]$ are close, that is, the number of times a' is selected in environment $\mathcal{E}$ and that in environment $\mathcal{E}'$ are similar (just as in the previous informal argument). Indeed, using $\mathbb{P}$ and $\mathbb{P}'$ to denote the distributions of the observation sequence $\tilde{\ell}_{1:T}$ in $\mathcal{E}$ and $\mathcal{E}'$ respectively, we have

$$\begin{aligned} \mathbb{E}'[n(a')] - \mathbb{E}[n(a')] &= \sum_{\tilde{\ell}_{1:T} \in \{0,1\}^T} n(a') \left(\mathbb{P}'(\tilde{\ell}_{1:T}) - \mathbb{P}(\tilde{\ell}_{1:T}) \right) \\ &\leq T \sum_{\tilde{\ell}_{1:T} \in \{0,1\}^T} \left| \mathbb{P}'(\tilde{\ell}_{1:T}) - \mathbb{P}(\tilde{\ell}_{1:T}) \right| = T \|\mathbb{P}' - \mathbb{P}\|_1 \leq T \sqrt{2\text{KL}(\mathbb{P}' \parallel \mathbb{P})}, \end{aligned}$$

where the last step is by the Pinsker's inequality.¹ To calculate $\text{KL}(\mathbb{P} \parallel \mathbb{P}')$, we apply a handy divergence decomposition lemma (Lemma 3, included after this proof):

$$\begin{aligned} \text{KL}(\mathbb{P} \parallel \mathbb{P}') &= \mathbb{E}[n(a')] \cdot \text{KL}(\text{Ber}(1/2) \parallel \text{Ber}(1/2 - \epsilon)) \\ &= \frac{\mathbb{E}[n(a')]}{2} \left(\ln \frac{1/2}{1/2 + \epsilon} + \ln \frac{1/2}{1/2 - \epsilon} \right) \\ &= \frac{\mathbb{E}[n(a')]}{2} \ln \left(\frac{1}{1 - 4\epsilon^2} \right) \\ &\leq 8\mathbb{E}[n(a')] \epsilon^2, \end{aligned}$$

¹We in fact have proven Pinsker's inequality in Lecture 2 when arguing the strong convexity of negative entropy; see if you can make the connection.

where in the last step we use the fact $\ln\left(\frac{1}{1-x}\right) \leq 4x$ for any $x \leq \frac{1}{2}$. Finally we have shown

$$\mathbb{E}'[n(a')] \leq \mathbb{E}[n(a')] + 4T\epsilon\sqrt{\mathbb{E}[n(a')]} \leq \frac{T}{K} + 4T\epsilon\sqrt{\frac{T}{K}}$$

(recall a' is selected such that $\mathbb{E}[n(a')] \leq T/K$) and thus

$$\mathbb{E}'[\mathcal{R}_T] \geq T \left(1 - \frac{1}{K} - 4\epsilon\sqrt{\frac{T}{K}}\right) \epsilon \geq T \left(\frac{1}{2} - 4\epsilon\sqrt{\frac{T}{K}}\right) \epsilon$$

Setting $\epsilon = \frac{1}{16}\sqrt{\frac{K}{T}}$ (to maximize the lower bound above) shows $\mathbb{E}'[\mathcal{R}_T] = \Omega(\sqrt{TK})$, finishing the proof. $\square$

The following divergence decomposition lemma is very powerful and is used extensively in proving lower bounds.

Lemma 3 (Divergence decomposition). *Let $\mathcal{E}$ and $\mathcal{E}'$ be two stochastic MAB environments where for each $a \in [K]$ the losses of arm a are i.i.d. samples of $\mathcal{P}_a$ and $\mathcal{P}'_a$ respectively. Let $\tilde{\ell}_t = \ell_t(a_t)$ be the observation of a deterministic learner at time t and $\mathbb{P}$ and $\mathbb{P}'$ be the distributions of $\tilde{\ell}_{1:T}$ for environments $\mathcal{E}$ and $\mathcal{E}'$ respectively. Then we have*

$$\text{KL}(\mathbb{P} \parallel \mathbb{P}') = \sum_{a=1}^K \mathbb{E}[n(a)] \text{KL}(\mathcal{P}_a \parallel \mathcal{P}'_a).$$

where $n(a)$ is the total number of times a is selected in $\mathcal{E}$.

Proof. By definition and direct calculation we have (for conciseness, we omit the range in the subscript of a summation; for example, $\sum_{\tilde{\ell}_{1:T} \in \{0,1\}^T}$ is simply written as $\sum_{\tilde{\ell}_{1:T}}$)

$$\begin{aligned} \text{KL}(\mathbb{P} \parallel \mathbb{P}') &= \sum_{\tilde{\ell}_{1:T}} \mathbb{P}(\tilde{\ell}_{1:T}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_{1:T})}{\mathbb{P}'(\tilde{\ell}_{1:T})} \right) = \sum_{\tilde{\ell}_{1:T}} \mathbb{P}(\tilde{\ell}_{1:T}) \ln \left(\frac{\prod_{t=1}^T \mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\prod_{t=1}^T \mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{t=1}^T \sum_{\tilde{\ell}_{1:T}} \mathbb{P}(\tilde{\ell}_{1:T}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{t=1}^T \sum_{\tilde{\ell}_{1:t}} \left(\sum_{\tilde{\ell}_{t+1:T}} \mathbb{P}(\tilde{\ell}_{t+1:T} | \tilde{\ell}_{1:t}) \right) \mathbb{P}(\tilde{\ell}_{1:t}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{t=1}^T \sum_{\tilde{\ell}_{1:t}} \mathbb{P}(\tilde{\ell}_{1:t}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{a=1}^K \sum_{t=1}^T \sum_{\tilde{\ell}_{1:t}: a_t=a} \mathbb{P}(\tilde{\ell}_{1:t}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{a=1}^K \sum_{t=1}^T \sum_{\tilde{\ell}_{1:t-1}: a_t=a} \mathbb{P}(\tilde{\ell}_{1:t-1}) \sum_{\tilde{\ell}_t} \mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1}) \ln \left(\frac{\mathbb{P}(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})}{\mathbb{P}'(\tilde{\ell}_t | \tilde{\ell}_{1:t-1})} \right) \\ &= \sum_{a=1}^K \sum_{t=1}^T \mathbb{P}(a_t = a) \text{KL}(\mathcal{P}_a \parallel \mathcal{P}'_a) = \sum_{a=1}^K \mathbb{E}[n(a)] \text{KL}(\mathcal{P}_a \parallel \mathcal{P}'_a), \end{aligned}$$

which completes the proof. $\square$

3 Minimax Optimal MAB Algorithms

Given the gap between the Exp3 regret upper bound $\mathcal{O}(\sqrt{TK \ln K})$ and the lower bound $\Omega(\sqrt{TK})$ from the last section, it is more than natural to ask whether this gap can be closed and which one is the exact minimax optimal bound. While a gap of $\sqrt{\ln K}$ might seem quite negligible especially given that we already pay for $\sqrt{K}$ in the bound, the effort in understanding and closing this gap in the literature turns out to be highly fruitful (we will see some examples in the future).

The importance of local norms. To see if we can derive a better algorithm to match the lower bound, we take a closer look at the stability term $\sum_{a=1}^K p_t(a) \hat{\ell}_t(a)^2$ in regret bound (1), which as we showed is at most K in expectation. Recall that the $p_t(a)$ factor is critical in canceling the large variance, so wouldn't it be nice if we have even "more" $p_t(a)$ in this bound? For example, if the stability term is instead $\sum_{a=1}^K p_t(a)^{3/2} \hat{\ell}_t(a)^2$, then after taking expectation, this is at most $\sum_{a=1}^K \sqrt{p_t(a)} \leq \sqrt{K}$, better than the original bound K .

If such a bound on the stability term indeed exists, then intuitively we should pay more for the penalty term (too good to be true otherwise). To better understand this trade-off, first note that the term $\sum_{a=1}^K p_t(a) \hat{\ell}_t(a)^2$ for Hedge can in fact be written as a norm of the loss vector: $\|\hat{\ell}_t\|_{\nabla^{-2}\psi(p_t)}^2$, that is, the quadratic norm weighted by the inverse Hessian of ψ at p_t , where ψ is the negative entropy regularizer (verify it yourself). This is often called the *local norm* of the loss vector, since it depends on some local value of the Hessian, in contrast to the $\|\hat{\ell}_t\|_*^2$ bound we proved in Lecture 2 which is in terms of some fixed norm $\|\cdot\|_*$ (and is not good enough for MAB as discussed). Assuming for a moment that such a local-norm bound $\|\hat{\ell}_t\|_{\nabla^{-2}\psi(p_t)}^2$ on the stability term holds generally for FTRL with any regularizer, then we can reverse-engineer the correct regularizer ψ that leads to the aforementioned smaller bound $\sum_{a=1}^K p_t(a)^{3/2} \hat{\ell}_t(a)^2$ — it should be $\psi(p) = -4 \sum_{a=1}^K \sqrt{p(a)}$.

Now that we have figured out the regularizer, it is easy to find out how large the penalty term is: the range of this regularizer is $B_\psi = 4(\sqrt{K} - 1)$ and the penalty term is bounded by B_ψ/η as shown in Lecture 2. Combining everything, the overall regret would be $\mathcal{O}(\frac{\sqrt{K}}{\eta} + \eta T \sqrt{K})$, which, after picking the optimal η , is $\mathcal{O}(\sqrt{TK})$, exactly matching the lower bound!

It remains to understand whether such a local-norm bound always holds for FTRL. Let's start with some encouraging evidence. Recall the (stability+negative) term $\langle p_t - p_{t+1}, \hat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(p_{t+1}, p_t)$ proven in the general regret bound of Lemma 3 in Lecture 2. Also recall that by definition, the Bregman divergence $D_\psi(p_{t+1}, p_t)$ is equal to $\frac{1}{2} \|p_t - p_{t+1}\|_{\nabla^2 \psi(\xi)}^2$ for some ξ between p_t and p_{t+1} . Therefore, applying Hölder's inequality, we have

$$\langle p_t - p_{t+1}, \hat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(p_{t+1}, p_t) \leq \|p_t - p_{t+1}\|_{\nabla^2 \psi(\xi)} \|\hat{\ell}_t\|_{\nabla^{-2} \psi(\xi)} - \frac{1}{2\eta} \|p_t - p_{t+1}\|_{\nabla^2 \psi(\xi)}^2,$$

which is at most $\frac{\eta}{2} \|\hat{\ell}_t\|_{\nabla^{-2} \psi(\xi)}^2$ by the fact $2xy \leq x^2 + y^2$. This is very encouraging since it remains to connect $\|\hat{\ell}_t\|_{\nabla^{-2} \psi(\xi)}^2$ and $\|\hat{\ell}_t\|_{\nabla^{-2} \psi(p_t)}^2$.

Unfortunately, these two quantities can be quite different in general, especially when p_t and p_{t+1} are too far away from each other (which could happen if the loss estimator $\hat{\ell}_t$ is too "large" in some sense). Connecting these two quantities and finally getting a local-norm bound on the stability term is always the central piece of designing and analyzing online learning algorithms with bandit feedback, and we will see many more examples beyond MAB in the future.

Tsallis Entropy. Fortunately, for MAB, such a local-norm bound does hold for a broad family of regularizers, including the one obtained earlier via reverse engineering. Specifically, consider the following family of regularizers called (negative) *Tsallis entropy*, parameterized by $\beta \in (0, 1)$:

$$\psi(p) = \frac{1 - \sum_{a=1}^K p(a)^\beta}{1 - \beta}. \quad (2)$$

When $\beta = 1/2$, this essentially recovers the previously reverse-engineered regularizer (up to some constants that do not affect FTRL). Moreover, when β approaches 1, applying L'Hôpital's rule we

have $\psi(p) \rightarrow \sum_a p(a) \ln p(a)$, exactly recovering the negative Shannon entropy regularizer. Using such a regularizer in FTRL is first proposed by [Audibert and Bubeck, 2010] and later simplified by [Abernethy et al., 2015]. Below, we prove that this algorithm indeed enjoys the desired local-norm bound.

Theorem 3. Consider the following FTRL algorithm

$$p_t = \operatorname{argmin}_{p \in \Delta(K)} \left\langle p, \sum_{s < t} \widehat{\ell}_s \right\rangle + \frac{1}{\eta} \psi(p) \quad (3)$$

where $\eta > 0$ is a learning rate, ψ is the Tsallis entropy defined in Eq. (2) with a parameter $\beta \in (0, 1)$, and $\widehat{\ell}_1, \dots, \widehat{\ell}_T \in \mathbb{R}_+^K$ are arbitrary loss vectors. Then the following holds for any $a^* \in [K]$,

$$\sum_{t=1}^T \left\langle p_t, \widehat{\ell}_t \right\rangle - \sum_{t=1}^T \widehat{\ell}_t(a^*) \leq \frac{B_\psi}{\eta} + \frac{\eta}{2} \sum_{t=1}^T \left\| \widehat{\ell}_t \right\|_{\nabla^{-2}\psi(p_t)}^2 \quad (4)$$

$$= \frac{K^{1-\beta} - 1}{\eta(1-\beta)} + \frac{\eta}{2\beta} \sum_{t=1}^T \sum_{a=1}^K p_t(a)^{2-\beta} \widehat{\ell}_t(a)^2. \quad (5)$$

Proof. Eq. (5) is by direct calculations: $\psi(p)$ is maximized when p concentrates on one action, and minimized when p is uniform, leading to $B_\psi = \max_p \psi(p) - \min_p \psi(p) = \frac{K^{1-\beta}-1}{1-\beta}$; the Hessian of ψ is a diagonal matrix with the a -th diagonal entry being $\beta p(a)^{\beta-2}$.

To prove Eq. (4), as mentioned, with Lemma 3 of Lecture 2 it suffices to show

$$\langle p_t - p_{t+1}, \widehat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(p_{t+1}, p_t) \leq \frac{\eta}{2} \left\| \widehat{\ell}_t \right\|_{\nabla^{-2}\psi(p_t)}^2.$$

To this end, we extend the definition of Tsallis entropy to the entire nonnegative orthant $\mathbb{R}_+^K$ following the same formula (2), and first bound $\langle p_t - p_{t+1}, \widehat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(p_{t+1}, p_t)$ by

$$\max_{q \in \mathbb{R}_+^K} \langle p_t - q, \widehat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(q, p_t).$$

Let q_t a maximizer of the above (which exists as we will show). Then, we repeat the analysis in our earlier discussion:

$$\begin{aligned} & \langle p_t - q_t, \widehat{\ell}_t \rangle - \frac{1}{\eta} D_\psi(q_t, p_t) \\ &= \langle p_t - q_t, \widehat{\ell}_t \rangle - \frac{1}{2\eta} \|q_t - p_t\|_{\nabla\psi^2(\xi)}^2 \quad (\text{for some } \xi \text{ between } p_t \text{ and } q_t) \\ &\leq \|p_t - q_t\|_{\nabla\psi^2(\xi)} \left\| \widehat{\ell}_t \right\|_{\nabla\psi^{-2}(\xi)} - \frac{1}{2\eta} \|q_t - p_t\|_{\nabla\psi^2(\xi)}^2 \quad (\text{H\"older's inequality}) \\ &\leq \frac{\eta}{2} \left\| \widehat{\ell}_t \right\|_{\nabla\psi^{-2}(\xi)}^2 \quad (2xy \leq x^2 + y^2) \\ &\leq \frac{\eta}{2} \left\| \widehat{\ell}_t \right\|_{\nabla\psi^{-2}(p_t)}^2, \end{aligned}$$

where the last step uses the monotonicity of the local norm and the fact $\xi(a) \leq p_t(a)$ for all a . This is true because by the definition of q_t , we have by setting the gradient to zero:

$$\nabla\psi(q_t) = \nabla\psi(p_t) - \eta \widehat{\ell}_t,$$

which is equivalent to

$$\frac{1}{q_t(a)^{1-\beta}} = \frac{1}{p_t(a)^{1-\beta}} + \frac{1-\beta}{\beta} \eta \widehat{\ell}_t(a).$$

Since $\widehat{\ell}_t(a) \geq 0$, we must have $q_t(a) \leq p_t(a)$ for all a , and being a point between p_t and q_t , ξ must also satisfy $\xi(a) \leq p_t(a)$ for all a . This finishes the proof for Eq. (4). $\square$

By L'Hôpital's rule, we have $\lim_{\beta \rightarrow 1} \frac{K^{1-\beta}-1}{1-\beta} = \ln K$ and thus Eq. (5) generalizes our analysis Eq. (1) for Hedge. One might notice that Theorem 2 holds for any nonnegative losses, while Eq. (1) only requires a slightly weaker condition $\eta \widehat{\ell}_t(a) \geq -1$. This can be addressed by a more careful analysis, which you will need to do in HW3.

With the help of this local-norm bound, we immediately obtain the following results for MAB.

Corollary 1. Consider the following MAB algorithm: at time t sample a_t from p_t defined in Eq. (3) with $\hat{\ell}_t(a) = \frac{\ell_t(a)}{p_t(a)} \mathbf{1}\{a = a_t\}$ being the inverse importance weighted estimator. Then we have

$$\mathbb{E}[\mathcal{R}_T] \leq \frac{K^{1-\beta} - 1}{\eta(1-\beta)} + \frac{\eta K^\beta T}{2\beta}.$$

Therefore, by picking $\beta = 1/2$ and $\eta = 1/\sqrt{T}$, we obtain the minimax optimal regret $\mathcal{O}(\sqrt{TK})$.

Proof. We directly take expectation on both sides of Eq. (5) and apply Lemma 2, arriving at

$$\mathbb{E}[\mathcal{R}_T] \leq \frac{K^{1-\beta} - 1}{\eta(1-\beta)} + \frac{\eta}{2\beta} \sum_{t=1}^T \sum_{a=1}^K p_t(a)^{1-\beta}.$$

Applying Hölder's inequality to the last term

$$\sum_{a=1}^K p_t(a)^{1-\beta} \leq \left(\sum_{a=1}^K (p_t(a)^{1-\beta})^{\frac{1}{1-\beta}} \right)^{1-\beta} \left(\sum_{a=1}^K 1^{\frac{1}{\beta}} \right)^{\beta} = K^{\beta}$$

finishes the proof (or you can just argue that the LHS above is maximized when p is uniform). $\square$

Clearly, picking other constants such $\beta = 1/3$ (along with the optimal η) also leads to the same minimax optimal bound $\mathcal{O}(\sqrt{TK})$. Compared to Hedge, whose penalty-stability trade-off is $\frac{\ln K}{\eta}$ versus ηTK , the trade-off here for any constant β is essentially $\frac{K^{1-\beta}}{\eta}$ versus ηTK^β , leading a slight improvement in their product. Beside this improvement, in the next lecture we will also discuss one surprising application of this algorithm.

Question 2. On the other hand, is Tsallis entropy a good regularizer for the expert problem with full information?

Finally, note that this algorithm does not admit a closed-form update. Nevertheless, by writing down the Lagrangian and setting the gradient to zero, one can find that p_t satisfies

$$\frac{1}{p_t(a)^{1-\beta}} = \frac{1-\beta}{\beta} \left(\lambda + \eta \sum_{s < t} \hat{\ell}_s(a) \right), \quad \forall a \in [K] \quad (6)$$

for some constant λ (the Lagrangian multiplier) such that p_t is a distribution, which can be found efficiently using a simple binary search.

4 Comparisons between Full-Information and Bandit Feedback

We conclude by highlighting some connections and differences between online learning problems with full information and those with bandit feedback. First, even with the more challenging bandit feedback, many problems still admit $\mathcal{O}(\sqrt{T})$ regret bound, though with extra dependence on other parameters characterizing the price of having less information. Almost all such $\mathcal{O}(\sqrt{T})$ bounds are obtained by feeding a full-information counterpart algorithm with some loss estimators and a careful analysis that makes use of certain local norm to handle the potentially large variance of the loss estimators. Thus, the critical part in the algorithm design is to find the “right” combination of the loss estimator and the regularizer to enable such variance cancellation effect. Most of the adaptive bounds we discussed in Lecture 3 (e.g. small-loss bounds, path-length bounds, and almost constant regret for stochastic losses) also have their counterparts for the bandit setting.

Despite these similarities, there are also important distinctions between these two settings. For example, in Lecture 5 we derived strongly adaptive OCO algorithms with $\mathcal{O}(\sqrt{|I|})$ regret for all interval I simultaneously. It turns out that the same is *impossible* for bandit problems (see HW3). As for achieving $\mathcal{O}(\sqrt{ST})$ switching regret for all S when competing against a sequence with S switches (which is possible for the full-information setting), it has also been proven impossible lately for MAB with an adaptive adversary and more than two arms [Marinov and Zimmert, 2021]. Interestingly, the case with exactly two arms or an oblivious adversary remains open.

References

- Jacob D Abernethy, Chansoo Lee, and Ambuj Tewari. Fighting bandits with a new kind of smoothness. In *Advances in Neural Information Processing Systems* 28, 2015.
- Jean-Yves Audibert and Sébastien Bubeck. Regret bounds and minimax policies under partial monitoring. *Journal of Machine Learning Research*, 11(Oct):2785–2836, 2010.
- Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E Schapire. The nonstochastic multi-armed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002.
- Tze Leung Lai and Herbert Robbins. Asymptotically efficient adaptive allocation rules. *Advances in applied mathematics*, 6(1):4–22, 1985.
- Teodor Vanislavov Marinov and Julian Zimmert. The pareto frontier of model selection for general contextual bandits. *Advances in Neural Information Processing Systems*, 34:17956–17967, 2021.

CSCI 659 Lecture 7

Fall 2022

Instructor: Haipeng Luo

1 Stochastic Multi-Armed Bandits

In the last lecture, we discussed algorithms for the *adversarial* MAB problem where there is no assumption on how the loss vectors are generated, similar to all the problems we have analyzed so far. On the other hand, there is also a huge literature on the *stochastic* version of MAB, where each arm represents an unknown distribution and each pull of the arm generates an independent sample of that distribution. While this is clearly just a special case of adversarial MAB, the goal here is usually to derive regret bounds that are instance-dependent and in some situations better than the worst-case $\mathcal{O}(\sqrt{TK})$ bound. Moreover, unlike the full-information setting (where the stochastic assumption makes the problem much easier), stochastic MAB is still a highly non-trivial problem due to the partial information feedback.

Formally, in stochastic MAB, we assume that for each action a , there is an unknown distribution with support $[0, 1]$ and mean $\mu(a)$, such that $\ell_1(a), \dots, \ell_T(a)$ are independent samples of this distribution. For this problem we usually care about a slightly different version of regret, called *pseudo-regret*, defined as

$$\bar{\mathcal{R}}_T = \max_{a \in [K]} \mathbb{E} \left[\sum_{t=1}^T \ell_t(a_t) - \sum_{t=1}^T \ell_t(a) \right],$$

where the expectation is over the randomness of both the environment and the algorithm. It is easy to see that pseudo-regret is a weaker measure compared to standard expected regret since the latter moves the “max” inside the expectation. However, pseudo-regret not only is more natural in stochastic settings but also allows us to ignore the deviation of the samples $\ell_1(a), \dots, \ell_T(a)$ from their mean $\mu(a)$ and derive better regret bounds as we will see.

Let $a^* \in \arg\min_a \mu(a)$ be an optimal action in terms of the expected loss. Then, pseudo-regret for stochastic MAB can be simplified as

$$\bar{\mathcal{R}}_T = \mathbb{E} \left[\sum_{t=1}^T (\mu(a_t) - \mu(a^*)) \right] = \mathbb{E} \left[\sum_{t=1}^T \sum_{a=1}^K \Delta_a \mathbf{1}\{a_t = a\} \right] = \sum_{a: \Delta_a > 0} \Delta_a \mathbb{E}[n_T(a)] \quad (1)$$

where $\Delta_a = \mu(a) - \mu(a^*)$ is the *suboptimality gap* of action a and $n_t(a) = \sum_{s=1}^t \mathbf{1}\{a_s = a\}$ is the number of times action a has been pulled up to round t .

In stochastic MAB, the trade-off between exploration and exploitation is perhaps even more intuitive. To see this, let $\hat{\mu}_t(a) = \frac{1}{n_t(a)} \sum_{s=1}^t \mathbf{1}\{a_s = a\} \ell_s(a)$ be the empirical average loss of action a up to round t . Since the environment is stochastic, $\hat{\mu}_t(a)$ should be a good approximation of $\mu(a)$ as long as $n_t(a)$ is large enough. This is formally stated in the following concentration lemma.

Lemma 1. *For stochastic MAB, no matter what the learner’s strategy is, we have with probability at least $1 - 2K/T$, for all action $a \in [K]$ and all time $t \in [T]$:*

$$|\hat{\mu}_t(a) - \mu(a)| \leq 2\sqrt{\frac{\ln T}{n_t(a)}}.$$

The lemma is essentially proven by the standard Hoeffding’s inequality, except for the extra technicality needed to deal with the fact that $n_t(a)$ itself is also random. We omit the proof for simplicity.

Therefore, on the one hand, we want to exploit by picking the empirically best action $\operatorname{argmin}_a \hat{\mu}_t(a)$, but on the other hand, we also need to explore so that all actions are picked frequently enough to make sure that $\hat{\mu}_t(a)$ is indeed a good approximation of $\mu(a)$.

Explore-then-Exploit. Based on the intuition above, one naive approach would be “explore-then-exploit”, that is, first spend a certain number of rounds to pick all the actions equally often, and then simply stick with the empirically best action for all remaining rounds. While simple, this approach is also intuitively wasteful and nonadaptive in the exploration phase, in the sense that every action is selected equally often, even if some of them look much worse than others. A quick analysis also shows that the algorithm is suboptimal: suppose that we want the estimation error for every action to be ϵ (with high probability), then based on Lemma 1 we need to select every action roughly $(\ln T)/\epsilon^2$ rounds; therefore, the total regret would be of order $\tilde{O}(\frac{K}{\epsilon^2} + \epsilon T)$ (since in the exploration phase, per round regret could be as large as a constant, while in the exploitation phase, per round regret can at most be 2ϵ), which is at best $\tilde{O}(T^{2/3} K^{1/3})$ with the optimally-chosen ϵ .

2 The UCB Algorithm

To improve over this naive approach, we need a more adaptive way to trade off exploration and exploitation. The classical algorithm to do so is the UCB (Upper Confidence Bound) algorithm [Auer et al., 2002]. Since we consider “losses” instead of “rewards” (the original setting of [Auer et al., 2002]), the algorithm that we will discuss is actually LCB (Lower Confidence Bound). For convention, however, we will still call it UCB.

UCB applies a very powerful principle called “optimism in face of uncertainty”, which is useful in many other stochastic problems with bandit feedback. The main idea of the principle is the following: among all plausible environments that are consistent with the data observed so far, *the learner should be optimistic and act as if the environment is the best possible one*.

Specifically, at each round t , Lemma 1 already tells us what the plausible environments are given the data observed so far. Among them, the most favorable one to the learner is the one with the smallest loss mean, that is, when the loss mean for each action a is the following lower confidence bound:

$$\text{LCB}_t(a) \stackrel{\text{def}}{=} \hat{\mu}_{t-1}(a) - 2\sqrt{\frac{\ln T}{n_{t-1}(a)}}.$$

Therefore, an optimistic learner would wishfully believe that this is the true environment and naturally selects

$$a_t = \operatorname{argmin}_{a \in [K]} \text{LCB}_t(a). \quad (2)$$

This is exactly the UCB algorithm.

A couple of remarks are in order. First, note that $n_{t-1}(a)$ is initially 0, leading to negative infinity for $\text{LCB}_t(a)$, so the algorithm will be forced to pick each action exactly once for the first K rounds. Afterwards, the two terms in $\text{LCB}_t(a)$ are essentially playing the role of exploitation and exploration respectively: the first term suggests picking actions with low empirical mean (exploitation), while the second term suggests picking actions that have not been selected often enough (exploration). Together, they achieve an adaptive trade-off between exploitation and exploration, which does not waste many rounds to explore an infrequently selected action if it already looks pretty bad. Also note that optimism is indeed important to derive exploration — think about what would happen if we adopt pessimism instead and pick the arm with the smallest upper confidence bound $\hat{\mu}_{t-1}(a) + 2\sqrt{(\ln T)/n_{t-1}(a)}$. Moreover, in contrast to Exp3, UCB is a deterministic algorithm, that is, no randomness is used in deciding which actions to play.

Question 1. *In adversarial MAB, can a deterministic algorithm be no-regret?*

To analyze the algorithm, we start with the following key lemma.

Lemma 2. *Under the event stated in Lemma 1, UCB ensures $\Delta_{a_t} \leq 4\sqrt{\frac{\ln T}{n_{t-1}(a_t)}}$ for each time t .*

Proof. This is a direct consequence of optimism:

$$\begin{aligned}\Delta_{a_t} &= \mu(a_t) - \mu(a^*) \leq \mu(a_t) - \text{LCB}_t(a^*) && \text{(Lemma 1)} \\ &\leq \mu(a_t) - \text{LCB}_t(a_t) && \text{(by Eq. (2), } a_t \text{ has the smallest LCB)} \\ &\leq 4\sqrt{\frac{\ln T}{n_{t-1}(a_t)}}, && \text{(by the definition of LCB and Lemma 1)}\end{aligned}$$

which completes the proof. $\square$

This simple fact immediately tells us how many times each action can be selected by UCB.

Theorem 1. *Under the event stated in Lemma 1, we have $n_T(a) \leq \frac{16 \ln T}{\Delta_a^2} + 1$ for every action a . Consequently, the pseudo-regret of UCB satisfies $\bar{\mathcal{R}}_T = \mathcal{O}(\sum_{a: \Delta_a > 0} \frac{\ln T}{\Delta_a})$*

Proof. For each action a , applying Lemma 2 with t being the last time a is selected shows: $\Delta_a \leq 4\sqrt{\frac{\ln T}{n_T(a)-1}}$, which, after rearranging, proves the first statement. Therefore, if we denote the event stated in Lemma 1 by E , then the pseudo-regret (recall Eq. (1)) of UCB is

$$\begin{aligned}\bar{\mathcal{R}}_T &\leq \Pr(E) \times \sum_{a: \Delta_a > 0} \Delta_a \mathbb{E}[n_T(a) \mid E] + \Pr(\neg E) \times T \\ &\leq \sum_{a: \Delta_a > 0} \left(\frac{16 \ln T}{\Delta_a} + \Delta_a \right) + 2K = \mathcal{O} \left(\sum_{a: \Delta_a > 0} \frac{\ln T}{\Delta_a} \right),\end{aligned}$$

which proves the second statement. $\square$

This regret bound is in spirit similar to that in Theorem 6 of Lecture 3 for the full information setting, in the sense that they both depend on the difficulty of the specific problem instance, measured by some suboptimality gaps, and in exchange enjoy much better dependence on T compared to the worst-case $\sqrt{T}$ -bound. In fact, it can be shown that such a bound is *instance-optimal* for stochastic MAB, meaning that for each problem instance, this is the best pseudo-regret bound one can achieve using a “reasonable” algorithm.

Question 2. *The $\Omega(\sqrt{TK})$ lower bound construction for adversarial MAB in Lecture 6 is in fact also a stochastic environment, so why is that not a contradiction with this $(\ln T)$ -bound? Is this due to the difference between expected regret and pseudo-regret?*

In this instance-optimal bound, the smaller the gaps, the larger the pseudo-regret, which makes sense to some degree because smaller gaps make it harder to distinguish the optimal actions from the rest. On the other hand, however, if an action really has a tiny suboptimality gap, then by definition selecting it does not lead to large regret and thus there is really no point in distinguishing it from the optimal actions. While this intuition is not reflected in this instance-optimal regret bound, a different analysis below shows that UCB indeed still guarantees $\mathcal{O}(\sqrt{TK \ln T})$ regret at the same time, no matter how small the gaps are.

Theorem 2. *For any algorithm, if t_0 is the first time such that $n_{t_0}(a) \geq 1$ for all a (that is, all actions have been selected at least once), then $\sum_{t=t_0+1}^T \frac{1}{\sqrt{n_{t-1}(a_t)}} \leq 2\sqrt{TK}$. Consequently, UCB ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\sqrt{TK \ln T})$.*

Proof. The first statement can be shown by direct calculation:

$$\begin{aligned}\sum_{t=t_0+1}^T \frac{1}{\sqrt{n_{t-1}(a_t)}} &= \sum_{a=1}^K \sum_{t=t_0+1}^T \frac{\mathbf{1}\{a = a_t\}}{\sqrt{n_{t-1}(a)}} \\ &\leq \sum_{a=1}^K \sum_{s=1}^{n_T(a)-1} \frac{1}{\sqrt{s}} \leq 2 \sum_{a=1}^K \sqrt{n_T(a)} \quad \left(\sum_{s=1}^m \frac{1}{\sqrt{s}} \leq 2 \sum_{s=1}^m (\sqrt{s} - \sqrt{s-1}) = 2\sqrt{m} \right) \\ &\leq 2 \sqrt{\left(\sum_{a=1}^K n_T(a) \right) K} = 2\sqrt{TK}. \quad \text{(Cauchy-Schwarz inequality)}\end{aligned}$$

For UCB, t_0 is simply K as discussed. Thus, using Lemma 2 we can bound the pseudo-regret as

$$\bar{\mathcal{R}}_T \leq (K + 8\sqrt{TK \ln T}) + \Pr(\text{event in Lemma 1 does not hold}) \times T = \mathcal{O}(\sqrt{TK \ln T}),$$

which proves the second statement. $\square$

3 Best of Both Worlds

Given the two different types of results we have seen so far: $\mathcal{O}(\sqrt{TK})$ regret for adversarial MAB and $\mathcal{O}(\sum_{a: \Delta_a > 0} \frac{\ln T}{\Delta_a})$ regret for stochastic MAB, it is natural to ask whether there is one single algorithm that achieves the best of these two worlds *simultaneously*, without knowing ahead of time whether the world is stochastic or adversarial. It can be verified either theoretically or empirically that neither UCB or vanilla Exp3 achieves such a best-of-both-worlds result. In fact, the possibility of such results had been highly unclear, until the seminal work of [Bubeck and Slivkins \[2012\]](#), where they propose to start with running UCB, and then switch to Exp3 if some sophisticated tests show that the environment is unlikely to be stochastic.

However, such algorithms are not only complicated and impractical, but also suboptimal in several aspects. Ideally, we would like to achieve such best-of-both-worlds results via a simple algorithm, or, even better, via a simple adaptive regret bound, independent of the concrete algorithm, similar to the result of Theorem 6 from Lecture 3 for the full information setting. Recent research has made significant progress in this direction and indeed provides a positive answer to this question. The following result, taken from [\[Zimmert and Seldin, 2019\]](#), shows that if a certain adaptive regret bound holds, then the algorithm not only automatically achieves the best of the both stochastic and adversarial worlds, but also smoothly bridges these two worlds with a natural performance guarantee for any environments in between.

Theorem 3. *Suppose that an MAB algorithm always ensures for any action $a^* \in [K]$,*

$$\bar{\mathcal{R}}_T \leq \mathcal{O} \left(\mathbb{E} \left[\sum_{t=1}^T \sum_{a \neq a^*} \sqrt{\frac{p_t(a)}{t}} \right] \right), \quad (3)$$

where $p_t(a)$ is the probability of picking action a at time t . Then the same algorithm

- always ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\sqrt{TK})$, even if the environment is adversarial;¹
- ensures $\bar{\mathcal{R}}_T = \mathcal{O} \left(\sum_{a \neq a^*} \frac{\ln T}{\Delta_a} + \sqrt{C \sum_{a \neq a^*} \frac{\ln T}{\Delta_a}} \right)$ if the environment is such that

$$\bar{\mathcal{R}}_T \geq \mathbb{E} \left[\sum_{t=1}^T \sum_{a \neq a^*} p_t(a) \Delta(a) \right] - C \quad (4)$$

for some action a^* , gap measures $\Delta(a) > 0$ for $a \neq a^*$, and a constant $C > 0$.

The first statement is clear: the algorithm is always minimax optimal in the adversarial world. The second statement is a bit obscure, and let's unpack its meaning. First, note that stochastic MAB satisfies the required condition (4) with a^* being the optimal action (assumed to be unique here for simplicity), $\Delta(a)$ being the suboptimality gap defined earlier, and $C = 0$. In fact, in this case Eq. (4) holds with equality. Thus, the theorem says that for stochastic MAB, the algorithm automatically enjoys the instance-optimal pseudo-regret $\mathcal{O}(\sum_{a \neq a^*} \frac{\ln T}{\Delta_a})$.

But the second statement says much more than that. For example, consider a *corrupted stochastic setting* that smoothly interpolates between the stochastic setting and the adversarial setting: the environment first follows the protocol of stochastic MAB and generates losses $\ell'_1(a), \dots, \ell'_T(a)$ for each action a according to a fixed distribution with mean $\mu(a)$, then, an adversary arbitrarily corrupts the losses to generate the final loss sequence $\ell_1(a), \dots, \ell_T(a)$, subject to a “budget” C such that $2 \sum_{t=1}^T \|\ell_t - \ell'_t\|_\infty \leq C$. This setting satisfies (4) again since

$$\bar{\mathcal{R}}_T \geq \max_a \mathbb{E} \left[\sum_{t=1}^T \ell'_t(a_t) - \sum_{t=1}^T \ell'_t(a) \right] - C = \mathbb{E} \left[\sum_{t=1}^T \sum_{a \neq a^*} p_t(a) \Delta(a) \right] - C,$$

¹Note that for an oblivious adversary, pseudo-regret is the same as regular expected regret.

where a^* and $\Delta(a)$ are respectively the optimal action and the suboptimality gap of the original stochastic environment. In this case, the bound stated in Theorem 3, $\bar{\mathcal{R}}_T = \mathcal{O}\left(\sum_{a \neq a^*} \frac{\ln T}{\Delta(a)} + \sqrt{C \sum_{a \neq a^*} \frac{\ln T}{\Delta(a)}}\right)$, naturally increases in C (the amount of corruption), smoothly interpolating between the instance-optimal bound for stochastic MAB with no corruption and the $\sqrt{T}$ type of bound for adversarial MAB (since C is at most $2T$). Importantly, the theorem also implicitly implies that the algorithm does not need the knowledge of C !

More generally, condition (4) can be even weaker than the settings discussed above, since no independence (across actions or time) is really required. Below, we provide the proof for this theorem.

Proof. The first statement is by a direct application of Cauchy-Schwarz inequality following Eq. (3):

$$\bar{\mathcal{R}}_T \leq \mathcal{O}\left(\mathbb{E}\left[\sum_{t=1}^T \sum_{a \neq a^*} \sqrt{\frac{p_t(a)}{t}}\right]\right) = \mathcal{O}\left(\mathbb{E}\left[\sum_{t=1}^T \sqrt{\frac{K \sum_{a \neq a^*} p_t(a)}{t}}\right]\right) = \mathcal{O}(\sqrt{TK}).$$

For the second statement, we let B be the constant hidden in the $\mathcal{O}(\cdot)$ notation of Eq. (3), and then bound the regret by *a faction of itself*:

$$\bar{\mathcal{R}}_T \leq \mathbb{E}\left[B \sum_{t=1}^T \sum_{a \neq a^*} \sqrt{\frac{p_t(a)}{t}}\right] \leq \mathbb{E}\left[\sum_{t=1}^T \sum_{a \neq a^*} \frac{p_t(a)\Delta(a)}{2z} + \frac{zB^2}{2t\Delta(a)}\right] \leq \frac{\bar{\mathcal{R}}_T + C}{2z} + 2zU,$$

where the second step is by AM-GM inequality and holds for any $z \geq 0$, and the third step uses condition (4), the fact $\sum_{t=1}^T \frac{1}{t} \leq 2 \ln T$, and a shorthand $U = \frac{B^2}{2} \sum_{a \neq a^*} \frac{\ln T}{\Delta(a)}$. As long as $z \geq 1/2$, rearranging shows $\bar{\mathcal{R}}_T \leq \frac{4z^2}{2z-1}U + \frac{C}{2z-1}$. With $x = 2z - 1$, this can be rewritten as $\bar{\mathcal{R}}_T \leq 2U + Ux + \frac{U+C}{x}$, which is of order $\mathcal{O}(U + \sqrt{(U+C)U}) = \mathcal{O}(U + \sqrt{CU})$ after picking the optimal x to minimize the bound (note that x only appears in the analysis and is not a parameter of the algorithm). This completes the proof. $\square$

The next question, of course, is what algorithms achieve this magical Eq. (3). In fact, we have seen one such example already: FTRL with Tsallis entropy regularizer $\psi(p) = -2 \sum_{a=1}^K \sqrt{p(a)}$. The only modification needed is that at time t , we use a time-varying learning rate $\eta_t = 1/\sqrt{t}$. We leave the proof to HW3, and only point out that it is quite surprising that such a simple algorithm exhibits strong optimality and adaptivity to many different types of environments simultaneously.

4 Stochastic Linear Bandits and LinUCB

Next, we come back to the stochastic setting and study a generalization of stochastic MAB. One issue of stochastic MAB is that it completely ignores the possible dependence across different actions, while in practice they are often correlated one way or another. Taking such correlation into account is especially important when the number of actions is huge. Here, we consider a setting where such correlation is represented in the simplest linear manner, known as *stochastic linear bandits*.

The learning protocol is as follows: for each round $t = 1, \dots, T$,

1. An *arbitrary* set of actions $A_t \subset \mathbb{R}^d$ is revealed to the learner;
2. the learner picks an action $a_t \in A_t$ and observe its loss $c_t = \langle a_t, \theta^* \rangle + \epsilon_t$ where $\theta^* \in \mathbb{R}^d$ is a fixed and unknown parameter and $\epsilon_t \sim \mathcal{N}(0, 1)$ is independent standard Gaussian noise.

Let $a_t^* \in \arg\min_{a \in A_t} \langle a, \theta^* \rangle$ be an optimal action at time t . The pseudo-regret of this problem is naturally defined as

$$\bar{\mathcal{R}}_T = \mathbb{E}\left[\sum_{t=1}^T \langle a_t - a_t^*, \theta^* \rangle\right].$$

Note that stochastic MAB is essentially a special case of this model, where $d = K$, A_t is always $\{e_1, \dots, e_d\}$ (the standard basis vectors of $\mathbb{R}^d$), and $\theta^* = (\mu(1), \dots, \mu(K))$ is the vector of loss

mean for each action. The only slight difference is that now we consider Gaussian noise for each observation of the loss, which is less general compared to the arbitrary bounded noise model in stochastic MAB, but more general in that the noise can be unbounded.

In general, the stochastic linear bandit model is much more powerful since 1) it allows each action to come with an arbitrary “feature”; 2) the number of actions can be arbitrary (even infinite); and 3) the set of available actions can be different at different time. This allows us to capture real-life problems such as building a personalized news recommendation system [Li et al., 2010]. In this application, each time t corresponds to a visit of some user to the website. The available news articles at that time as well as the user’s information are then used to generate a feature vector for each article. Afterwards, a linear bandit algorithm selects an action and recommends the corresponding article to the user. The loss is then based on whether the user clicks on the recommended article or not. It is assumed that the expected loss of an action can be perfectly predicted by an unknown linear predictor θ^* (but we will see generalization to nonlinear models in the future).

Note that because of the changing action sets, it only makes sense to define the pseudo-regret so that it compares the expected loss of the algorithm to the expected loss of the best action *at each time*. This relates to the notion of dynamic regret discussed before. However, while in general sublinear dynamic regret is impossible, due to the stochastic assumption, regret of order $\mathcal{O}(\sqrt{T})$ is in fact achievable here as we will see soon.

Finally, without loss of generality, we make two scaling assumptions: $\max_{a \in A_t} \|a\|_2 \leq 1$ for all t and $\|\theta^*\|_2 \leq 1$.

4.1 The LinUCB Algorithm

To solve this problem, let’s apply the same “optimism in face of uncertainty” principle. The first step is to come up with the set of plausible environments that are consistent with the observed data. Here, the only parameter of the environment is the linear predictor θ^* , so the first goal would be to come up with a confidence set Θ_t for time t based on the observation $a_1, c_1, \dots, a_t, c_t$, so that $\theta^* \in \Theta_t$ with high probability. With such a confidence set, similarly to the UCB algorithm, at time $t + 1$ we optimistically assume that the loss for each action $a \in A_{t+1}$ is

$$\text{LCB}_{t+1}(a) = \min_{\theta \in \Theta_t} \langle a, \theta \rangle,$$

and finally pick action $a_{t+1} = \operatorname{argmin}_{a \in A_{t+1}} \text{LCB}_{t+1}(a)$.

It remains to come up with the confidence set Θ_t . First, we need to figure out what the “center” of this set is. For UCB, the center of the confidence set is simply and naturally the empirical average of observations. For linear bandit, note that we are observing $c_s \approx \langle a_s, \theta^* \rangle$ for $s = 1, \dots, t$. It is thus natural to perform least square regression to obtain an estimate of θ^* as the center:

$$\hat{\theta}_t = \operatorname{argmin}_{\theta \in \mathbb{R}^d} \sum_{s=1}^t (\langle a_s, \theta \rangle - c_s)^2.$$

By direct calculations, one can verify $\hat{\theta}_t = M_t^{-1} \sum_{s=1}^t c_s a_s$ where $M_t = \sum_{s=1}^t a_s a_s^\top$ is called the empirical covariance matrix and is assumed to be invertible for now. Note that this is consistent with UCB: when $A_t = \{e_1, \dots, e_d\}$, M_t is a diagonal matrix with $M_t(i, i)$ being the number of times action i has been selected, and thus $\hat{\theta}_t$ is exactly the vector of empirical mean for each action.

Next, we need to figure out what Θ_t should look like around the center $\hat{\theta}_t$, that is, to understand the distribution of $\hat{\theta}_t$. To get an intuition, we first ignore the fact that each a_s is itself random variables and think of them as fixed vectors (all assumptions will be dropped eventually). Then, the only randomness in $\hat{\theta}_t$ comes from the noises $\epsilon_1, \dots, \epsilon_t$. Plugging in $c_s = \langle a_s, \theta^* \rangle + \epsilon_s$, we rewrite $\hat{\theta}_t$ as

$$\hat{\theta}_t = \left(M_t^{-1} \sum_{s=1}^t (\langle a_s, \theta^* \rangle + \epsilon_s) a_s \right) = M_t^{-1} M_t \theta^* + M_t^{-1} \sum_{s=1}^t \epsilon_s a_s = \theta^* + M_t^{-1} Z_t$$

where $Z_t = \sum_{s=1}^t \epsilon_s a_s$ is a zero-mean d -dimensional Gaussian variable with covariance matrix

$$\mathbb{E} [Z_t Z_t^\top] = \sum_{s_1=1}^t \sum_{s_2=1}^t \mathbb{E} [\epsilon_{s_1} \epsilon_{s_2}] a_{s_1} a_{s_2}^\top = \sum_{s=1}^t \mathbb{E} [\epsilon_s^2] a_s a_s^\top = M_t.$$

Therefore, the random variable $M_t^{1/2}(\hat{\theta}_t - \theta^*)$ is distributed as $\mathcal{N}(0, I)$, the d -dimensional standard Gaussian. By standard concentration results (specifically, tail bounds of the χ_d^2 distribution), the following holds for any $\delta \in (0, 1)$ (details omitted)

$$\Pr \left(\left\| M_t^{1/2}(\hat{\theta}_t - \theta^*) \right\|_2^2 \leq d + 2\sqrt{d \ln \frac{1}{\delta}} + 2 \ln \frac{1}{\delta} \right) \geq 1 - \delta.$$

Inspired by this, the confidence set can be constructed as (recall the notation $\|v\|_M = \sqrt{v^\top M v}$)

$$\Theta_t = \left\{ \theta \in \mathbb{R}^d : \left\| \theta - \hat{\theta}_t \right\|_{M_t}^2 \leq d + 2\sqrt{d \ln \frac{1}{\delta}} + 2 \ln \frac{1}{\delta} \right\},$$

which is in fact an ellipsoid centered at $\hat{\theta}_t$. (The set defined by $\|v\|_M \leq 1$ is the standard analytic form of an ellipsoid centered at the origin, where the eigenvectors of M define the principal axes of the ellipsoid and the corresponding eigenvalues are the reciprocals of the square of the semi-axes.)

In the derivation above, we made two assumptions. First, M_t is invertible, which is not true until $a_1, \dots, a_t$ span $\mathbb{R}^d$. This can be resolved by adding an ℓ_2 -regularization to the least square regression

$$\hat{\theta}_t = \operatorname{argmin}_{\theta \in \mathbb{R}^d} \sum_{s=1}^t (\langle a_s, \theta \rangle - c_s)^2 + \lambda \|\theta\|_2^2 = M_t^{-1} \sum_{s=1}^t c_s a_s$$

where M_t is redefined as $\lambda I + \sum_{s=1}^t a_s a_s^\top$ for some parameter $\lambda > 0$ and is always invertible now. However, what is more difficult to get rid of is the second assumption that each a_s is fixed and not random. Fortunately, with some probability tools, this can still be addressed. We omit the details and only show the final lemma taken from [Abbasi-Yadkori et al., 2011].

Lemma 3 (Confidence Ellipsoid). *For any stochastic linear bandit algorithm and fixed parameters $\lambda > 0$ and $\delta \in (0, 1)$, we have with probability at least $1 - \delta$, $\theta^* \in \Theta_t$ holds for all t , where*

$$\Theta_t = \left\{ \theta \in \mathbb{R}^d : \left\| \theta - \hat{\theta}_t \right\|_{M_t} \leq \beta \right\},$$

$$M_t = \lambda I + \sum_{s=1}^t a_s a_s^\top, \hat{\theta}_t = M_t^{-1} \sum_{s=1}^t c_s a_s, \text{ and } \beta = \sqrt{\lambda} + \sqrt{2 \ln \frac{1}{\delta} + d \ln \left(1 + \frac{T}{d\lambda}\right)}.$$

Finally, having this confidence set Θ_t , we can further simplify the algorithm by noting

$$\begin{aligned} \text{LCB}_{t+1}(a) &= \min_{\theta \in \Theta_t} \langle a, \theta \rangle = \min_{\|\theta - \hat{\theta}_t\|_{M_t} \leq \beta} \langle a, \theta \rangle \\ &= \min_{\|\theta'\|_2 \leq \beta} \left\langle a, M_t^{-\frac{1}{2}} \theta' + \hat{\theta}_t \right\rangle && (\text{define } \theta' = M_t^{\frac{1}{2}}(\theta - \hat{\theta}_t)) \\ &= \left\langle a, \hat{\theta}_t \right\rangle + \min_{\|\theta'\|_2 \leq \beta} \left\langle M_t^{-\frac{1}{2}} a, \theta' \right\rangle \\ &= \left\langle a, \hat{\theta}_t \right\rangle - \beta \|a\|_{M_t^{-1}} && (\min_{\|v\|_2 \leq 1} \langle x, v \rangle = -\|x\|_2) \end{aligned}$$

and thus

$$a_{t+1} = \operatorname{argmin}_{a \in A_{t+1}} \text{LCB}_{t+1}(a) = \operatorname{argmin}_{a \in A_{t+1}} \left(\langle a, \hat{\theta}_t \rangle - \beta \|a\|_{M_t^{-1}} \right).$$

This algorithm is called LinUCB (or OFUL, shorf for Optimism in Face of Uncertainty for Linear bandit). Very similar to UCB, the term $\langle a, \hat{\theta}_t \rangle$ represents exploitation while the term $-\beta_t \|a\|_{M_t^{-1}}$ drives exploration for unobserved directions (since $\|a\|_{M_t^{-1}}$ is large when a represents a direction that is very different from the observed ones $a_1, \dots, a_t$). Indeed, when $A_t = \{e_1, \dots, e_d\}$, one can verify that LinUCB recovers UCB (other than the difference in the coefficient β).

4.2 Regret Analysis

The analysis of LinUCB is also a generalization of that for UCB, starting with the following key lemma enabled by optimism, which is an analogue of Lemma 2.

Lemma 4. *With probability at least $1 - \delta$, LinUCB ensures $\langle a_t - a_t^*, \theta^* \rangle \leq 2\beta \|a_t\|_{M_{t-1}^{-1}}$ for all t .*

Proof. Recall that under the event stated in Lemma 3, $\text{LCB}_t(a) = \min_{\theta \in \Theta_{t-1}} \langle a, \theta \rangle = \langle a, \hat{\theta}_{t-1} \rangle - \beta \|a\|_{M_{t-1}^{-1}}$ is indeed a lower bound on the true loss $\langle a, \theta^* \rangle$ for any action a . Therefore,

$$\begin{aligned} \langle a_t - a_t^*, \theta^* \rangle &\leq \langle a_t, \theta^* \rangle - \text{LCB}_t(a_t^*) \\ &\leq \langle a_t, \theta^* \rangle - \text{LCB}_t(a_t) && (a_t \text{ has the smallest LCB}) \\ &= \langle a_t, \theta^* - \hat{\theta}_{t-1} \rangle + \beta \|a_t\|_{M_{t-1}^{-1}} \\ &\leq \|a_t\|_{M_{t-1}^{-1}} \|\theta^* - \hat{\theta}_{t-1}\|_{M_{t-1}} + \beta \|a_t\|_{M_{t-1}^{-1}} && (\text{Hölder's inequality}) \\ &\leq 2\beta \|a_t\|_{M_{t-1}^{-1}}, && (\theta^* \in \Theta_t) \end{aligned}$$

which finishes the proof. $\square$

We now proceed with a statement analogous to Theorem 2, whose proof is purely an exercise of linear algebra and is independent of the algorithm.

Theorem 4. *For any algorithm, we have $\sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}} \leq \sqrt{T \left(\frac{d}{\lambda} + d \ln \left(1 + \frac{T}{\lambda d} \right) \right)}$. Consequently, LinUCB with $\lambda = 1$ and $\delta = 1/T$ ensures $\bar{\mathcal{R}}_T = \tilde{\mathcal{O}}(d\sqrt{T})$.*

Proof. First, we apply Cauchy-Schwarz and obtain $\sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}} \leq \sqrt{T \sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}}^2}$. Note that a slightly different expression $\|a_t\|_{M_t^{-1}}^2$ can be bounded as

$$\|a_t\|_{M_t^{-1}}^2 = \langle M_t^{-1}, a_t a_t^\top \rangle = \langle M_t^{-1}, M_t - M_{t-1} \rangle \leq \ln \det(M_t) - \ln \det(M_{t-1})$$

where the last step uses the concavity of the $\ln \det(M)$ function and that its gradient is exactly M^{-1} . Therefore, $\sum_{t=1}^T \|a_t\|_{M_t^{-1}}^2 \leq \ln \det(M_T) - \ln \det(M_0) = \ln \det(M_T) - d \ln \lambda$. To bound $\det(M_T)$, we apply AM-GM inequality and the assumption $a_t^\top a_t \leq 1$:

$$\det(M_T) \leq \left(\frac{\text{TR}(M_T)}{d} \right)^d = \left(\frac{\lambda d + \sum_{t=1}^T \text{TR}(a_t a_t^\top)}{d} \right)^d = \left(\lambda + \frac{\sum_{t=1}^T a_t^\top a_t}{d} \right)^d \leq \left(\lambda + \frac{T}{d} \right)^d.$$

We have thus shown $\sum_{t=1}^T \|a_t\|_{M_t^{-1}}^2 \leq d \ln \left(1 + \frac{T}{\lambda d} \right)$, and it remains to bound the difference between $\sum_{t=1}^T \|a_t\|_{M_t^{-1}}^2$ and $\sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}}^2$. Indeed, they are very close:

$$\begin{aligned} \sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}}^2 - \sum_{t=1}^T \|a_t\|_{M_t^{-1}}^2 &= \sum_{t=1}^T \langle M_{t-1}^{-1} - M_t^{-1}, a_t a_t^\top \rangle \\ &\leq \sum_{t=1}^T \langle M_{t-1}^{-1} - M_t^{-1}, I_d \rangle \leq \langle M_0^{-1}, I_d \rangle = \frac{d}{\lambda}, \end{aligned} \quad (I_d \text{ is the identity matrix})$$

where the first inequality is by the fact that both $M_{t-1}^{-1} - M_t^{-1}$ and $I_d - a_t a_t^\top$ are PSD matrices. Combining all results above proves the first statement. The second statement is clear after plugging in the value of λ and δ , since $\sum_{t=1}^T \langle a_t - a_t^*, \theta^* \rangle$ is at most $2\beta \sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}}$ with probability at least $1 - 1/T$ (Lemma 4), and at most $2T$ with probability $1/T$. $\square$

Importantly, this regret bound has no dependence on the number of actions at all! Instead, it pays for linear dependence on the dimension d of the features. While this does not recover the $\tilde{\mathcal{O}}(\sqrt{TK})$ bound of UCB, such linear dependence on d is known to be generally unavoidable for this problem.

Can we also derive an instance-dependent regret bound analogous to Theorem 1 using some suboptimality gap as a difficulty measure? The answer is yes, via a quite simple argument.

Theorem 5. Define the minimal suboptimal gap as $\Delta = \min_t \min_{a \in A_t: \langle a - a_t^*, \theta^* \rangle > 0} \langle a - a_t^*, \theta^* \rangle$. Then LinUCB with $\lambda = 1$ and $\delta = 1/T$ ensures $\bar{\mathcal{R}}_T = \mathcal{O}\left(\frac{(d \ln T)^2}{\Delta}\right)$.

Proof. Simply bound the per-round regret $\langle a_t - a_t^*, \theta^* \rangle$ by $\frac{1}{\Delta} \langle a_t - a_t^*, \theta^* \rangle^2$, which, under the event stated in Lemma 3, is further bounded by $\frac{4\beta^2}{\Delta} \|a_t\|_{M_{t-1}^{-1}}^2$. In the proof of Theorem 4, we have already shown $\sum_{t=1}^T \|a_t\|_{M_{t-1}^{-1}}^2 \leq \frac{d}{\lambda} + d \ln\left(1 + \frac{T}{\lambda d}\right)$. The proof is finished after plugging in the value of λ and δ and trivially bounding the case when the high-probability event does not hold. $\square$

While this looks very similar to the UCB result, unlike the case for stochastic MAB, such a gap-dependent bound is *not* instance-optimal. In fact, somewhat surprisingly, it has been shown that optimism-based approaches can never be instance-optimal for stochastic linear bandits; see [Lattimore and Szepesvári, 2017]. Therefore, while the “optimism in face of uncertainty” principle is very powerful, especially in obtaining $\mathcal{O}(\sqrt{T})$ type of regret bounds, it is by no means always the best approach.

References

- Yasin Abbasi-Yadkori, Dávid Pál, and Csaba Szepesvári. Improved algorithms for linear stochastic bandits. In *Advances in Neural Information Processing Systems 24*, 2011.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine learning*, 47(2-3):235–256, 2002.
- Sébastien Bubeck and Aleksandrs Slivkins. The best of both worlds: Stochastic and adversarial bandits. In *Conference on Learning Theory*, pages 42–1, 2012.
- Tor Lattimore and Csaba Szepesvári. The end of optimism? an asymptotic analysis of finite-armed linear bandits. In *Artificial Intelligence and Statistics*, pages 728–737. PMLR, 2017.
- Lihong Li, Wei Chu, John Langford, and Robert E Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670. ACM, 2010.
- Julian Zimmert and Yevgeny Seldin. An optimal algorithm for stochastic and adversarial bandits. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pages 467–475. PMLR, 2019.

CSCI 659 Lecture 8

Fall 2022

Instructor: Haipeng Luo

1 Contextual Bandits

In the last lecture, we discussed stochastic linear bandits, which assumes a linear model on the loss of each action. The goal of this lecture is to relax this assumption by considering a more general function class or even allowing arbitrarily losses without any structures. To this end, consider the following general *contextual bandit* problem: at each round $t = 1, \dots, T$,

1. the environment first decides a *context* $x_t \in \mathcal{X}$ for an arbitrary context space $\mathcal{X}$ and a loss vector $\ell_t \in [0, 1]^K$ specifying the loss of K actions;
2. the learner observes the context x_t and then selects an action $a_t \in [K]$;
3. the learner suffers and observes $\ell_t(a_t)$.

Clearly, compared to the standard MAB problem, the only extra element here is the contexts. To explain what the role of these contexts is, we consider the following two settings.

Realizable Setting. One natural role of the context at each round is that it directly determines the loss (or at least its mean) of each action for this round, through some fixed function from a regressor class. This is known as the *realizable setting* since the loss is realized by some fixed regressor. Formally, let $\mathcal{F} \subseteq (\mathcal{X} \times [K]) \rightarrow [0, 1]$ be a regressor class known to the learner, and assume that there exists a regressor $f^* \in \mathcal{F}$, unknown to the learner, such that $\mathbb{E}[\ell_t(a)] = f^*(x_t, a)$ for all $t \in [T]$ and $a \in [K]$, where the expectation is with respect to the randomness of the environment when generating ℓ_t . In other words, there is a ground truth f^* that perfectly predicts the expected loss of each action given the context.

The stochastic linear bandit model discussed last time is essentially a special case of this realized setting, where the context x_t consists of the features $x_t^1, \dots, x_t^K \in \mathbb{R}^d$ of the actions, that is, $x_t = (x_t^1, \dots, x_t^K)$, and the regressor class is a set of linear functions: $\mathcal{F} = \{(x, a) \rightarrow \langle \theta, x^a \rangle \mid \theta \in \mathbb{R}^d, \|\theta\|_2 \leq 1\}$. One slight difference is that now for simplicity we consider a fixed number of actions. However, note that this does not mean the action set itself is fixed — each round we can have completely different K actions, encoded via different features (in the recommendation system example, this means that we can still have different items to recommend at each round). In other words, unlike standard MAB, here the index of each action $a \in [K]$ is not meaningful, in the sense that it does not matter if we arbitrarily shuffle the indices at each round, as long as their features are also shuffled in the same way.

By allowing an arbitrary regressor class beyond the linear model, this setting is much more powerful and practical. The realizability assumption also becomes less restricted when we use a richer regressor class such as a set of neural nets. To further relax the realizability assumption, sometimes we also allow a certain level of *misspecification* in the model by changing the assumption to: there exists a fixed $\epsilon \in [0, 1]$ and a regressor $f^* \in \mathcal{F}$ such that $|f^*(x_t, a) - \mathbb{E}[\ell_t(a)]| \leq \epsilon$ for all $t \in [T]$ and $a \in [K]$. The no-misspecification case discussed earlier is clearly just a special case with $\epsilon = 0$.

How should we measure the learner's performance in this problem? Note that each regressor $f \in \mathcal{F}$ naturally defines a *policy* $\pi_f : \mathcal{X} \rightarrow [K]$ that selects the action with the lowest expect loss for the

given context, that is, $\pi_f(x) = \operatorname{argmin}_{a \in [K]} f(x, a)$. It is then natural to measure the learner's performance against the best policy, via the following pseudo-regret:

$$\bar{\mathcal{R}}_T = \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=1}^T \ell_t(a_t) - \sum_{t=1}^T \ell_t(\pi(x_t)) \right], \quad (1)$$

where $\Pi = \{\pi_f : f \in \mathcal{F}\}$ is the set of all policies induced by $\mathcal{F}$. When there is no misspecification, it is clear that pseudo-regret becomes

$$\bar{\mathcal{R}}_T = \max_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=1}^T f^*(x_t, a_t) - \sum_{t=1}^T f^*(x_t, \pi(x_t)) \right] = \mathbb{E} \left[\sum_{t=1}^T f^*(x_t, a_t) - \sum_{t=1}^T f^*(x_t, \pi^*(x_t)) \right], \quad (2)$$

where we use π^* as a shorthand for π_{f^*} .

Agnostic Setting. Note that when the misspecification level ϵ is as bad as 1, that is, when there are no reasonable regressors at all, it does not mean there is nothing we can learn at all, because a policy can be bad at predicting the exact value of the losses but at the same time somehow still good at identifying the good actions most of the time. In this case, the corresponding regressor that a policy uses to make its decisions is not that important anymore, and what really matters is only the policy itself, that is, what action it takes under each context. Motivated by this, we consider the *agnostic setting* where there is no explicit relation between the contexts and the losses at all, and the learner's goal is, given a fixed policy class $\Pi \subseteq \mathcal{X} \rightarrow [K]$, to compete with what the best policy from this class can do, that is, to minimize the same pseudo-regret defined in Eq. (1) (with Π now being an arbitrary given policy class).

It is clear that the realizable setting is a special case where $\Pi = \{\pi_f : f \in \mathcal{F}\}$ and there is an additional assumption on how the contexts and losses are connected via a ground truth f^* . The standard MAB is also a special case where Π contains only K policies, the a -th of which always selects action a regardless of the context.

Naive Approaches. Note that the most interesting scenario is when the context space $\mathcal{X}$ is large (and potentially high-dimensional), and so is the regressor class $\mathcal{F}$ or the policy class Π ; just consider again the example where $\mathcal{F}$ is a set of neural nets. We will thus not make any restriction on the size of $\mathcal{X}$ (could even be infinity), and consider the case where the size of $\mathcal{F}$ or Π , denoted by N and assumed to be finite in most discussions for simplicity, is huge so that only $\operatorname{poly}(\ln N)$ dependence is acceptable for the regret guarantees (and also the computational complexity; more on this to follow). This makes the following two naive approaches uninteresting.

First, one can see contextual bandits as a combination of many independent K -armed bandit problems, one for each distinct context. In other words, for each possible $x \in \mathcal{X}$, we maintain a K -armed bandit algorithm, and at time t , we use the algorithm corresponding to context x_t to make the decision. This is clearly not a good approach since it is possible that we do not even see the same context twice over T rounds, in which case there is no learning at all in this approach as each K -armed bandit algorithm is run for at most one round.

Another approach is to treat each policy as an arm, making this an N -armed bandit problem. Indeed, at each time t , deciding which arm/policy π to pull naturally tells us which action a_t to play by following the suggestion of this policy given the current context, that is, $a_t = \pi(x_t)$. Then, we can treat the observation $\ell(a_t)$ as the loss for this arm. The issue is of course that this leads to regret of order $\sqrt{TN}$, which is unacceptable due to the polynomial dependence on N .

2 Adversarial Contextual Bandits

To get a sense of what a good regret guarantee looks like for contextual bandits, we start by analyzing a simple (but inefficient) algorithm for the most general case: agnostic setting with adversarial contexts and losses.

The algorithm is in fact very close to the first naive approach mentioned in the last section. At time t , we compute a distribution $P_t \in \Delta(N)$ over the policies using Hedge and the past loss estimators

$\widehat{\ell}_1, \dots, \widehat{\ell}_{t-1} \in \mathbb{R}_+^N$: $P_t(\pi) \propto \exp\left(-\eta \sum_{s < t} \widehat{\ell}_s(\pi)\right)$. Then, we sample a policy $\pi_t \sim P_t$ and play action $a_t = \pi_t(x_t)$, which is equivalent to sampling action a_t based on distribution $p_t \in \Delta(K)$ where $p_t(a) = \sum_{\pi \in \Pi: \pi(x) = a} P_t(\pi)$.

The key difference is how we construct estimator $\widehat{\ell}_t$ based on the feedback $\ell_t(a_t)$. What the naive approach does is $\widehat{\ell}_t(\pi) = \frac{\ell_t(\pi(x_t))}{P_t(\pi)} \mathbf{1}\{\pi = \pi_t\}$, completely following Exp3's importance weighted estimator by treating each policy as an arm. This unbiased estimator is non-zero only for the selected policy, and has a large variance: $\mathbb{E}_t[\widehat{\ell}_t(\pi)^2] = \frac{\ell_t(\pi(x_t))^2}{P_t(\pi)} \leq \frac{1}{P_t(\pi)}$.

However, note that the observation $\ell_t(a_t)$ tells us not only the loss for the selected policy, but also the loss for all other policies that selects the same action a_t . In other words, the missing information is really just the $K-1$ unobserved coordinates of ℓ_t , instead of $N-1$ values for the unselected policies. If we consider the problem in this way, then it is more natural to construct a K -dimensional estimator for ℓ_t , which we denote by the same notation $\widehat{\ell}_t \in \mathbb{R}_+^K$ with a slight abuse of notation (thus, whether $\widehat{\ell}_t$ is N dimensional or K dimensional depends on whether its input is a policy π or an action a). The construction is also by the same idea of inverse importance weighting: $\widehat{\ell}_t(a) = \frac{\ell_t(a)}{p_t(a)} \mathbf{1}\{a = a_t\}$.

Then we set the estimator $\widehat{\ell}_t(\pi)$ for policy π naturally as $\widehat{\ell}_t(\pi(x_t))$. Compared to the earlier naive estimator, this new one is also unbiased ($\mathbb{E}_t[\widehat{\ell}_t(\pi)] = \ell_t(\pi(x_t))$), but with a smaller variance:

$$\mathbb{E}_t[\widehat{\ell}_t(\pi)^2] = p_t(\pi(x_t)) \times \frac{\ell_t(\pi(x_t))^2}{p_t(\pi(x_t))^2} + (1 - p_t(\pi(x_t))) \times 0 = \frac{\ell_t(\pi(x_t))^2}{p_t(\pi(x_t))} \leq \frac{1}{p_t(\pi(x_t))}, \quad (3)$$

since $p_t(\pi(x_t))$ is by definition at least $P_t(\pi)$ (in fact, the former is usually much larger).

The resulting algorithm is called Exp4 [Auer et al., 2002], short for *Exponential-weight algorithm for Exploration and Exploitation using Expert advice* (since the role of each policy is similar to an expert as in the expert problem). We summarize the algorithm in the pseudocode below, followed by its formal regret guarantee.

Algorithm 1: Exp4

Input: learning rate $\eta > 0$

for $t = 1, \dots, T$ **do**

compute $P_t \in \Delta(N)$ such that $P_t(\pi) \propto \exp\left(-\eta \sum_{s < t} \widehat{\ell}_s(\pi(x_s))\right)$

sample a_t from $p_t \in \Delta(K)$ where $p_t(a) = \sum_{\pi \in \Pi: \pi(x) = a} P_t(\pi)$

observe $\ell_t(a_t)$ and construct $\widehat{\ell}_t \in \mathbb{R}_+^K$ such that $\widehat{\ell}_t(a) = \frac{\ell_t(a)}{p_t(a)} \mathbf{1}\{a = a_t\}$

Theorem 1. With $\eta = \sqrt{\frac{\ln N}{TK}}$, Exp4 ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\sqrt{TK \ln N})$.

Proof. Similar to the Exp3 analysis, the proof starts from the following local-norm bound for Hedge, which holds due to the nonnegativity of loss estimator: for any π^* ,

$$\sum_{t=1}^T \langle P_t, \widehat{\ell}_t \rangle - \sum_{t=1}^T \widehat{\ell}_t(\pi^*) \leq \frac{\ln N}{\eta} + \eta \sum_{t=1}^T \sum_{\pi \in \Pi} P_t(\pi) \widehat{\ell}_t(\pi)^2.$$

By the unbiasedness of $\widehat{\ell}_t$, we have

$$\mathbb{E}_t[\langle P_t, \widehat{\ell}_t \rangle] = \sum_{\pi \in \Pi} P_t(\pi) \ell_t(\pi(x_t)) = \sum_{a=1}^K \sum_{\pi: \pi(x_t) = a} P_t(\pi) \ell_t(a) = \sum_{a=1}^K p_t(a) \ell_t(a) = \mathbb{E}_t[\ell_t(a_t)],$$

and $\mathbb{E}_t[\widehat{\ell}_t(\pi^*)] = \ell_t(\pi^*(x_t))$. On the other hand, the expectation of the local-norm term can be bounded by K just as Exp3 (instead of N), thanks to the smaller variance of the estimator shown earlier in Eq. (3):

$$\mathbb{E}_t \left[\sum_{\pi \in \Pi} P_t(\pi) \widehat{\ell}_t(\pi)^2 \right] \leq \sum_{\pi \in \Pi} \frac{P_t(\pi)}{p_t(\pi(x_t))} = \sum_{a=1}^K \sum_{\pi: \pi(x_t) = a} \frac{P_t(\pi)}{p_t(a)} = \sum_{a=1}^K \frac{p_t(a)}{p_t(a)} = K.$$

Therefore, taking expectation on both sides, we obtain $\bar{\mathcal{R}}_T \leq \frac{\ln N}{\eta} + \eta TK$, which is $\mathcal{O}(\sqrt{TK \ln N})$ after picking the optimal η . $\square$

This proof shows again the importance of getting a right combination of local-norm and loss estimators. The regret bound $\mathcal{O}(\sqrt{TK \ln N})$ has only logarithmic dependence on N , same as Hedge for the expert problem, and $\sqrt{K}$ dependence on the number of actions, same as Exp3 for MAB. In fact, this is known to be near optimal for adversarial contextual bandits; see [Seldin and Lugosi, 2016].

Question 1. *What happens if we instead use FTRL with Tsallis entropy to determine P_t ?*

While the regret bound depends on N only logarithmically, one obvious issue of Exp4 is that its time/space complexity is linear in N . Just like the expert problem, generally there is no way to get pass this barrier unfortunately. To resolve this issue, we need to assume some *oracle access* to the regressor class $\mathcal{F}$ or the policy class Π , and derive algorithms that never search over all policies explicitly and instead only make use of the regressor/policy class through such oracles. These oracles themselves might not be efficiently implementable exactly, but as long as they are practically reasonable, we can often approximately implement them using different heuristics that are widely used in practice anyway. The question then becomes, given such oracles, how to derive algorithms that make a small number of oracle calls while ensuring low regret (ideally comparable to the optimal bound $\mathcal{O}(\sqrt{TK \ln N})$). This turns out to be a highly nontrivial problem, and in the remaining of this lecture we will discuss some examples from recent research.

3 Realizable Setting with an Online Regression Oracle

Consider the realizable setting with a regressor class $\mathcal{F}$, and assume that the learner is given access to an *online regression oracle* $\mathcal{O}_{sq}$ that solves the following online regression problem:

1. the environment decides and reveals an instance $z_t \in \mathcal{X} \times [K]$;
2. the oracle $\mathcal{O}_{sq}$ predicts a value $\hat{y}_t \in [0, 1]$;
3. the environment reveals the true value $y_t \in [0, 1]$.

By “solving”, we mean that $\mathcal{O}_{sq}$ satisfies the following regret guarantee in terms of square loss: for any (possibly adaptively chosen) sequence $z_{1:T}$ and $y_{1:T}$,

$$\sum_{t=1}^T (\hat{y}_t - y_t)^2 - \min_{f \in \mathcal{F}} \sum_{t=1}^T (f(z_t) - y_t)^2 \leq B(T) \quad (4)$$

for some regret bound $B(T) = o(T)$. Below are some examples (in fact, in all these examples, the oracle satisfies Eq. (4) even without the realizable assumption):

- **(Finite Class)** When $\mathcal{F}$ is finite, Hedge is one such oracle with regret $B(T) = \mathcal{O}(\sqrt{T \ln N})$. In fact, because of the nice properties of square loss (in particular its exp-concavity), it can be shown that Hedge with a constant learning rate achieves only $\mathcal{O}(\ln N)$ regret in this case.
- **(Linear Class)** When $\mathcal{F}$ is the linear model mentioned earlier: $\mathcal{F} = \{z = (x, a) \rightarrow \langle \theta, x^a \rangle \mid \theta \in \mathbb{R}^d, \|\theta\|_2 \leq 1\}$, the online regression problem is just an instance of OCO, and thus OGD is one such oracle with regret $B(T) = \mathcal{O}(\sqrt{T})$ as long as the L_2 norm of the contexts is bounded (recall that there is no d dependence at all). Again, thanks to the exp-concavity of square loss, there exist other algorithms (such as Online Newton Step [Hazan et al., 2007]) with $B(T) = \mathcal{O}(d \ln T)$ regret in this case. Importantly, these algorithms are all efficiently implementable (with $\text{poly}(d)$ dependence per round).

While there are not too many examples beyond the linear model where such an oracle can be implemented efficiently, the point is really that regression is so basic in machine learning practice and people have been successfully using complicated regressor class such as neural nets (by simply running gradient descent) to “solve” this problem already, without worrying about whether it truly enjoys low regret or not. Therefore, assuming such an oracle is practically reasonable. Doing so also allows any future advances in regression to be immediately translated to solving contextual bandits.

Algorithm Design. To solve contextual bandits using such an oracle, consider the following natural algorithm framework. At time t , given context x_t , ask the oracle $\mathcal{O}_{sq}$ to predict the loss for each of the K actions, denoted by $\hat{y}_t(1), \dots, \hat{y}_t(K)$. Then, based on these predictions, come up with a distribution $p_t \in \Delta(K)$, sample $a_t \sim p_t$, observe $\ell_t(a_t)$, and finally feed the instance (x_t, a_t) along with its true value $\ell_t(a_t)$ to the oracle for it to perform some internal updates. It remains to figure out the construction of p_t .

To this end, first observe that under this framework, in expectation the square loss regret of the oracle against the ground truth f^* can be written as:

$$\begin{aligned} & \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - \ell_t(a_t))^2 - \sum_{t=1}^T (f^*(x_t, a_t) - \ell_t(a_t))^2 \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))(\hat{y}_t(a_t) + f^*(x_t, a_t) - 2\ell_t(a_t)) \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))^2 \right] = \mathbb{E} \left[\sum_{t=1}^T \sum_{a=1}^K p_t(a) (\hat{y}_t(a) - f^*(x_t, a))^2 \right] \end{aligned} \quad (5)$$

where the second step is because $\ell_t(a_t)$ is in expectation equal to $f^*(x_t, a_t)$ due to the realizable assumption. On the other hand, in light of Eq. (2), the pseudo-regret of the learner is $\mathbb{E}[\sum_{t=1}^T (\sum_{a=1}^K p_t(a) f^*(x_t, a) - f^*(x_t, \pi^*(x_t)))]$. Therefore, one natural goal is to design p_t so that $\sum_{a=1}^K p_t(a) f^*(x_t, a) - f^*(x_t, \pi^*(x_t))$ is connected to $\sum_{a=1}^K p_t(a) (\hat{y}_t(a) - f^*(x_t, a))^2$, for example, in the form that the former is bounded by some fixed factor times the latter plus a small $o(1)$ term. Let this factor be $\gamma/4$ for some parameter $\gamma > 0$ to be specified later (the constant 4 is only for convenience as it will become clear). To understand the best way to design p_t and what the $o(1)$ small term is, we study the following optimization problem for any given $\hat{y} \in \mathbb{R}^K$:

$$\text{OPT}(\hat{y}) = \min_{p \in \Delta(K)} \max_{a^* \in [K]} \max_{\mu \in \mathbb{R}^K} \langle p - e_{a^*}, \mu \rangle - \frac{\gamma}{4} \|\hat{y} - \mu\|_{\text{diag}(p)}^2.$$

Here, $\hat{y}$ corresponds to the prediction vector of the oracle $\hat{y}_t$, p corresponds to the distribution p_t we want to find, a^* corresponds to the best action $\pi^*(x_t)$, $\mu \in \mathbb{R}^K$ corresponds to the true loss mean vector $f^*(x_t, \cdot)$, $e_{a^*} \in \mathbb{R}^K$ is the standard basis vector with the a^* -th coordinate being 1, and $\text{diag}(p)$ is a diagonal matrix with p on the diagonal. It is then clear that, if we let p_t to be the argmin of the problem defined in $\text{OPT}(\hat{y}_t)$, then

$$\sum_{a=1}^K p_t(a) f^*(x_t, a) - f^*(x_t, \pi^*(x_t)) \leq \frac{\gamma}{4} \sum_{a=1}^K p_t(a) (\hat{y}_t(a) - f^*(x_t, a))^2 + \text{OPT}(\hat{y}_t),$$

and thus the overall regret is connected to the square loss regret of the oracle as:

$$\bar{\mathcal{R}}_T \leq \mathbb{E} \left[\frac{\gamma}{4} \sum_{t=1}^T \sum_{a=1}^K p_t(a) (\hat{y}_t(a) - f^*(x_t, a))^2 + \sum_{t=1}^T \text{OPT}(\hat{y}_t) \right] \leq \frac{\gamma}{4} B(T) + \mathbb{E} \left[\sum_{t=1}^T \text{OPT}(\hat{y}_t) \right], \quad (6)$$

where the second step uses Eq. (5) and the assumption of the oracle Eq. (4). It now all boils down to solving $\text{OPT}(\hat{y})$. Interestingly, the solution turns out to be connected to FTRL with learning rate γ and a special regularizer known as *log barrier*.

Lemma 1. For any $\hat{y} \in \mathbb{R}^K$, we have $\text{OPT}(\hat{y}) = \frac{K-1}{\gamma}$, and the corresponding minimizer is $p^* = \text{argmin}_{p \in \Delta(K)} \langle p, \hat{y} \rangle + \frac{1}{\gamma} \sum_{a=1}^K \ln \frac{1}{p(a)}$, or equivalently $p^*(a) = \frac{1}{\gamma(\hat{y}(a) + \lambda)}$ where λ is such that p^* is a distribution (and can be found efficiently using a binary search).

Proof. Let's first fix p and a^* , and work on the innermost problem: $\max_{\mu \in \mathbb{R}^K} \langle p - e_{a^*}, \mu \rangle - \frac{\gamma}{4} \|\hat{y} - \mu\|_{\text{diag}(p)}^2$. This is simply a quadratic in μ , with maximizer $\mu^* = \frac{2}{\gamma} \text{diag}(p)^{-1} (p - e_{a^*}) + \hat{y}$ (verify it by setting the gradient to zero). The maximum is

$$\frac{1}{\gamma} \|p - e_{a^*}\|_{\text{diag}(p)^{-1}}^2 + \langle p - e_{a^*}, \hat{y} \rangle = \frac{1}{\gamma} \sum_{a=1}^K \frac{(p(a) - \mathbf{1}\{a = a^*\})^2}{p(a)} + \langle p - e_{a^*}, \hat{y} \rangle$$

$$\begin{aligned}
&= \frac{1}{\gamma} \sum_{a=1}^K \left(p(a) - 2\mathbf{1}\{a = a^*\} + \frac{\mathbf{1}\{a = a^*\}}{p(a)} \right) + \langle p - e_{a^*}, \hat{y} \rangle \\
&= \frac{1}{\gamma} \left(\frac{1}{p(a^*)} - 1 \right) + \langle p - e_{a^*}, \hat{y} \rangle.
\end{aligned}$$

Then, we focus on solving the rest of the problem:

$$\min_{p \in \Delta(K)} \max_{a^* \in [K]} \frac{1}{\gamma} \left(\frac{1}{p(a^*)} - 1 \right) + \langle p - e_{a^*}, \hat{y} \rangle. \quad (7)$$

We claim that the optimal p is such that the objective above is the same for all a^* (in which case the part $\max_{a^* \in [K]}$ becomes trivial). To see this, simply note that

$$\begin{aligned}
\max_{a^* \in [K]} \frac{1}{\gamma} \left(\frac{1}{p(a^*)} - 1 \right) + \langle p - e_{a^*}, \hat{y} \rangle &\geq \mathbb{E}_{a^* \sim p} \left[\frac{1}{\gamma} \left(\frac{1}{p(a^*)} - 1 \right) + \langle p - e_{a^*}, \hat{y} \rangle \right] \\
&= \frac{1}{\gamma} \sum_{a=1}^K p(a) \left(\frac{1}{p(a)} - 1 \right) = \frac{K-1}{\gamma}, \quad (\mathbb{E}_{a^* \sim p} [\langle p - e_{a^*}, \hat{y} \rangle] = 0)
\end{aligned}$$

and the inequality becomes equality when $\frac{1}{\gamma} \left(\frac{1}{p(a^*)} - 1 \right) + \langle p - e_{a^*}, \hat{y} \rangle$ is the same for all a^* . Ignoring all terms that are independent of a^* , we see that this is the same as saying that p^* should make sure $\frac{1}{\gamma p(a^*)} - \hat{y}(a^*) = \lambda$ for all $a^* \in [K]$, where λ is such that p^* is a valid distribution. This is the same as $p^* = \operatorname{argmin}_{p \in \Delta(K)} \langle p, \hat{y} \rangle + \frac{1}{\gamma} \sum_{a=1}^K \ln \frac{1}{p(a)}$ since it is exactly equivalent to setting the gradient of the Lagrangian of this FTRL objective to zero. $\square$

The FTRL form of the solution is quite intriguing — instead of using the cumulative estimated losses in FTRL (which does not really make sense for contextual bandits), here we use the predicted losses from the regression oracle, importantly with a special log-barrier regularizer. The analysis also becomes quite different as we have seen. In fact, another difference is that there is no need to solve FTRL very accurately — any approximate solution to $\operatorname{OPT}(\hat{y})$ or equivalently Eq. (7) with a constant approximation factor suffices. We provide one such approximate solution below, which is only two times worse compared to the optimal one but importantly enjoys a closed form.

Lemma 2. *For any $\hat{y} \in \mathbb{R}^K$, let $b = \operatorname{argmin}_a \hat{y}(a)$ and $\hat{p} \in \Delta(K)$ be such that $\hat{p}(a) = \frac{1}{\gamma(\hat{y}(a) - \hat{y}(b)) + K}$ for $a \neq b$ (the definition of $\hat{p}(b)$ is implicit via $1 - \sum_{a \neq b} \hat{p}(a)$). Then $\hat{p}$ approximately solves Eq. (7) with an approximation factor of 2.*

Proof. This is by direct calculations. Fix any a^* , let's first work on the part $\langle \hat{p} - e_{a^*}, \hat{y} \rangle$:

$$\begin{aligned}
\langle \hat{p} - e_{a^*}, \hat{y} \rangle &= \sum_{a \neq a^*} \hat{p}(a)(\hat{y}(a) - \hat{y}(a^*)) \\
&= (1 - \hat{p}(a^*))(\hat{y}(b) - \hat{y}(a^*)) + \sum_{a \notin \{a^*, b\}} \hat{p}(a)(\hat{y}(a) - \hat{y}(b)) \\
&= \hat{y}(b) - \hat{y}(a^*) + \sum_{a \neq b} \hat{p}(a)(\hat{y}(a) - \hat{y}(b)) \leq \hat{y}(b) - \hat{y}(a^*) + \frac{K-1}{\gamma},
\end{aligned}$$

where the last step uses the definition of $\hat{p}(a)$ for $a \neq b$. Therefore, the objective in Eq. (7) is at most

$$\frac{1}{\gamma} \left(\frac{1}{\hat{p}(a^*)} - 1 \right) + \hat{y}(b) - \hat{y}(a^*) + \frac{K-1}{\gamma}.$$

When $a^* = b$, this is at most $\frac{2(K-1)}{\gamma}$ since $\hat{p}(b) \geq 1/K$; when $a^* \neq b$, plugging in the definition of $\hat{p}(a^*) = \frac{1}{\gamma(\hat{y}(a^*) - \hat{y}(b)) + K}$, we get exactly $\frac{2(K-1)}{\gamma}$. Recalling that the optimal value is $\frac{(K-1)}{\gamma}$ according to Lemma 1 finishes the proof. $\square$

Based on all these discussions, we summarize the final algorithm, called SquareCB [Foster and Rakhlin, 2020], in the pseudocode below, followed by its formal regret guarantees.

Algorithm 2: SquareCB

Input: online regression oracle $\mathcal{O}_{sq}$ and parameter $\gamma > 0$

for $t = 1, \dots, T$ **do**

 receive context x_t

 ask $\mathcal{O}_{sq}$ to predict the loss for each action, denoted by $\hat{y}_t(1), \dots, \hat{y}_t(K)$

 compute $p_t \in \Delta(K)$ such that

$$\begin{cases} \text{Option I:} & p_t(a) = \frac{1}{\gamma(\hat{y}_t(a) + \lambda)} \text{ for all } a, \text{ where } \lambda \text{ is found via a binary search} \\ \text{Option II:} & p_t(a) = \frac{1}{\gamma(\hat{y}_t(a) - \min_b \hat{y}_t(b) + K)} \text{ for } a \neq \operatorname{argmin}_b \hat{y}_t(b) \end{cases}$$

 sample $a_t \sim p_t$, observe $\ell_t(a_t)$, and feed $((x_t, a_t), \ell_t(a_t))$ to $\mathcal{O}_{sq}$

Theorem 2. Under assumption (4), SquareCB (either option) ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\gamma B(T) + \frac{TK}{\gamma})$, which is $\mathcal{O}(\sqrt{TB(T)K})$ after picking the optimal γ .

Proof. This is by directly combining Eq. (6) and Lemma 1 for Option I or Lemma 2 for Option II. $\square$

Note that the bound $\mathcal{O}(\sqrt{TB(T)K})$ is always sublinear as $B(T) = o(T)$. Applying this result to the earlier examples, we get the following conclusions:

- **(Finite Class)** Recall that Hedge with a constant learning rate ensures $B(T) = \ln N$ for a finite regressor class. Thus, SquareCB with this oracle ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\sqrt{TK \ln N})$, exactly matching that of Exp4 (but note that the former only works for the realizable setting).
- **(Linear Class)** For the linear regressor class, if we use OGD as the oracle, then $B(T) = \mathcal{O}(\sqrt{T})$ and SquareCB ensures $\bar{\mathcal{R}}_T = \mathcal{O}(T^{3/4} \sqrt{K})$. While this has a worse dependence on T compared to LinUCB's regret $\tilde{\mathcal{O}}(d\sqrt{T})$, note that it has no dependence on d at all and is thus suitable for high-dimensional problems as long as the number of actions K is not too large. Another benefit is that the resulting algorithm is much more efficient than LinUCB since its per-round time complexity is linear in d while LinUCB requires at least d^2 time. If we instead use the aforementioned Online Newton Step as the oracle with $B(T) = \mathcal{O}(d \ln T)$, then SquareCB ensures $\tilde{\mathcal{O}}(\sqrt{dT K})$, closer to what LinUCB achieves.

Robustness to Misspecification. While we have focused on the realizable setting, SquareCB turns out to be highly robust to misspecification. Recall that in this case, $|f^*(x_t, a) - \mathbb{E}[\ell_t(a)]| \leq \epsilon$ holds for some misspecification level $\epsilon \in [0, 1]$. By just slightly adapting the analysis, we obtain:

Theorem 3. In the ϵ -misspecification setting, SquareCB (with either option) ensures $\bar{\mathcal{R}}_T = \mathcal{O}(\gamma(B(T) + \epsilon^2 T) + \frac{TK}{\gamma} + \epsilon T)$, which is $\mathcal{O}(\sqrt{TB(T)K} + \epsilon T \sqrt{K})$ with the optimal γ .

Proof. First, revisiting how we obtained Eq. (2) from Eq. (1), it is clear that with misspecification, the regret should become $\bar{\mathcal{R}}_T \leq \mathbb{E} \left[\sum_{t=1}^T f^*(x_t, a_t) - \sum_{t=1}^T f^*(x_t, \pi^*(x_t)) \right] + 2\epsilon T$. Then, we revise Eq. (5) as

$$\begin{aligned} & \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - \ell_t(a_t))^2 - \sum_{t=1}^T (f^*(x_t, a_t) - \ell_t(a_t))^2 \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))(\hat{y}_t(a_t) + f^*(x_t, a_t) - 2\ell_t(a_t)) \right] \\ &\geq \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))^2 \right] - 2\epsilon \mathbb{E} \left[\sum_{t=1}^T |\hat{y}_t(a_t) - f^*(x_t, a_t)| \right] \\ &\geq \frac{1}{2} \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))^2 \right] - 2\epsilon^2. \quad (\alpha\beta \leq \alpha^2 + \frac{1}{4}\beta^2) \end{aligned}$$

Finally, by Lemma 1 or Lemma 2, the way we select p_t still ensures:

$$\mathbb{E} \left[\sum_{t=1}^T f^*(x_t, a_t) - \sum_{t=1}^T f^*(x_t, \pi^*(x_t)) \right] \leq \frac{\gamma}{4} \mathbb{E} \left[\sum_{t=1}^T (\hat{y}_t(a_t) - f^*(x_t, a_t))^2 \right] + \mathcal{O} \left(\frac{TK}{\gamma} \right).$$

Combining all steps proves $\bar{\mathcal{R}}_T = \mathcal{O}(\gamma(B(T) + \epsilon^2 T) + \frac{TK}{\gamma} + \epsilon T)$. $\square$

While paying additional $\mathcal{O}(\epsilon T)$ regret is very natural in the ϵ -misspecification setting, one should not take such robustness guarantee for granted. For example, it can be shown that LinUCB is not robust and could suffer linear regret for some $\epsilon = o(1)$; see [Lattimore et al., 2020].

4 Offline Oracles

We briefly mention two other types of oracle and the corresponding approaches. While the online regression oracle is natural, it is after all solving a non-trivial online problem. An even more basic oracle would be one that only solves the corresponding *offline* problem. For example, an offline regression oracle solves the following problem

$$\operatorname{argmin}_{f \in \mathcal{F}} \sum_{(x,a,y) \in \mathcal{S}} (f(x,a) - y)^2$$

for any given dataset $\mathcal{S} \subseteq \mathcal{X} \times [K] \times [0, 1]$. In other words, it solves the problem defining the benchmark in Eq. (4), which is indeed a fairly standard offline regression problem. In light of SquareCB, a natural approach to solve contextual bandit using this oracle would be: at each time t , feed the previous observed data to the oracle, use the returned regressor to predict the loss of each action given context x_t , and finally construct a distribution $p_t \in \Delta(K)$ in the same way as SquareCB, from which a_t is sampled. This algorithm was introduced by Simchi-Levi and Xu [2021], and it enjoys $\tilde{\mathcal{O}}(\sqrt{TK \ln N})$ regret for a finite $\mathcal{F}$, almost the same as SquareCB, except that the analysis relies on an additional assumption that the contexts $x_1, \dots, x_T$ are drawn from a fixed and unknown distribution (while SquareCB allows them to be even chosen by an adaptive adversary). It was also shown that there is no need to call the oracle at every round; in fact, only $\mathcal{O}(\ln(\ln T))$ total number of calls over T rounds is needed to achieve the same regret bound.

The oracle-based approaches discussed so far only work for the realizable setting. To handle the agnostic setting with an arbitrary policy class Π , another type of offline oracle was proposed, which solves the following *cost-sensitive classification* problem:

$$\operatorname{argmin}_{\pi \in \Pi} \sum_{(x,\ell) \in \mathcal{S}} \ell(\pi(x))$$

for any given dataset $\mathcal{S} \subset \mathcal{X} \times \mathbb{R}^K$. This is a cost-sensitive classification problem since each policy π is essentially predicting the “class” for each context x out of K possibilities, and the cost for predicting each class is different as specified by the cost vector ℓ . This is also just the problem defining the benchmark in the regret definition of Eq. (1).

To solve contextual bandits using this classification oracle, earlier works consider a stochastic setting where $(x_1, \ell_1), \dots, (x_T, \ell_T)$ are independent samples of a fixed joint distribution over $\mathcal{X} \times [0, 1]^K$. To construct a dataset fed to the oracle by only observing x_t and $\ell_t(a_t)$ at time t , it is natural to construct the same importance weighted estimator $\hat{\ell}_t \in \mathbb{R}_+^K$ as in Exp3/Exp4. Suppose that at time t , we feed $(x_1, \hat{\ell}_1), \dots, (x_{t-1}, \hat{\ell}_{t-1})$ to the oracle and obtain π_t as the returned policy. Simply picking $a_t = \pi_t(x_t)$ is a bad idea since there is no exploration at all (also note that randomness is required for constructing importance weighted estimators). Instead, one can set aside a small probability to uniformly explore all actions (and pick $\pi_t(x_t)$ with the rest of the probability). Because of this simple exploration scheme, however, it is not difficult to show that this approach at best achieves $\tilde{\mathcal{O}}(T^{2/3}(K \ln N)^{1/3})$ regret.

How to improve the regret to $\tilde{\mathcal{O}}(\sqrt{TK \ln N})$ was answered by the work [Agarwal et al., 2014]. Interestingly, the solution is again related to FTRL with the log-barrier regularizer (although not

explicitly mentioned in the original work). Specifically, at time t , we find a distribution P_t over the policies via:

$$P_t = \operatorname{argmin}_{P \in \Delta(N)} \left\langle P, \sum_{s < t} \widehat{\ell}_s \right\rangle + \frac{1}{\gamma} \sum_{s < t} \sum_{a=1}^K \ln \frac{1}{P(a|x_s)}$$

where we recall the overloading of the notation $\widehat{\ell}_s \in \mathbb{R}_+^N$ used earlier for Exp4 (so that $\widehat{\ell}_s(\pi) = \widehat{\ell}_s(\pi(x_s))$), and use $P(a|x)$ to denote $\sum_{\pi: \pi(x)=a} P(\pi)$. With this distribution P_t , we simply sample a_t from $P_t(\cdot|x_t)$. The only difference between Exp4 and this algorithm is in the regularizer — Exp4 uses the entropy regularizer in the policy space: $\sum_{\pi} P(\pi) \ln P(\pi)$, while this algorithm uses a log-barrier regularizer in the action space induced by the previous observed contexts: $\sum_{s < t} \sum_{a=1}^K \ln \frac{1}{P(a|x_s)}$. It is this importance difference that makes this algorithm efficiently implementable, even though it is written in the space of the policy class and a naive implementation would require at least $\mathcal{O}(N)$ time. Specifically, it turns out that one can find an approximate and, more importantly, *sparse* solution to the FTRL using a only few number of calls to the classification oracle (sparse in the sense that the size of the support of P_t only depends on N logarithmically).

[Agarwal et al., 2014] shows that the regret of this algorithm is $\tilde{\mathcal{O}}(\sqrt{TK \ln N})$ (again, comparable to Exp4 except that Exp4 works for the adversarial setting). The analysis is yet again very different from the standard FTRL analysis covered in this course; see [Luo, 2017] for a short summary. The fact that the log-barrier regularizer plays an important (but somewhat different) role in both the setting with a regression oracle and that with a classification oracle is intriguing. In HW3, you will also see more applications of this regularizer.

References

- Alekh Agarwal, Daniel Hsu, Satyen Kale, John Langford, Lihong Li, and Robert Schapire. Taming the monster: A fast and simple algorithm for contextual bandits. In *Proceedings of the 31st International Conference on Machine Learning*, 2014.
- Peter Auer, Nicolò Cesa-Bianchi, Yoav Freund, and Robert E Schapire. The nonstochastic multi-armed bandit problem. *SIAM Journal on Computing*, 32(1):48–77, 2002.
- Dylan Foster and Alexander Rakhlin. Beyond ucb: Optimal and efficient contextual bandits with regression oracles. In *International Conference on Machine Learning*, pages 3199–3210. PMLR, 2020.
- Elad Hazan, Amit Agarwal, and Satyen Kale. Logarithmic regret algorithms for online convex optimization. *Machine Learning*, 69(2-3):169–192, 2007.
- Tor Lattimore, Csaba Szepesvari, and Gellert Weisz. Learning with good feature representations in bandits and in rl with a generative model. In *International Conference on Machine Learning*, pages 5662–5670. PMLR, 2020.
- Haipeng Luo. Lecture notes 21, introduction to online learning, 2017. URL <https://haipeng-luo.net/courses/CSCI699/lecture21.pdf>.
- Yevgeny Seldin and Gábor Lugosi. A lower bound for multi-armed bandits with expert advice. In *13th European Workshop on Reinforcement Learning (EWRL)*, 2016.
- David Simchi-Levi and Yunzong Xu. Bypassing the monster: A faster and simpler optimal algorithm for contextual bandits under realizability. *Mathematics of Operations Research*, 2021.

CSCI 659 Lecture 9

Fall 2022

Instructor: Haipeng Luo

1 Adversarial Bandit Linear/Convex Optimization

So far most of our discussions on bandit problems focused on a set of discrete actions. Put differently, the decision set of the learner is always a simplex. In this lecture, we go back to the general adversarial OCO setting with a general compact convex decision set $\Omega \subseteq \mathbb{R}^d$ and study bandit feedback in this setting. Concretely, consider the following adversarial Bandit Convex Optimization (BCO) problem: at each round $t = 1, \dots, T$,

1. the learner decides an action $w_t \in \Omega$ while simultaneously the adversary decides a convex loss function $f_t : \Omega \rightarrow [-1, 1]$;
2. the learner suffers and observes (only) $f_t(w_t)$.

We again assume that the adversary is oblivious for simplicity. The learner's goal is to minimize her expected regret against the best fixed action: $\mathbb{E}[\mathcal{R}_T] = \mathbb{E}[\sum_{t=1}^T f_t(w_t)] - \sum_{t=1}^T f_t(w^*)$ where $w^* \in \operatorname{argmin}_{w \in \Omega} \sum_{t=1}^T f_t(w)$ and the expectation is with respect to the learner's internal randomness.

While in the full-information setting, we argue that considering only linear loss functions is without loss of generality via the linearization trick: $f_t(w_t) - f_t(w^*) \leq \langle \nabla f_t(w_t), w_t - w^* \rangle$, this is no longer true in the bandit setting since our observation is only $f_t(w_t)$, instead of $\langle \nabla f_t(w_t), w_t \rangle$. For this reason, BCO is an extremely challenging problem, even for the simpler case where all loss functions are drawn from a fixed distribution, in which case the problem is also known under many different names such as blackbox/zeroth order/derivative-free optimization and has many applications in practice.

We will therefore start by considering the special case, Bandit Linear Optimization (BLO), where each loss function f_t is a linear function parameterized by a vector $\ell_t \in \mathbb{R}^d$, that is, $f_t(w) = \langle w, \ell_t \rangle$. This still captures many interesting and important applications. For example, consider the bandit version of the combinatorial problems discussed in Lecture 2 (also known as *combinatorial bandits*), where there is a set of combinatorial actions $A = \{a_1, \dots, a_K\} \subseteq \{0, 1\}^d$ and picking action a at time t incurs loss $\langle a, \ell_t \rangle$ for some loss vector ℓ_t , which is also the only observation for the learner. We have discussed examples such as m -set where each action corresponds to picking exactly m out of d items (e.g. recommending m out of d products to the customer), or online shortest path where each action corresponds to picking one path of a given graph (e.g. deciding which route to commute to work each day). The bandit feedback fits particularly well for the online shortest path example since most often we only observe/record the total loss (travel time) of the selected path.

To solve combinatorial bandits using BLO, one can take Ω as the convex hull of A as we did in Lecture 2. The only extra subtlety is that after the BLO algorithm selects $w_t \in \Omega$, if w_t is not already one of the combinatorial actions, we need to sample $a_t \in A$ with expectation w_t (recall that w_t , being a point in the convex hull of A , exactly corresponds to a distribution over the elements in A). This makes the feedback to the learner $\langle a_t, \ell_t \rangle$, instead of $\langle w_t, \ell_t \rangle$ as the protocol of BLO specifies. As we will see, however, this will not be an issue for the algorithms we consider. This also makes MAB a special case of BLO with $A = \{e_1, \dots, e_K\}$ (the set of standard basis vectors).

Compared to the stochastic linear bandit problem discussed in Lecture 7, the key difference here is that the parameter deciding the loss (i.e. ℓ_t) is changing over time arbitrarily, but the action set is fixed and we care about competing to the overall best fixed action, while in stochastic linear bandits we consider a fixed parameter θ , allow the action set to be changing over time arbitrarily, and compare to the best action at each time.

2 The Exp2 Algorithm for BLO

We start with an inefficient but optimal algorithm that operates over a discrete subset A of Ω of size K and in a sense treats BLO as a K -armed bandit problem with a linear structure. This subset A can be obtained by discretizing Ω so that any two points in A are $\frac{1}{T}$ -close (say in terms of L_2 norm), in which case K is of order $\mathcal{O}(T^d)$ and the extra regret introduced by this discretization is only $\mathcal{O}(1)$. On the other hand, if Ω is itself already a convex hull of a discrete set, which is the case for combinational bandits for example, then we can directly take this set as A since in this case the best action w^* can always be selected from A (the minimum of a linear function over a polytope can always be achieved by one of its corners). We define $B = \max_{a \in A} \|a\|_2$ as the largest size of these discrete actions, and also assume without loss of generality that A is full rank (since otherwise we can first project them onto a full-rank subspace with lower dimension).

If we simply treat this as a standard K -armed bandit problem, then the regret is $\mathcal{O}(\sqrt{TK})$, clearly unacceptable since K can be exponentially large as mentioned. The issue of this approach is that it completely ignores the linear structure of the losses. As the simplest example, if A contains two actions a and $2a$, then no matter what ℓ_t is, knowing one action's loss completely reveals the loss for the other. In other words, similar to the case for Exp4, bandit feedback here does not really mean only $1/K$ fraction of information is available. Instead, since there are at most d independent directions in $\mathbb{R}^d$, bandit feedback should be thought of as having only $1/d$ fraction of information.

To make use of this structure, we will directly construct a loss estimator $\hat{\ell}_t \in \mathbb{R}^d$ for ℓ_t , and then estimate the loss for each action $a \in A$ naturally as $\langle a, \hat{\ell}_t \rangle$. Having these estimators for all actions, we use Hedge to come up with a distribution $p_{t+1} \in \Delta(K)$ based on $p_{t+1}(a) \propto \exp(-\eta \sum_{s \leq t} \langle a, \hat{\ell}_s \rangle)$, and sample a_{t+1} from p_{t+1} . It remains to figure out how to construct $\hat{\ell}_t$.

Unlike the stochastic linear bandit problem where ℓ_t is fixed and can be estimated by standard linear regression based on the past t observations, here, ℓ_t can be arbitrarily changing over time and we have only one sample $\langle a_t, \ell_t \rangle$. Thanks to the randomness in selecting a_t , however, it is in fact possible to do a "one-point regression" by imagining having K samples, each with probability $p_t(a)$. More specifically, we construct the estimator as:

$$\hat{\ell}_t = M_t^{-1} a_t a_t^\top \ell_t \quad \text{where} \quad M_t = \sum_{a \in A} p_t(a) a a^\top = \mathbb{E}_{a \sim p_t} [a a^\top] \quad (1)$$

is the covariance matrix with respect to p_t . Note that 1) although ℓ_t appears in this formula, the dependence is only through $a_t^\top \ell_t$, a quantity that we indeed observe; and 2) M_t is indeed invertible since A is full rank and p_t , computed based on the exponential weight, has a full support. In fact, when $A = \{e_1, \dots, e_K\}$, this exactly recovers the importance weighted estimator for MAB (verify it yourself). The following lemma shows that this estimator is not only unbiased, but also leads to a nice bound for the local-norm term of Hedge.

Lemma 1. *For any distribution $p_t \in \Delta(K)$ with a full support, let $\hat{\ell}_t$ be the loss estimator defined in Eq. (1) where a_t is sampled from p_t . Then we have (expectations below are with respect to $a_t \sim p_t$)*

$$\mathbb{E}[\hat{\ell}_t] = \ell_t \quad \text{and} \quad \mathbb{E} \left[\sum_{a \in A} p_t(a) \langle a, \hat{\ell}_t \rangle^2 \right] \leq d.$$

Proof. Direct calculations show: $\mathbb{E}[\hat{\ell}_t] = M_t^{-1} \mathbb{E}[a_t a_t^\top] \ell_t = M_t^{-1} M_t \ell_t = \ell_t$, and

$$\mathbb{E} \left[\sum_{a \in A} p_t(a) (a^\top \hat{\ell}_t)^2 \right] = \sum_{a \in A} p_t(a) \mathbb{E}[(a^\top \ell_t)^2 a^\top M_t^{-1} a_t a_t^\top M_t^{-1} a]$$

$$\begin{aligned}
&\leq \sum_{a \in A} p_t(a) a^\top M_t^{-1} \mathbb{E}[a_t a_t^\top] M_t^{-1} a = \sum_{a \in A} p_t(a) a^\top M_t^{-1} a \\
&= \sum_{a \in A} p_t(a) \text{TR}(a^\top M_t^{-1} a) = \sum_{a \in A} p_t(a) \text{TR}(a a^\top M_t^{-1}) = \text{TR}(M_t M_t^{-1}) = d,
\end{aligned}$$

where the inequality is by the fact $|a^\top \ell_t| \leq 1$ for all a (coming from the assumption that the range of f_t is in $[-1, 1]$ in the BCO problem description). $\square$

Therefore, if we still have the following local-norm regret bound from Hedge: for any a^* ,

$$\sum_{t=1}^T \sum_{a \in A} p_t(a) \langle a, \hat{\ell}_t \rangle - \sum_{t=1}^T \langle a^*, \hat{\ell}_t \rangle \leq \frac{\ln K}{\eta} + \eta \sum_{t=1}^T \sum_{a \in A} p_t(a) \langle a, \hat{\ell}_t \rangle^2, \quad (2)$$

then taking expectation on both sides would imply a regret bound of $\mathcal{O}(\sqrt{dT \ln K})$ after optimally tuning η , much better than the aforementioned $\mathcal{O}(\sqrt{TK})$ bound. However, there is one caveat: recall that Eq. (2) holds only when $\eta \langle a, \hat{\ell}_t \rangle \geq -1$ holds for all a and t . This condition trivially holds for Exp3/Exp4, since in MAB or contextual bandits the loss estimators are always nonnegative. On the other hand, $\langle a, \hat{\ell}_t \rangle = a^\top M_t^{-1} a_t a_t^\top \ell_t$ can now be very negative and violate the condition if a is a direction that has small correlation with a_t with high probability (this is true even if we assume $\langle a, \ell_t \rangle \geq 0$ for all $a \in A$). Also recall that this is not just a technical requirement in the analysis, but is in fact related to the necessity of exploration as discussed in Lecture 6: Hedge with a loss estimator that is too negative will discourage exploration.

To address this issue, we modify the algorithm slightly and explicitly enforce a small amount of exploration. Specifically, let γ be the probability of performing explicit exploration and $q \in \Delta(K)$ be an exploration distribution over A to be specified later. We now redefine p_t as $(1 - \gamma)p'_t + \gamma q$ where p'_t is the Hedge distribution with $p'_t(a) \propto \exp(-\eta \sum_{s < t} \langle a, \hat{\ell}_s \rangle)$, and then sample a_t from p_t and construct estimator $\hat{\ell}_t$ the same way as Eq. (1). The resulting algorithm is called by many names such as Exp2 (Expanded Exponential weight) or GeometricHedge [Dani et al., 2008, Cesa-Bianchi and Lugosi, 2012, Bubeck et al., 2012] and is summarized below.

Algorithm 1: Exp2

Input: learning rate $\eta > 0$, exploration probability $\gamma \in (0, 1)$ and distribution $q \in \Delta(K)$

for $t = 1, \dots, T$ **do**

compute $p'_t \in \Delta(K)$ such that $p'_t(a) \propto \exp(-\eta \sum_{s < t} \langle a, \hat{\ell}_s \rangle)$
 sample a_t from $p_t = (1 - \gamma)p'_t + \gamma q$
 observe $\langle a_t, \ell_t \rangle$ and construct $\hat{\ell}_t = M_t^{-1} a_t a_t^\top \ell_t$ where $M_t = \sum_{a \in A} p_t(a) a a^\top$

The following lemma tells us what property we need from the exploration distribution q .

Lemma 2. *Let $\lambda_{\min}$ be the minimum eigenvalue of the covariance matrix $\mathbb{E}_{a \sim q}[a a^\top]$ of the exploration distribution. If $\eta \leq \frac{\gamma \lambda_{\min}}{B^2}$, then $\eta |\langle a, \hat{\ell}_t \rangle| \leq 1$ holds for all a and t .*

Proof. Note that the covariance matrix M_t with respect to p_t is now $(1 - \gamma)\mathbb{E}_{a \sim p'_t}[a a^\top] + \gamma \mathbb{E}_{a \sim q}[a a^\top]$, and thus its minimum eigenvalue is at least $\gamma \lambda_{\min}$. Therefore, we have

$$\begin{aligned}
|\langle a, \hat{\ell}_t \rangle| &= |a^\top M_t^{-1} a_t| |a_t^\top \ell_t| \leq |a^\top M_t^{-1} a_t| = |a^\top M_t^{-1/2} M_t^{-1/2} a_t| \\
&\leq \sqrt{a^\top M_t^{-1} a} \cdot \sqrt{a_t^\top M_t^{-1} a_t} \leq \frac{B}{\sqrt{\gamma \lambda_{\min}}} \cdot \frac{B}{\sqrt{\gamma \lambda_{\min}}} = \frac{B^2}{\gamma \lambda_{\min}},
\end{aligned}$$

where the second inequality is by Cauchy-Schwarz inequality and the last inequality uses the definition of B and that the maximum eigenvalue of M_t^{-1} is at most $1/(\gamma \lambda_{\min})$. Using the condition on η then finishes the proof. $\square$

Therefore, as long as we pick η smaller than $\frac{\gamma \lambda_{\min}}{B^2}$, Eq. (2) (with p_t replaced by p'_t) holds and the analysis goes through. Note that the smaller the learning rate, the larger the term $\frac{\ln N}{\eta}$ in Eq. (2).

This motivates us to pick q such that $\lambda_{\min}$ is as large as possible, which makes sense since such q explores every direction in $\mathbb{R}^d$ with reasonable probability. The role of $\lambda_{\min}$ in the final regret bound is made clear in the following theorem.

Theorem 1. *If $\eta \leq \frac{\gamma \lambda_{\min}}{B^2}$, then Exp2 ensures $\mathbb{E}[\mathcal{R}_T] \leq \frac{\ln K}{\eta} + 2\gamma T + \eta T d$. Thus, setting $\gamma = \frac{B^2 \eta}{\lambda_{\min}}$ and $\eta = \min \left\{ \sqrt{\frac{\ln K}{(d+2B^2/\lambda_{\min})T}}, \frac{\lambda_{\min}}{B^2} \right\}$ leads to $\mathbb{E}[\mathcal{R}_T] = \mathcal{O} \left(\frac{B^2 \ln K}{\lambda_{\min}} + \sqrt{\left(\frac{2B^2}{\lambda_{\min}} + d \right) T \ln K} \right)$.*

Proof. By Lemma 2 and the analysis of Hedge, we have for any $a^* \in A$,

$$\sum_{t=1}^T \sum_{a \in A} p'_t(a) \langle a, \hat{\ell}_t \rangle - \sum_{t=1}^T \langle a^*, \hat{\ell}_t \rangle \leq \frac{\ln K}{\eta} + \eta \sum_{t=1}^T \sum_{a \in A} p'_t(a) \langle a, \hat{\ell}_t \rangle^2.$$

Plugging in $p'_t(a) = \frac{p_t(a) - \gamma q(a)}{1 - \gamma}$, multiplying both sides by $1 - \gamma$, and rearranging give

$$\begin{aligned} & \sum_{t=1}^T \sum_{a \in A} p_t(a) \langle a, \hat{\ell}_t \rangle - \sum_{t=1}^T \langle a^*, \hat{\ell}_t \rangle \\ & \leq \frac{(1 - \gamma) \ln K}{\eta} + \gamma \sum_{t=1}^T \sum_{a \in A} q(a) \langle a, \hat{\ell}_t \rangle - \gamma \sum_{t=1}^T \langle a^*, \hat{\ell}_t \rangle + \eta \sum_{t=1}^T \sum_{a \in A} (p_t(a) - \gamma q(a)) \langle a, \hat{\ell}_t \rangle^2 \\ & \leq \frac{\ln K}{\eta} + \gamma \sum_{t=1}^T \sum_{a \in A} q(a) \langle a, \hat{\ell}_t \rangle - \gamma \sum_{t=1}^T \langle a^*, \hat{\ell}_t \rangle + \eta \sum_{t=1}^T \sum_{a \in A} p_t(a) \langle a, \hat{\ell}_t \rangle^2. \end{aligned}$$

Taking expectation on both sides and using Lemma 1 and the fact $|\langle a, \ell_t \rangle| \leq 1$, we arrive at

$$\mathbb{E}[\mathcal{R}_T] = \mathbb{E} \left[\sum_{t=1}^T \langle a_t, \ell_t \rangle \right] - \min_{a \in A} \sum_{t=1}^T \langle a, \ell_t \rangle \leq \frac{\ln K}{\eta} + 2\gamma T + \eta T d,$$

proving the first statement. The second statement is by setting γ to make the condition $\eta \leq \frac{\gamma \lambda_{\min}}{B^2}$ an equality and then picking the optimal η . $\square$

2.1 Finding the Optimal Exploration Distribution

Once again, the regret bound in Theorem 1 suggests that we need to find an exploration distribution q with a large minimum eigenvalue $\lambda_{\min}$ for the covariance matrix $\mathbb{E}_{a \sim q}[aa^\top]$. Note that the sum of the eigenvalues of this matrix is $\text{TR}(\mathbb{E}_{a \sim q}[aa^\top]) = \mathbb{E}_{a \sim q}[\text{TR}(aa^\top)] \leq B^2$. Therefore, being the smallest eigenvalue, the largest possible $\lambda_{\min}$ we can hope for is $\Theta(B^2/d)$. It turns out that, after some preprocessing, one can indeed always find a q with special geometric properties such that $\lambda_{\min} = \Omega(B^2/d)$; see [Bubeck et al., 2012]. With such an optimal exploration scheme, the regret bound in Theorem 1 becomes $\mathcal{O}(d \ln K + \sqrt{dT \ln K})$ (note that the parameter B in fact does not play a role). Going back to the earlier discussion that K is $\mathcal{O}(T^d)$ when dealing with an arbitrary convex decision set Ω via discretization, we see that Exp2 enjoys $\tilde{\mathcal{O}}(d\sqrt{T})$ regret, which is known to be near optimal [Dani et al., 2008].

While we will not discuss how to find such optimal q , we point out that in many problems the uniform distribution over A is already good enough. We provide two such examples for combinatorial bandits (so $A \subseteq \{0, 1\}^d$) below, and will use the following observations to calculate $\lambda_{\min}$: $\lambda_{\min} = \min_{\|v\|_2=1} v^\top \mathbb{E}_{a \sim q}[aa^\top] v = \min_{\|v\|_2=1} \mathbb{E}_{a \sim q}[(a^\top v)^2]$ and

$$\begin{aligned} \mathbb{E}_{a \sim q}[(a^\top v)^2] &= \mathbb{E}_{a \sim q} \left[\left(\sum_{i=1}^d a(i)v(i) \right)^2 \right] = \mathbb{E}_{a \sim q} \left[\sum_{i=1}^d a(i)^2 v(i)^2 + \sum_{i \neq j} a(i)a(j)v(i)v(j) \right] \\ &= \sum_{i=1}^d \Pr(a(i) = 1)v(i)^2 + \sum_{i \neq j} \Pr(a(i) = a(j) = 1)v(i)v(j). \end{aligned}$$

Hypercube. The first example is when A is the entire hypercube $\{0, 1\}^d$. This corresponds to a setting where there are d items and each time we can pick any subset of them and observe the sum of the losses of the selected items. When q is the uniform distribution, we clearly have $\Pr(a(i) = 1) = 1/2$ and $\Pr(a(i) = a(j) = 1) = 1/4$, and thus for any v with $\|v\|_2 = 1$,

$$\mathbb{E}_{a \sim q} [(a^\top v)^2] = \frac{1}{2} \|v\|_2^2 + \frac{1}{4} \sum_{i \neq j} v(i)v(j) = \frac{1}{4} \|v\|_2^2 + \frac{1}{4} \left(\sum_{i=1}^d v(i) \right)^2 \geq \frac{1}{4}.$$

The minimum is achievable as long as $\sum_{i=1}^d v(i) = 0$, which means $\lambda_{\min}$ is exactly $1/4$ in this case. This is the ideal case since $B = \sqrt{d}$ and $\mathcal{O}(B^2/d) = \mathcal{O}(1)$. With this optimal exploration distribution, Exp2 achieves $\mathcal{O}(d\sqrt{T})$ regret (since $K = 2^d$).

m -sets. The next example is when $A = \{a \in \{0, 1\}^d : \|a\|_1 = m\}$, that is, each time we can only pick exactly m items. When q is the uniform distribution over A , we have $\Pr(a(i) = 1) = \binom{d-1}{m-1} / \binom{d}{m}$ and $\Pr(a(i) = a(j) = 1) = \binom{d-2}{m-2} / \binom{d}{m}$, and thus for any v with $\|v\|_2 = 1$,

$$\begin{aligned} \mathbb{E}_{a \sim q} [(a^\top v)^2] &= \frac{\binom{d-1}{m-1}}{\binom{d}{m}} \|v\|_2^2 + \frac{\binom{d-2}{m-2}}{\binom{d}{m}} \sum_{i \neq j} v(i)v(j) \\ &= \left(\frac{m}{d} - \frac{m(m-1)}{d(d-1)} \right) \|v\|_2^2 + \frac{m(m-1)}{d(d-1)} \left(\sum_{i=1}^d v(i) \right)^2 \geq \frac{m(d-m)}{d(d-1)}. \end{aligned}$$

This again shows that $\lambda_{\min} = \frac{m(d-m)}{d(d-1)}$. As long as $m \leq cd$ for some constant $c < 1$ (which is often the more realistic case), we have $\lambda_{\min} = \Omega(\frac{m}{d})$, the largest possible since $B^2/d = m/d$. With this optimal exploration distribution, Exp2 achieves $\mathcal{O}\left(\sqrt{dT \ln \binom{d}{m}}\right) = \mathcal{O}\left(\sqrt{dmT \ln \frac{d}{m}}\right)$ regret.

3 The SCRiBLE Algorithm for BLO

While Exp2 achieves the optimal regret, it is computationally inefficient since it explicitly maintains a distribution over potentially exponentially many actions. Similar to our discussion for combinatorial problems in Lecture 2, to obtain an efficient algorithm, we need to directly perform FTRL over the d -dimensional decision space Ω , that is, at time t compute $w_t = \operatorname{argmin}_{w \in \Omega} \langle w, \sum_{s < t} \hat{\ell}_s \rangle + \frac{1}{\eta} \psi(w)$ for some loss estimators $\hat{\ell}_1, \dots, \hat{\ell}_{t-1}$ and regularizer ψ . Note, however, that we cannot directly play w_t as the final decision, since randomness is required to construct the loss estimators as it should have become clear by now after seeing so many examples. Thus, another thing we need to figure out is how to randomly decide the final decision, denoted by $\tilde{w}_t \in \Omega$, based on the FTRL solution w_t . These three elements (random decision, loss estimators, and regularizer) are all tied together closely, and there happens to be a delicate combination of the three that makes thing work.

First, having w_t , we will randomly explore a local region centered at w_t . One possibility of this local region is just a small L_2 ball, but this does not take into account the “shape” of the decision set Ω at all. For example, if w_t is very close to the boundary of Ω , then this ball needs to be very small, limiting the exploration in all directions. Directly considering the shape of Ω , an arbitrary convex set, is indeed highly challenging. Instead, we will somehow let the regularizer take care of this and explore over the surface of an ellipsoid defined with respect to the local behavior of the regularizer. Specifically, we play $\tilde{w}_t = w_t + H_t^{-1/2} s_t$, where $H_t = \nabla^2 \psi(w_t)$ (invertible as long as ψ is strictly convex) and s_t is uniformly at random sampled from the d -dimensional sphere, denoted by $\mathbb{S}^d$. If H_t is the identity matrix, then $\|\tilde{w}_t - w_t\|_2 = 1$ and thus $\tilde{w}_t$ is exactly a uniform sample from the surface of a unit L_2 ball centered at w_t . More generally, for a positive definite H_t , we have $\|\tilde{w}_t - w_t\|_{H_t} = 1$ and thus $\tilde{w}_t$ is a uniform sample from the surface of an ellipsoid centered at w_t . The eigenvectors of H_t define the principal axes of this ellipsoid and the corresponding eigenvalues are the reciprocals of the square of the semi-axes. It is clear that $\mathbb{E}_t[\tilde{w}_t] = w_t$.

Of course, for this scheme to be valid, we need to make sure that $\tilde{w}_t$ is indeed within Ω , an issue that we will come back later. Assuming its validity, after playing $\tilde{w}_t$ and observing $\tilde{w}_t^\top \ell_t$ we construct

the loss estimator as $\widehat{\ell}_t = dH_t^{1/2} s_t \widetilde{w}_t^\top \ell_t$. This is in fact closely related to the estimator used in Exp2, since (expectation below is with respect to the randomness of s_t)

$$\left(\mathbb{E} [(\widetilde{w}_t - w_t)(\widetilde{w}_t - w_t)^\top] \right)^{-1} (\widetilde{w}_t - w_t) = \left(\mathbb{E} [H_t^{-1/2} s_t s_t^\top H_t^{-1/2}] \right)^{-1} H_t^{-1/2} s_t = dH_t^{1/2} s_t,$$

where we use the fact $\mathbb{E}[s_t s_t^\top] = \frac{1}{d} I_d$ (I_d is the d by d identity matrix). The lemma below shows that the estimator enjoys not only unbiasedness but also a small local norm.

Lemma 3. *The estimator defined above satisfies: $\mathbb{E}[\widehat{\ell}_t] = \ell_t$ and $\|\widehat{\ell}_t\|_{H_t^{-1}} \leq d$ where the expectation is with respect to the randomness of s_t .*

Proof. By direct calculations and the facts $\mathbb{E}[s_t] = \mathbf{0}$ and $\mathbb{E}[s_t s_t^\top] = \frac{1}{d} I_d$, we have

$$\mathbb{E} [\widehat{\ell}_t] = \mathbb{E} [dH_t^{1/2} s_t \widetilde{w}_t^\top \ell_t] + \mathbb{E} [dH_t^{1/2} s_t s_t^\top H_t^{-1/2} \ell_t] = \ell_t,$$

and

$$\|\widehat{\ell}_t\|_{H_t^{-1}}^2 = \widehat{\ell}_t^\top H_t^{-1} \widehat{\ell}_t = d^2 (\widetilde{w}_t^\top \ell_t)^2 s_t^\top H_t^{1/2} H_t^{-1} H_t^{1/2} s_t = d^2 (\widetilde{w}_t^\top \ell_t)^2 \leq d^2,$$

where in the last step we further use $s_t^\top s_t = 1$ and the assumption $|\widetilde{w}_t^\top \ell_t| \leq 1$. $\square$

We point out that, unlike all other local norm calculations we have seen, this one is bounded *always*, instead of only in expectation, and it holds for any strictly convex regularizer. As before, however, we still need to argue that the stability term of FTRL is indeed related to the local norm. This, together with the earlier issue on the validity of $\widetilde{w}_t$, can be simultaneously addressed by using a special type of regularizers called *self-concordant barriers*. Self-concordant barriers play a fundamental role in optimization theory (in particular, the Interior Point Method), and its (somewhat surprising) role for BLO was discovered by the seminal work by [Abernethy et al. \[2008\]](#), who proposed the following SCRiBL (Self-Concordant Regularization in Bandit Learning) algorithm.¹

Algorithm 2: SCRiBL

Input: learning rate $\eta > 0$ and a self-concordant barrier ψ for Ω

for $t = 1, \dots, T$ **do**

compute $w_t = \operatorname{argmin}_{w \in \Omega} \left\langle w, \sum_{s < t} \widehat{\ell}_s \right\rangle + \frac{1}{\eta} \psi(w)$

sample $s_t \in \mathbb{S}^d$ uniformly at random and play $\widetilde{w}_t = w_t + H_t^{-1/2} s_t$ where $H_t = \nabla^2 \psi(w_t)$

observe $\widetilde{w}_t^\top \ell_t$ and construct estimator $\widehat{\ell}_t = dH_t^{1/2} s_t \widetilde{w}_t^\top \ell_t$

A barrier on Ω is a function that approaches $+\infty$ on the boundary of Ω . We defer the formal definition of self-concordance to the end of the discussion and first list the following useful facts (all can be found in [\[Nesterov and Nemirovskii, 1994\]](#)).

Fact 1. *If ψ is a self-concordant barrier on Ω , then for any w in the interior of Ω , the ellipsoid $\{\widetilde{w} : \|\widetilde{w} - w\|_{\nabla^2 \psi(w)} \leq 1\}$, called the Dikin ellipsoid centered at w , is contained by Ω .*

Since ψ is a barrier, the FTRL solution w_t is always in the interior of Ω . This tells us that $\widetilde{w}_t$, being on the surface of the Dikin ellipsoid center at w_t , is indeed a valid decision. Compared to simply exploring a small ball centered at w_t , the Dikin ellipsoid can make much better use of the space and explore more adaptively and aggressively.

Fact 2. *Let ψ be a self-concordant barrier on Ω and w^* be its minimizer. For any $w \in \Omega$, if its Newton decrement $\lambda_\psi(w)$, defined as $\|\nabla \psi(w)\|_{\nabla^{-2} \psi(w)}$, is at most $1/2$, then $\|w - w^*\|_{\nabla^2 \psi(w)} \leq 2\lambda_\psi(w)$.*

This fact says that by looking at the Newton decrement of w , which is the local norm of the gradient of w , one can tell how far away w is from the minimizer w^* (as long as this Newton decrement is not too vacuously large). This fact helps us relate the stability of FTRL to the local norm of the loss estimator, as shown in the following lemma.

¹This version is slightly different from their original algorithm which samples s_t from the eigenbasis of H_t instead, but this makes no real difference to the regret analysis.

Lemma 4. If $\eta \leq \frac{1}{2d}$, SCRiBLE ensures $\langle w_t - w_{t+1}, \widehat{\ell}_t \rangle \leq 2\eta \|\widehat{\ell}_t\|_{H_t^{-1}}^2$ for all t .

Proof. By Hölder's inequality, we first bound $\langle w_t - w_{t+1}, \widehat{\ell}_t \rangle$ by $\|w_t - w_{t+1}\|_{H_t} \|\widehat{\ell}_t\|_{H_t^{-1}}$. Then, note that w_{t+1} is the minimizer of the function $F_t(w) = \eta \langle w, \sum_{s \leq t} \widehat{\ell}_s \rangle + \psi(w)$, which is a self-concordant barrier (the linear terms does not affect the self-concordance coming from ψ , as it will become clear once we see the definition). To apply Fact 2, we calculate the Newton decrement:

$$\lambda_F(w_t) = \|\nabla F(w_t)\|_{\nabla^{-2}F(w_t)} = \left\| \eta \sum_{s \leq t} \widehat{\ell}_s + \nabla \psi(w_t) \right\|_{H_t^{-1}} = \eta \|\widehat{\ell}_t\|_{H_t^{-1}}$$

where the last step uses the first-order condition: $\eta \sum_{s \leq t} \widehat{\ell}_s + \nabla \psi(w_t) = \mathbf{0}$, since w_t minimizes the barrier function $F_{t-1}(w) = \eta \langle w, \sum_{s \leq t} \widehat{\ell}_s \rangle + \psi(w)$. By Lemma 3 and the condition $\eta \leq \frac{1}{2d}$, we know $\lambda_F(w_t) \leq 1/2$ and thus Fact 2 implies $\|w_t - w_{t+1}\|_{H_t} \leq 2\lambda_F(w_t) = 2\eta \|\widehat{\ell}_t\|_{H_t^{-1}}$, which finishes the proof. $\square$

Note that the proof crucially relies on one fact mentioned earlier: the local-norm of the estimator is bounded always, not just in expectation. It remains to deal with the penalty term of FTRL, which is a bit trickier than what we have seen for other regularizers — in the past we have always bounded the penalty term by the range of the regularizer, but now the range of a barrier is by definition $+\infty$! To deal with this issue, we require an additional property from the regularizer, making it a so-called ν -self-concordant barrier for some parameter $\nu > 0$. We again defer the formal definition and first mention the following useful fact.

Fact 3. If ψ is a ν -self-concordant barrier on Ω , then for any $\epsilon > 0$, we have $\psi(u) - \psi(w_1) \leq \nu \ln(\frac{1}{\epsilon} + 1)$ for any u from a shrunk (towards w_1) version of Ω defined as $\{\frac{1}{1+\epsilon}w + \frac{\epsilon}{1+\epsilon}w_1 : w \in \Omega\}$.

Therefore, even though ψ has an infinite range on Ω , its range becomes only $\nu \ln(\frac{1}{\epsilon} + 1)$ if one looks at a slightly shrunk version of it, since a ν -self-concordant barrier changes its value rapidly close to the boundary. Based on all these discussions, we are now ready to prove the following regret bound for SCRiBLE.

Theorem 2. With a ν -self-concordant barrier regularizer and $\eta = \min\left\{\frac{1}{2d}, \sqrt{\frac{\nu \ln T}{Td^2}}\right\}$, SCRiBLE ensures $\mathbb{E}[\mathcal{R}_T] = \mathcal{O}(d\sqrt{\nu T \ln T} + d\nu \ln T)$.

Proof. Based on Lemma 3 of Lecture 2, FTRL ensures for any $u \in \Omega$:

$$\sum_{t=1}^T \langle w_t - u, \widehat{\ell}_t \rangle \leq \frac{\psi(u) - \psi(w_1)}{\eta} + \sum_{t=1}^T \langle w_t - w_{t+1}, \widehat{\ell}_t \rangle.$$

Pick $u = \frac{1}{1+\epsilon}w^* + \frac{\epsilon}{1+\epsilon}w_1$ for $\epsilon = 1/T$. Then in expectation the left-hand side is almost the regret of the learner due to the unbiasedness of the loss estimators:

$$\begin{aligned} \mathbb{E}_t[\langle w_t - u, \widehat{\ell}_t \rangle] &= \langle w_t - u, \ell_t \rangle = \mathbb{E}_t[\langle \widetilde{w}_t - u, \ell_t \rangle] = \mathbb{E}_t[\langle \widetilde{w}_t - w^*, \ell_t \rangle] + \mathbb{E}_t[\langle w^* - u, \ell_t \rangle] \\ &= \mathbb{E}_t[\langle \widetilde{w}_t - w^*, \ell_t \rangle] + \frac{\epsilon}{1+\epsilon} \langle w^* - w_1, \ell_t \rangle \geq \mathbb{E}_t[\langle \widetilde{w}_t - w^*, \ell_t \rangle] - \frac{2}{T}. \end{aligned}$$

For the right-hand side, the penalty term is at most $\frac{\nu \ln(T+1)}{\eta}$ based on Fact 3, and the stability term is at most $2\eta d^2 T$ based on Lemmas 3 and 4. Combining everything shows $\mathbb{E}[\mathcal{R}_T] \leq 2 + \frac{\nu \ln(T+1)}{\eta} + 2\eta d^2 T$, and plugging in the (optimal) value of the learning rate finishes the proof. $\square$

Finally, to get a sense of how good this regret bound is, we point out one last important fact.

Fact 4. For any closed convex set in $\mathbb{R}^d$, there exists a ν -self-concordant barrier with $\nu = \mathcal{O}(d)$.

Therefore, using such an $\mathcal{O}(d)$ -self-concordant barrier, SCRiBLE achieves $\widetilde{\mathcal{O}}(d^{3/2}\sqrt{T})$ regret, slightly worse than the $\widetilde{\mathcal{O}}(d\sqrt{T})$ regret of Exp2. The advantage of SCRiBLE, however, is that it can be implemented efficiently for most problems that we care about, since it only requires solving

a d -dimensional convex problem to find w_t . In fact, [Abernethy et al. \[2008\]](#) even showed that it suffices to do one Damped Newton step at each round to achieve the same regret, that is, instead of computing w_{t+1} as exactly the minimizer of $F_t(w) = \eta \langle w, \sum_{s \leq t} \hat{\ell}_s \rangle + \psi(w)$, we do the following:

$$w_{t+1} = w_t - \frac{1}{1 + \lambda_{F_t}(w_t)} \nabla^{-2} F_t(w_t) \nabla F_t(w_t).$$

Thus, the bottleneck is only in computing the Hessian inverse of F_t (or ψ equivalently).

Definition and Examples of Self-concordant Barriers. For completeness we now give the formal definition of ν -self-concordant barriers. First, consider the one dimensional case ($d = 1$). A function $\psi : \Omega \rightarrow \mathbb{R}$ is self-concordant if it is third-order differentiable, strictly convex, and satisfies the following Lipschitz Hessian condition: $|\psi(w)'''| \leq 2(\psi(w)'')^{3/2}$ for all w in the interior of Ω , and it is ν -self-concordant if in addition it satisfies the Lipschitz condition: $|\psi(w)'| \leq \sqrt{\nu \psi(w)''}$ again for all w in the interior of Ω . For the general d -dimensional case, ψ is ν -self-concordant if restricting it onto any direction gives a ν -self-concordant one-dimensional function.

These conditions say that both the Hessian and the function value move slowly relative to the movement of the gradient. Importantly, unlike common Lipschitz conditions (such as $|\psi(w)'| \leq C$ for some constant $C > 0$), these two conditions are both *affine-invariant*, meaning that if ψ satisfies them, then so does $\psi(Mw + u)$ for any affine transformation defined via M and u . Canonical examples include the following (try to verify them at least for $d = 1$ to convince yourself):

- $\psi(w) = -\sum_{i=1}^d \ln w_i$ (the log-barrier) is a d -self-concordant barrier for $\Omega = \mathbb{R}_+^d$;
- $\psi(w) = -\sum_{j=1}^m \ln(\alpha_j^\top w - \beta_j)$ is an m -self-concordant barrier for the polytope $\Omega = \{w \in \mathbb{R}^d : \alpha_j^\top w \geq \beta_j \text{ for } j = 1, \dots, m\}$;
- $\psi(w) = -\ln(1 - \|w\|_2^2)$ is a 1-self-concordant barrier for the unit ball $\Omega = \{w \in \mathbb{R}^d : \|w\|_2 \leq 1\}$ (note that the self-concordant parameter here is 1 instead of d).

Question 1. How would you use SCRiBLE to solve a combinatorial bandit problem? Think about what regularizer you will use and also what the final decision at each round you will play.

4 One-Point Gradient Estimate for BCO

We now go back to the general BCO problem where the loss function f_t is not necessarily linear. As mentioned, the linearization trick $f_t(w_t) - f_t(w^*) \leq \langle \nabla f_t(w_t), w_t - w^* \rangle$ does not reduce BCO to BLO, but it is still useful since it suggests that we do not need to estimate the entire function f_t based on just one value $f_t(w_t)$, a problem that sounds extremely challenging, but instead only need to estimate one gradient $\nabla f_t(w_t)$ based on $f_t(w_t)$. Such estimate is called a one-point gradient estimate. While this is still highly non-trivial, one can at least estimate the gradient of a smoothed version of f_t based on the following lemma.

Lemma 5. Given a function f and an invertible matrix M , define the smoothed version of f as $\hat{f}(w) = \mathbb{E}_{b \sim \mathbb{B}^d} [f(w + Mb)]$ where b is a uniform sample of the d -dimensional unit ball $\mathbb{B}^d = \{b \in \mathbb{R}^d : \|b\|_2 \leq 1\}$. Then the following holds

$$\nabla \hat{f}(w) = \mathbb{E}_{s \sim \mathbb{S}^d} [df(w + Ms)M^{-1}s] \quad (3)$$

where s is a uniform sample of the d -dimensional unit sphere $\mathbb{S}^d = \{s \in \mathbb{R}^d : \|s\|_2 = 1\}$.

We omit the proof here but one can simply verify this fact when $d = 1$ so that the unit ball is simply the segment $[-1, 1]$ and the unit sphere is simply two points -1 and 1 . Indeed, in this case, with F being the antiderivative of f , we have

$$\begin{aligned} \nabla \mathbb{E}_{b \sim \mathbb{B}^d} [f(w + Mb)] &= \frac{1}{2} \frac{d}{dw} \int_{-1}^1 f(w + Mb) db = \frac{1}{2M} \frac{d}{dw} (F(w + M) - F(w - M)) \\ &= \frac{1}{2M} (f(w + M) - f(w - M)) = \mathbb{E}_{s \sim \mathbb{S}^d} [df(w + Ms)M^{-1}s]. \end{aligned}$$

This lemma directly implies a way to construct the gradient estimator $\hat{\ell}_t$: draw a uniform sample s from the unit sphere, query the value of $f_t(w_t + Ms)$ for some M by playing $\tilde{w}_t = w_t + Ms$, and then use $\hat{\ell}_t = df(w + Ms)M^{-1}s$ as an unbiased estimator of the gradient $\nabla \hat{f}_t(w_t)$ where $\hat{f}_t(w) = \mathbb{E}_{b \sim \mathbb{B}^d}[f_t(w + Mb)]$ is a smoothed version of f_t . Inspired by SCRiBLE, we pick M based on the Hessian of a self-concordant regularizer together with an extra scaling parameter $\delta \in (0, 1]$, leading to the following algorithm proposed by [Saha and Tewari, 2011]. Note that when $\delta = 1$, this exactly recovers SCRiBLE.

Algorithm 3: A Generalization of SCRiBLE for BCO

Input: parameter $\delta \in (0, 1]$, learning rate $\eta > 0$, and a ν -self-concordant function ψ

for $t = 1, \dots, T$ **do**

compute $w_t = \operatorname{argmin}_{w \in \Omega} \langle w, \sum_{s < t} \hat{\ell}_s \rangle + \frac{1}{\eta} \psi(w)$

sample $s_t \in \mathbb{S}^d$ uniformly at random and play $\tilde{w}_t = w_t + \delta H_t^{-1/2} s_t$ where $H_t = \nabla^2 \psi(w_t)$

observe $f_t(\tilde{w}_t)$ and construct gradient estimator $\hat{\ell}_t = \frac{d}{\delta} f_t(\tilde{w}_t) H_t^{\frac{1}{2}} s_t$

Note that $\tilde{w}_t$ is a valid point as it is within the Dikin ellipsoid centered at w_t . Importantly, since $\hat{\ell}_t$ is not exactly an unbiased estimator for f_t itself, this leads to one key issue in this approach: bias-variance trade-off of the estimator, which is controlled by the parameter δ . When δ is close to 0, $\hat{f}_t$ is very close to f_t but $\hat{\ell}_t$ has a very large variance; on the other hand, when δ is large, the variance goes down while $\hat{f}_t$ becomes very different from f_t . Due to this trade-off, this algorithm at best achieves $\tilde{O}(T^{3/4})$ regret for Lipschitz loss functions or $\tilde{O}(T^{2/3})$ regret for smooth loss functions; see [Luo, 2017] for the analysis. The best upper bound for this problem is $\tilde{O}(d^{2.5} \sqrt{T})$ [Lattimore, 2020] (with no concrete algorithms given unfortunately), and the best polynomial-time algorithm achieves regret $\tilde{O}(d^{10.5} \sqrt{T})$ [Bubeck et al., 2017]. Somewhat surprisingly, the best existing lower bound is still $\Omega(d \sqrt{T})$ coming from the linear case [Dani et al., 2008]. Closing this gap with an efficient (and practical) algorithm is still a key open problem in the bandit literature.

References

- Jacob D Abernethy, Elad Hazan, and Alexander Rakhlin. Competing in the dark: An efficient algorithm for bandit linear optimization. In *21st Annual Conference on Learning Theory*, 2008.
- Sébastien Bubeck, Nicolò Cesa-Bianchi, and Sham Kakade. Towards minimax policies for online linear optimization with bandit feedback. In *25th Annual Conference on Learning Theory*, 2012.
- Sébastien Bubeck, Yin Tat Lee, and Ronen Eldan. Kernel-based methods for bandit convex optimization. In *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing*, 2017.
- Nicolò Cesa-Bianchi and Gábor Lugosi. Combinatorial bandits. *Journal of Computer and System Sciences*, 78(5):1404–1422, 2012.
- Varsha Dani, Sham M Kakade, and Thomas P Hayes. The price of bandit information for online optimization. In *Advances in Neural Information Processing Systems 21*, 2008.
- Tor Lattimore. Improved regret for zeroth-order adversarial bandit convex optimisation. *Mathematical Statistics and Learning*, 2(3):311–334, 2020.
- Haipeng Luo. Lecture notes 18, introduction to online learning, 2017. URL <https://haipeng-luo.net/courses/CSCI699/lecture18.pdf>.
- Yurii Nesterov and Arkadii Nemirovskii. *Interior-point polynomial algorithms in convex programming*. SIAM, 1994.
- Ankan Saha and Ambuj Tewari. Improved regret guarantees for online smooth convex optimization with bandit feedback. In *The 14th International Conference on Artificial Intelligence and Statistics*, 2011.

CSCI 659 Lecture 10

Fall 2022

Instructor: Haipeng Luo

1 Online Reinforcement Learning

In this lecture, we discuss how online learning has been extended to and combined with another important and popular area of machine learning: reinforcement learning (RL). RL can be seen as a generalization of MAB by incorporating the concept of *states* and the transition among them driven by the learner's actions. The reward/loss of the learner depends on not only her chosen action, but also the state that she is currently in. This allows RL to capture many more exciting applications (such as playing games or controlling robots), but also leads to one key challenge compared to other problems discussed so far: long-term planning, that is, a smart learner should not focus too much on the short-term reward generated by the current action, but should instead plan ahead in order to reach a region of states with high rewards in the future.

Formally, RL is often captured by a *Markov Decision Process* (MDP), which is a Markov chain driven by a learner's decisions. There are many variants of MDPs. Here, we focus on a variant called finite-horizon tabular MDPs, defined via a tuple (X, A, P, ℓ) :

1. X is a finite state space that can be partitioned into H layers $X_1, \dots, X_H$ for some fixed parameter H , where X_1 contains only an initial state x_{init} .
2. A is a finite action space containing the available actions for the learner at each state.
3. P is a transition function so that $P(\cdot|x, a)$ (for $x \notin X_H$) is the distribution of the next state after taking action a at state x (note the Markovian assumption here: the next state only depends on the current state and action, but not the past). It is assumed that $P(x'|x, a)$ is not zero only when $x \in X_h$ and $x' \in X_{h+1}$ for some $h < H$, that is, transition can only happen between consecutive layers.
4. Finally, $\ell : X \times A \rightarrow [0, 1]$ is a loss function with $\ell(x, a)$ specifying the loss of taking action a at state x .

A policy π is a mapping that maps each state in X to a distribution $\pi(\cdot|x)$ over the actions in A . Starting from the initial state x_{init} , if the learner acts according to π (that is, take an action sampled from $\pi(\cdot|x)$ when at state x), then an *episode* $x_1, a_1, \dots, x_H, a_H$ of H steps is generated with $x_1 = x_{\text{init}}$, $a_h \sim \pi(\cdot|x_h)$, and $x_{h+1} \sim P(\cdot|x_h, a_h)$. The expected total loss of the learner suffered in this episode is denoted by $V^\pi(x_{\text{init}}; \ell) = \mathbb{E}[\sum_{h=1}^H \ell(x_h, a_h)]$. More generally, we denote the expected loss starting from any state x and following π afterwards as $V^\pi(x; \ell)$, which can be written $\mathbb{E}_{a \sim \pi(\cdot|x)}[Q^\pi(x, a; \ell)]$ where

$$Q^\pi(x, a; \ell) = \ell(x, a) + \mathbf{1}\{x \notin X_H\} \mathbb{E}_{x' \sim P(\cdot|x, a)}[V^\pi(x'; \ell)] \quad (1)$$

is the so-called Q -function (specifying the expected loss starting from x , taking action a , and following π afterwards) and the above is known as the *Bellman equation*. While this definition looks recursive, it is really not since the transition is always from one layer to the next one, making $V^\pi(x; \ell)$ well-defined in a backward manner. Traditional RL concerns about finding the (approximately) optimal policy that minimizes $V^\pi(x_{\text{init}}; \ell)$ when P (and sometimes ℓ as well) is unknown, using as few episodes as possible.

We emphasize that assuming a layer structure is really without loss of generality since such a structure can always be created artificially by considering an expanded state space $X \times [H]$ where each state is duplicated H times, each with a different step index. The real restriction of a finite-horizon MDP is that it only considers interaction of a fixed (H) number of steps, after which the learner is reset to the initial state, making any “wrong” actions always recoverable in a sense. There are many other variants of MDP that address this limitation, such as the stochastic shortest path model where the interaction stops only when a certain goal state is reached, or the infinite-horizon model where the interaction never stops and one cares about either the average loss of the learner or her discounted total loss.

Question 1. *Assuming a fixed initial state is also in a sense without loss of generality, and the same model can deal with the case where the initial state is drawn from a fixed distribution. Why?*

Online RL. In online RL, a topic that receives increasing interest in more recent RL studies, the learner deals with a potentially changing MDP with the goal of minimizing regret. Specifically, consider a finite-horizon MDP with a fixed state space X , a fixed action space A , and a fixed transition function P . The learner interacts with the MDP through T episodes, where at each episode $t = 1, \dots, T$:

1. the learner decides a policy π_t while an oblivious adversary decides a loss function $\ell_t : X \times A \rightarrow [0, 1]$;
2. the learner starts from the initial state x_{init} , acts according to π_t , and ends at the terminate state x_H after H steps, generating and observing a trajectory $\{(x_{t,h}, a_{t,h}, \ell_t(x_{t,h}, a_{t,h}))\}_{h=1}^H$.

Let $V_t^\pi(x)$ be a shorthand for $V_t^\pi(x; \ell_t)$. The goal of the learner is to minimize her expected regret, defined as

$$\mathbb{E}[\mathcal{R}_T] = \mathbb{E} \left[\sum_{t=1}^T V_t^{\pi_t}(x_{\text{init}}) \right] - \sum_{t=1}^T V_t^{\pi^*}(x_{\text{init}})$$

where $\pi^* \in \operatorname{argmin}_\pi \sum_{t=1}^T V_t^\pi(x_{\text{init}})$ is the overall optimal policy in hindsight. More concretely, we would like to design an efficient algorithm with (sublinear in T) regret and time/space complexity that are both polynomial in $|X|$, $|A|$, and H .

Note that the feedback on the loss function is bandit-type, that is, the learner does not observe the entire ℓ_t at the end of episode t , but only its value for those visited state-action pairs. In fact, it is clear that adversarial MAB is exactly a special case of this setup with $H = 1$.

One might wonder why we only allow the loss function to be changing over time, but not the transition P as well. It turns out that, if we allow P to also be adversarially chosen, then in the worst case the regret can be as bad as $\Omega(2^H \sqrt{T})$; see [Tian et al., 2021, Lemma 1]. Thus, we assume a fixed P throughout. In fact, for simplicity, we will also by default assume that P is known to the learner in the following discussions and only briefly mention what modifications are needed to handle unknown P . Note that unlike the traditional RL setup where the MDP is fixed and thus knowing P significantly simplifies the problem, online RL is still highly non-trivial even with known P due to the changing loss functions.

2 FTRL over the Occupancy Measure Space

It is not difficult to see that $V_t^\pi(x_{\text{init}})$ is in fact nonconvex in π (try to convince yourself), making it difficult to directly perform FTRL over the policy space. It turns out that, however, it is possible to view this as a linear problem over a different space. Specifically, any policy π induces a corresponding *occupancy measure* $q^\pi : X \times A \rightarrow [0, 1]$ such that $q^\pi(x, a)$ is the probability of visiting state-action pair (x, a) when the learner starts from the initial state and acts according to π . By seeing q^π and a loss function ℓ as $|X||A|$ -dimensional vectors, we can then write $V^\pi(x_{\text{init}}; \ell)$ as $\langle q^\pi, \ell \rangle$ and thus the expected regret as

$$\mathbb{E}[\mathcal{R}_T] = \mathbb{E} \left[\sum_{t=1}^T \langle q_t - q^*, \ell_t \rangle \right]$$

where $q_t = q^{\pi_t}$ and $q^* = q^{\pi^*}$. Importantly, this translates the problem into an online linear optimization problem!

This naturally suggests performing FTRL over the space of occupancy measures to directly come up with q_t . Specifically, let $\Omega = \{q^\pi : \pi \text{ is a policy}\}$ be the space of all valid occupancy measures. While it might not be clear at first glance, this set is in fact a relatively simple polytope with $\mathcal{O}(|X|)$ linear constraints, as shown by the following lemma.

Lemma 1. *The set of valid occupancy measures Ω can be equivalently written as*

$$\Omega = \left\{ q \in [0, 1]^{|X| \times |A|} : q(x_{\text{init}}) = 1 \text{ and } \sum_{x \in X_h} \sum_{a \in A} q(x, a) P(x' | x, a) = q(x'), \forall x' \in X_{h+1} \text{ and } h \in [H-1], \right\}$$

where $q(x)$ is a shorthand for $\sum_{a \in A} q(x, a)$.

Proof. For any π , the induced occupancy measure q^π must belong to this polytope since the first constraint trivially holds, and the second constraint says that the probability of visiting state x' is the sum of probabilities of visiting a state-action pair (x, a) in the last layer and then transiting to x' , which is also trivially true by definition.

On the other hand, for some q that belongs to this polytope, it corresponds to the occupancy measure induced by the policy π with $\pi(a|x) \propto q(x, a)$ (if $q(x) \neq 0$, this means $\pi(a|x) = q(x, a)/q(x)$; otherwise, $\pi(\cdot|x)$ can be an arbitrary distribution over A). To verify this, we need to argue $q^\pi(x, a) = q(x, a)$ for any (x, a) pair, which can be done via a simple induction on the layer index: when $x = x_{\text{init}}$, this holds since $q^\pi(x_{\text{init}}, a) = \pi(a|x_{\text{init}}) = q(x_{\text{init}}, a)/q(x_{\text{init}}) = q(x_{\text{init}}, a)$ using the first constraint $q(x_{\text{init}}) = 1$; now assuming $q^\pi(x, a) = q(x, a)$ for any $(x, a) \in X_h \times A$, then for any $(x', a') \in X_{h+1} \times A$, we have

$$\begin{aligned} q^\pi(x', a') &= \left(\sum_{x \in X_h} \sum_{a \in A} q^\pi(x, a) P(x' | x, a) \right) \pi(a' | x') && \text{(definition of occupancy measure)} \\ &= \left(\sum_{x \in X_h} \sum_{a \in A} q(x, a) P(x' | x, a) \right) \pi(a' | x') && \text{(inductive assumption)} \\ &= q(x') \pi(a' | x') && \text{(second constraint of the polytope)} \\ &= q(x', a'), && \text{(definition of } \pi) \end{aligned}$$

which finishes the proof. $\square$

Note that the proof also tells us how to extract a policy π from its induced occupancy measure q^π by simply normalizing $q^\pi(x, \cdot)$ at each state x . Thus, after obtaining q_t from FTRL, we can extract the policy π_t in this way. It remains to construct loss estimators for ℓ_t since we only observe its value for H state-action pairs. Due to the similarity with MAB, it is natural to again use the inverse importance weighting idea to construct $\hat{\ell}_t$ as

$$\hat{\ell}_t(x, a) = \frac{\ell_t(x, a)}{q_t(x, a)} \mathbf{1}_t(x, a) \quad (2)$$

where $\mathbf{1}_t(x, a)$ is 1 if (x, a) is visited during episode t and 0 otherwise (so there are at most H non-zero entries in $\hat{\ell}_t$, one for each layer). The following properties are direct generalization of those from MAB.

Lemma 2. *The estimator defined in Eq. (2) satisfies $\mathbb{E}_t[\hat{\ell}_t(x, a)] = \ell_t(x, a)$ (unbiasedness) and $\mathbb{E}_t[\hat{\ell}_t(x, a)^2] = \frac{\ell_t(x, a)^2}{q_t(x, a)} \leq \frac{1}{q_t(x, a)}$.*

Proof. Both are based on direct calculations using the fact $\mathbb{E}_t[\mathbf{1}_t(x, a)] = q_t(x, a)$. $\square$

This concludes the design of the algorithm, formally shown below with a general regularizer. To come up with a concrete regularizer, we can again take inspiration from MAB. For example, we can use the generalized Shannon entropy regularizer $\psi(q) = \sum_{(x, a) \in X \times A} q(x, a) \ln q(x, a)$. The resulting algorithm was proposed by [Zimin and Neu, 2013] and ensures the following regret bound.

Algorithm 1: FTRL for Finite-Horizon MDPs

Input: learning rate $\eta > 0$ and a regularizer ψ defined on the occupancy measure space Ω
for $t = 1, \dots, T$ **do**

compute $q_t = \operatorname{argmin}_{q \in \Omega} \left\langle q, \sum_{s < t} \widehat{\ell}_s \right\rangle + \frac{1}{\eta} \psi(q)$
extract policy π_t such that $\pi_t(a|x) \propto q_t(x, a)$
execute policy π_t and observe trajectory $\{(x_{t,h}, a_{t,h}, \ell_t(x_{t,h}, a_{t,h}))\}_{h=1}^H$
construct estimator $\widehat{\ell}_t$ based on Eq. (2)

Theorem 1. Algorithm 1 with regularizer $\psi(q) = \sum_{(x,a) \in X \times A} q(x, a) \ln q(x, a)$ ensures $\mathbb{E}[\mathcal{R}_T] \leq \frac{H \ln(|X||A|)}{\eta} + \eta |X||A|T$, which is $\tilde{\mathcal{O}}(\sqrt{H|X||A|T})$ after picking the optimal η .

Proof. By the exact same analysis as Hedge, one can show (we have seen several approaches by now and you should verify this yourself):

$$\sum_{t=1}^T \left\langle q_t - q^*, \widehat{\ell}_t \right\rangle \leq \frac{B_\psi}{\eta} + \eta \sum_{t=1}^T \sum_{(x,a) \in X \times A} q_t(x, a) \widehat{\ell}_t(x, a)^2$$

where B_ψ is the range of ψ and is at most

$$\begin{aligned} \max_{q \in \Omega} \sum_{(x,a) \in X \times A} q(x, a) \ln \frac{1}{q(x, a)} &= \max_{q \in \Omega} \sum_{h \in [H]} \left(\sum_{(x,a) \in X_h \times A} q(x, a) \ln \frac{1}{q(x, a)} \right) \\ &\leq \sum_{h \in [H]} \ln(|X_h||A|) \leq H \ln(|X||A|) \end{aligned}$$

where the first inequality is by realizing $\sum_{(x,a) \in X_h \times A} q(x, a) = 1$ (based on the definition of occupancy measure) and treating $\sum_{(x,a) \in X_h \times A} q(x, a) \ln \frac{1}{q(x, a)}$ as the Shannon entropy of a distribution over $|X_h||A|$ elements, which is thus at most $\ln(|X_h||A|)$. The rest of the proof is simply by taking expectation and applying Lemma 2. $\square$

Ignoring logarithmic terms, this regret bound is known to be optimal. When $H = 1$ (and thus $|X| = 1$ as well), the algorithm/regret also recovers Exp3. Moreover, the algorithm can be implemented in time/space polynomial in all the parameters since the FTRL optimization is a convex problem defined over a simple polytope with $\mathcal{O}(|X|)$ constraints.

Question 2. What happens if we use a direct generalization of Tsallis entropy, such as $\psi(q) = -2 \sum_{(x,a) \in X \times A} \sqrt{q(x, a)}$, as the regularizer?

Handling Unknown Transition. When P is unknown (the more interesting situation), we need to handle two issues in Algorithm 1: first, the decision set Ω is no longer known ahead of time, and second, the exact loss estimator defined in Eq. (2) is also no longer constructable since q_t , the occupancy measure of π_t , is unknown. To deal with both issues, we need to apply the optimism principle again. Indeed, the challenge coming from unknown transition is very much similar to that in solving stochastic MAB, since both are about adaptively getting samples from fixed unknown distributions. (So interestingly, this problem combines the challenges of both stochastic and adversarial MAB.)

Specifically, at each time t , we build a confidence set $\mathcal{P}_t$ that contains the true transition P with high probability. We omit the concrete form of $\mathcal{P}_t$ but point out that it is based on standard concentration inequalities and in spirit similar to the confidence set of the loss of each arm in stochastic MAB. Let $q^{\pi, \widehat{P}}$ be the occupancy measure induced by policy π for an MDP with transition function $\widehat{P}$ (so the earlier notation q^π is simply a shorthand for $q^{\pi, P}$). Then, we make the following two modifications to Algorithm 1:

- replace the unknown decision set Ω in FTRL with $\Omega_t = \{q^{\pi, \widehat{P}} : \widehat{P} \in \mathcal{P}_t, \pi \text{ is a policy}\}$, the set of all plausible occupancy measures, which incorporates optimism into the FTRL update.

- similarly, when constructing the loss estimator, replace the unknown $q_t(x, a) = q^{\pi_t, P}(x, a)$ with $\max_{\hat{P} \in \mathcal{P}_t} q^{\pi_t, \hat{P}}(x, a)$, the largest plausible probability of visiting (x, a) , thus incorporating optimism into the loss estimators as well.

It turns out that the resulting algorithm is still implementable in polynomial time. By carefully analyzing the error coming from the transition estimation, [Jin et al. \[2020\]](#) showed that this algorithm ensures $\mathcal{O}(H|X|\sqrt{|A|T})$ regret, which only exhibits a small gap compared to the best known lower bound $\Omega(H\sqrt{|X||A|T})$. Closing this gap (with any algorithm, even inefficient ones) is still open.

3 Policy Optimization Methods

Earlier, we mentioned that $V^\pi(x_{\text{init}}; \ell)$ is nonconvex in π , but this in fact does not immediately rule out the possibility of optimizing directly over the policy space. Such approaches are generally called policy optimization methods. Here, we introduce one such method, which in a sense decomposes the online RL problem into $|X|$ different instances of the adversarial MAB problem, one for each state. This is enabled by the following simple yet fundamental *performance difference lemma*.

Lemma 3. *For any loss function ℓ and any two policies π and π^* , the difference of their expected total losses starting from the initial state can be decomposed as*

$$V^\pi(x_{\text{init}}; \ell) - V^{\pi^*}(x_{\text{init}}; \ell) = \sum_{x \in X} q^*(x) \sum_{a \in A} (\pi(a|x) - \pi^*(a|x)) Q^\pi(x, a; \ell)$$

where q^* is a shorthand for q^{π^*} .

Proof. This is by direct calculations (below, $\sum_x$ means $\sum_{x \in X}$ and $\sum_{x,a}$ means $\sum_{x \in X, a \in A}$):

$$\begin{aligned} & V^\pi(x_{\text{init}}; \ell) - V^{\pi^*}(x_{\text{init}}; \ell) \\ &= V^\pi(x_{\text{init}}; \ell) - \sum_{x,a} q^*(x, a) \ell(x, a) \\ &= V^\pi(x_{\text{init}}; \ell) - \sum_{x,a} q^*(x, a) \left(Q^\pi(x, a; \ell) - \mathbf{1}\{x \notin X_H\} \sum_{x'} P(x'|x, a) V^\pi(x'; \ell) \right) \\ & \quad \text{(Bellman equation (1))} \\ &= V^\pi(x_{\text{init}}; \ell) + \sum_{x'} \sum_{x,a} \mathbf{1}\{x \notin X_H\} q^*(x, a) P(x'|x, a) V^\pi(x'; \ell) - \sum_{x,a} q^*(x, a) Q^\pi(x, a; \ell) \\ &= V^\pi(x_{\text{init}}; \ell) + \sum_{x' \neq x_{\text{init}}} q^*(x') V^\pi(x'; \ell) - \sum_{x,a} q^*(x, a) Q^\pi(x, a; \ell) \\ & \quad (\sum_{x,a} \mathbf{1}\{x \notin X_H\} q^*(x, a) P(x'|x, a) = q^*(x') \text{ if } x' \neq x_{\text{init}}) \\ &= \sum_{x'} q^*(x') V^\pi(x'; \ell) - \sum_{x,a} q^*(x, a) Q^\pi(x, a; \ell) \quad (q^*(x_{\text{init}}) = 1) \\ &= \sum_{x,a} q^*(x) \pi(a|x) Q^\pi(x, a; \ell) - \sum_{x,a} q^*(x) \pi^*(a|x) Q^\pi(x, a; \ell), \end{aligned}$$

where the last step changes the name of the variable x' to x and uses the definition $V^\pi(x; \ell) = \sum_a \pi(a|x) Q^\pi(x, a; \ell)$. $\square$

Below is a direct consequence of the performance difference lemma.

Corollary 1. *Denote $Q^{\pi_t}(x, a; \ell_t)$ by $Q_t(x, a)$. Then the expected regret of the learner can be decomposed as*

$$\mathbb{E}[\mathcal{R}_T] = \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \langle \pi_t(\cdot|x) - \pi^*(\cdot|x), Q_t(x, \cdot) \rangle \right].$$

Note that for each x , the term $\sum_{t=1}^T \langle \pi_t(\cdot|x) - \pi^*(\cdot|x), Q_t(x, \cdot) \rangle$ is exactly the regret of a $|A|$ -armed bandit problem, with $Q_t(x, \cdot)$ being the “loss vector” for this MAB instance. Thus, Corollary 1 indicates that the regret of the online RL problem can be written as the weighted average of some MAB regret over all states, where the weight for each state is its (unknown) probability of being visited by π^* .

This suggests the following algorithmic idea: simply run an adversarial MAB algorithm on each state x to decide $\pi_t(\cdot|x)$. We emphasize that it is critical to use an *adversarial* MAB algorithm here, even if the loss functions $\ell_1, \dots, \ell_T$ are stochastic (thus the online RL problem is a stochastic one). This is because $Q_t(x, \cdot)$ is the Q -function with respect to π_t , and π_t itself is changing over time in a potentially complicated way, making the corresponding MAB instance non-stochastic.

One slight difference compared to a standard MAB problem is that after selecting an action a , we do not exactly observe $Q_t(x, a)$. However, we can still naturally construct an estimator $\hat{Q}_t$ for Q_t using some loss estimator $\hat{\ell}_t$ and the transition function P (again assumed to be known for now) via: $\hat{Q}_t(x, a) = Q^{\pi_t}(x, a; \hat{\ell}_t)$. For example, if we again use the unbiased loss estimator of Eq. (2), then clearly $\hat{Q}_t$ is also an unbiased estimator for Q_t .

The issue of this estimator is that its variance is quite different from that of $\hat{\ell}_t$, making it difficult to control the local-norm from the MAB regret. For example, if we use Exp3 to solve the MAB problem at each state, then the local-norm for time t is $\mathbb{E}[\sum_{a \in A} \pi_t(a|x) \hat{Q}_t(x, a)^2]$, which does not enjoy the critical variance cancellation effect any more. In fact, it is even difficult to control the magnitude of $\hat{Q}_t(x, a)$ in this case, since the importance weight $q_t(x, a)$ in Eq. (2) can be arbitrarily small (making $\hat{\ell}_t(x, a)$ arbitrarily large). Also note that unlike some other problems we have seen (such as Problem 3(b)(ii) of HW2), there is no way to modify the algorithm to enforce a lower bound on $q_t(x, a)$, since it might be the case that the transition of the MDP is such that regardless of how the learner behaves, the probability of visiting state x is always tiny.

To fix this issue, we have to sacrifice a little bit of the unbiasedness and add a small value to the importance weight. This leads to the following algorithm, taken from [Shani et al., 2020].

Algorithm 2: Policy Optimization

Input: parameter $\gamma > 0$, learning rate $\eta > 0$, and a regularizer ψ defined over $\Delta(A)$
for $t = 1, \dots, T$ **do**

for each state x , define $\pi_t(\cdot|x) = \operatorname{argmin}_{p \in \Delta(A)} \left\langle p, \sum_{s < t} \hat{Q}_s(\cdot, x) \right\rangle + \frac{1}{\eta} \psi(p)$
 execute policy π_t and observe trajectory $\{(x_{t,h}, a_{t,h}, \ell_t(x_{t,h}, a_{t,h}))\}_{h=1}^H$
 construct Q -function estimator $\hat{Q}_t$ such that $\hat{Q}_t(x, a) = Q^{\pi_t}(x, a; \hat{\ell}_t)$ where

$$\hat{\ell}_t(x, a) = \frac{\ell_t(x, a)}{q_t(x, a) + \gamma} \mathbf{1}_t(x, a) \quad (3)$$

The following lemma summarizes the bias and variance of the estimators.

Lemma 4. *The loss estimator defined in Eq. (3) satisfies $0 \leq \ell_t(x, a) - \mathbb{E}_t[\ell_t(x, a)] \leq \frac{\gamma \ell_t(x, a)}{q_t(x, a)}$ and $V^{\pi_t}(x_{\text{init}}; \ell_t) - \mathbb{E}_t[V^{\pi_t}(x_{\text{init}}; \hat{\ell}_t)] \leq \gamma |X| |A|$. Also, the corresponding Q -function estimator $\hat{Q}_t(x, a) = Q^{\pi_t}(x, a; \hat{\ell}_t)$ satisfies $\mathbb{E}_t[\hat{Q}_t(x, a)^2] \leq \frac{H^2}{\gamma}$.*

Proof. To prove the first statement, note that

$$\ell_t(x, a) - \mathbb{E}_t[\ell_t(x, a)] = \ell_t(x, a) \left(1 - \frac{q_t(x, a)}{q_t(x, a) + \gamma} \right) = \frac{\gamma \ell_t(x, a)}{q_t(x, a) + \gamma},$$

which is clearly nonnegative and at most $\frac{\gamma \ell_t(x, a)}{q_t(x, a)}$. Using this upper bound, we can prove the second statement:

$$V^{\pi_t}(x_{\text{init}}; \ell_t) - \mathbb{E}_t[V^{\pi_t}(x_{\text{init}}; \hat{\ell}_t)] = \left\langle q_t, \ell_t - \mathbb{E}_t[\hat{\ell}_t] \right\rangle \leq \sum_{x, a} q_t(x, a) \frac{\gamma \ell_t(x, a)}{q_t(x, a)} \leq \gamma |X| |A|.$$

To prove the last statement, note that the range of $\hat{\ell}_t(x, a)$ is $[0, 1/\gamma]$, and thus $\hat{Q}_t(x, a)$, being the Q -function with respect to $\hat{\ell}_t$, is in the range $[0, H/\gamma]$. Therefore, $\mathbb{E}_t[\hat{Q}_t(x, a)^2] \leq \frac{H}{\gamma} \mathbb{E}_t[\hat{Q}_t(x, a)] \leq \frac{H}{\gamma} Q_t(x, a) \leq \frac{H^2}{\gamma}$. $\square$

This lemma tells us that the loss estimator is underestimating the true loss, but even if the amount of underestimation can be very large for a specific (x, a) pair, importantly, the overall underestimation of the expected loss of π_t itself is only $\gamma|X||A|$. On the other hand, what this bias buys us is an explicit control on the variance of the Q -function estimator (which is at most H^2/γ). Together, these facts allow us to prove the following regret bound.

Theorem 2. *With the entropy regularizer $\psi(p) = \sum_{a \in A} p(a) \ln p(a)$, Algorithm 2 ensures $\mathbb{E}[\mathcal{R}_T] \leq \gamma|X||A|T + \frac{H \ln |A|}{\eta} + \frac{\eta H^3 T}{\gamma}$, which is $\tilde{O}((T^2 H^4 |X||A|)^{1/3})$ after picking the optimal γ and η .*

Proof. First, we decompose the regret as:

$$\begin{aligned} \mathbb{E}[\mathcal{R}_T] &= \mathbb{E} \left[\sum_{t=1}^T (V_t^{\pi_t}(x_{\text{init}}) - V_t^{\pi^*}(x_{\text{init}})) \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T (V_t^{\pi_t}(x_{\text{init}}; \ell_t) - V_t^{\pi_t}(x_{\text{init}}; \hat{\ell}_t)) + \sum_{t=1}^T (V_t^{\pi_t}(x_{\text{init}}; \hat{\ell}_t) - V_t^{\pi^*}(x_{\text{init}}; \ell_t)) \right] \\ &\leq \gamma|X||A|T + \mathbb{E} \left[\sum_{t=1}^T (V_t^{\pi_t}(x_{\text{init}}; \hat{\ell}_t) - V_t^{\pi^*}(x_{\text{init}}; \hat{\ell}_t)) \right] \end{aligned}$$

where the last step uses Lemma 4. We then apply the performance difference lemma to the second term:

$$\sum_{t=1}^T (V_t^{\pi_t}(x_{\text{init}}; \hat{\ell}_t) - V_t^{\pi^*}(x_{\text{init}}; \hat{\ell}_t)) = \sum_{x \in X} q^*(x) \sum_{t=1}^T \left\langle \pi_t(\cdot|x) - \pi^*(\cdot|x), \hat{Q}_t(x, \cdot) \right\rangle.$$

For each state x , note that $\pi_t(\cdot|x)$ is obtained by Hedge with the Q -function estimators as inputs. Thus, we can apply the regret bound of Hedge to obtain:

$$\sum_{t=1}^T \left\langle \pi_t(\cdot|x) - \pi^*(\cdot|x), \hat{Q}_t(\cdot, x) \right\rangle \leq \frac{\ln |A|}{\eta} + \eta \sum_{t=1}^T \sum_{a \in A} \pi_t(a|x) \hat{Q}_t(x, a)^2,$$

which is at most $\frac{\ln |A|}{\eta} + \frac{\eta H^2 T}{\gamma}$ after taking expectation on both sides and using Lemma 4. Finally, using $\sum_{x \in X} q^*(x) = H$ and combining everything finishes the proof. $\square$

As one can see in this analysis, because we are not able to make use of the critical variance cancellation effect from the local-norm term and instead directly enforce an H^2/γ upper bound on the variance, the final regret bound we obtain is of order $T^{2/3}$ instead of $\sqrt{T}$. However, compared to Algorithm 1, policy optimization methods like Algorithm 2 are even more efficient since it does not require solving a convex problem over the occupancy measure space. In fact, it is not even necessary to actually run an MAB algorithm for every state at every round; instead, it suffices to compute $\pi_t(\cdot|x)$ only when x is actually visited. Due to this property, policy optimization methods can even be generalized to problems with an infinite number of states.

3.1 A Different Q -Function Estimator and Extra Bonuses

Is there a way to enjoy both the nice properties of policy optimization methods and the $\sqrt{T}$ -type regret bound simultaneously? As mentioned, the variance cancellation is critical if we want $\sqrt{T}$ -type regret, and that requires a more careful treatment of the variance of the Q -function estimator. To this end, we study a different (and in a sense simpler) estimator for some parameter $\gamma \geq 0$:

$$\hat{Q}_t(x, a) = \frac{L_{t,h}}{q_t(x, a) + \gamma} \mathbf{1}_t(x, a) \quad \text{where } h \text{ is s.t. } x \in X_h \text{ and } L_{t,h} = \sum_{k=h}^H \ell_t(x_{t,k}, a_{t,k}). \quad (4)$$

Note that $L_{t,h}$ is the total loss suffered (and observed) by the learner starting from step h at round t , which by definition has conditional expectation $Q_t(x_{t,h}, a_{t,h})$ (given everything up to step h). Therefore, when $\gamma = 0$, $\hat{Q}_t(x, a)$ is indeed an unbiased estimator for $Q_t(x, a)$. The advantage of this estimator is that its variance is in a simple form similar to its bias, as summarized below.

Lemma 5. *The Q -function estimator defined in Eq. (4) satisfies $0 \leq Q_t(x, a) - \mathbb{E}_t[\hat{Q}_t(x, a)] \leq \frac{\gamma H}{q_t(x, a) + \gamma}$ and $\mathbb{E}_t[\hat{Q}_t(x, a)^2] \leq \frac{H^2}{q_t(x, a) + \gamma}$.*

Proof. To analyze the bias, note that

$$\mathbb{E}_t[\hat{Q}_t(x, a)] = \mathbb{E}_t[\mathbf{1}_t(x, a)] \mathbb{E}_t \left[\frac{L_{t,h}}{q_t(x, a) + \gamma} \middle| \mathbf{1}_t(x, a) = 1 \right] = q_t(x, a) \frac{Q_t(x, a)}{q_t(x, a) + \gamma},$$

and thus $0 \leq Q_t(x, a) - \mathbb{E}_t[\hat{Q}_t(x, a)] = \frac{\gamma Q_t(x, a)}{q_t(x, a) + \gamma} \leq \frac{\gamma H}{q_t(x, a) + \gamma}$. Similarly, the variance is

$$\mathbb{E}_t[\hat{Q}_t(x, a)^2] = \mathbb{E}_t[\mathbf{1}_t(x, a)] \mathbb{E}_t \left[\frac{L_{t,h}^2}{(q_t(x, a) + \gamma)^2} \middle| \mathbf{1}_t(x, a) = 1 \right] \leq \frac{q_t(x, a) H^2}{(q_t(x, a) + \gamma)^2} \leq \frac{H^2}{q_t(x, a) + \gamma},$$

proving the second statement. $\square$

If we run Hedge at each state with this Q -function estimator as inputs, then based on Corollary 1, Lemma 4, and the Hedge regret bound, we have

$$\begin{aligned} \mathbb{E}[\mathcal{R}_T] &= \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \langle \pi_t(\cdot|x) - \pi^*(\cdot|x), Q_t(x, \cdot) \rangle \right] \\ &\leq \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \left(\langle \pi_t(\cdot|x) - \pi^*(\cdot|x), \hat{Q}_t(x, \cdot) \rangle + \langle \pi_t(\cdot|x), Q_t(x, \cdot) - \hat{Q}_t(x, \cdot) \rangle \right) \right] \\ &\leq \frac{H \ln |A|}{\eta} + \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \sum_{a \in A} \frac{\eta H^2 \pi_t(a|x)}{q_t(x, a) + \gamma} \right] + \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \sum_{a \in A} \frac{\gamma H \pi_t(a|x)}{q_t(x, a) + \gamma} \right]. \quad (5) \end{aligned}$$

By taking $\gamma = \eta H$ and defining a “bonus” function $b_t(x) = \sum_{a \in A} \frac{2\gamma H \pi_t(a|x)}{q_t(x, a) + \gamma}$ for each state, the regret bound above can be simplified as $\frac{H \ln |A|}{\eta} + \sum_{t=1}^T \mathbb{E}[V^{\pi^*}(x_{\text{init}}; b_t)]$ (by seeing b_t as function on $X \times A$ so that $b_t(x, a) = b_t(x)$ for all a). While $V^{\pi^*}(x_{\text{init}}; b_t)$ itself is still not well bounded due to the mismatch between π^* and π_t (in the definition of b_t), note that $V^{\pi_t}(x_{\text{init}}; b_t)$ is on the other hand nicely bounded: $V^{\pi_t}(x_{\text{init}}; b_t) = \sum_{x \in X} q_t(x) b_t(x) = \sum_{x \in X} \sum_{a \in A} \frac{2\gamma H q_t(x, a)}{q_t(x, a) + \gamma} \leq 2\gamma H |X| |A|$. So is there a way to somehow move from $V^{\pi^*}(x_{\text{init}}; b_t)$ to $V^{\pi_t}(x_{\text{init}}; b_t)$?

In fact, we have seen the opposite of such techniques in Section 2 of Lecture 3 where by adding an extra penalty term to the losses fed to Hedge, we were able to turn a bound $\eta \sum_i p_t(i) (\ell_t(i) - m_t(i))^2$ (in terms of the learner’s strategy p_t) to a bound $\eta (\ell_t(i^*) - m_t(i^*))^2$ (in terms of the optimal action i^*). This suggests that if we want the opposite effect, we need to subtract a bonus term from the loss ℓ_t , and that bonus term is exactly b_t . This intuitively encourages more exploration to less frequently visited states (since b_t is large when $q_t(x)$ is small), and algorithmically amounts to subtracting a bonus $B_t(x, a) = Q^\pi(x, a; b_t)$ from the Q -function estimator fed to Hedge (or any other MAB algorithm). We summarize the algorithm and its regret guarantee below.

Theorem 3. *With the entropy regularizer $\psi(p) = \sum_{a \in A} p(a) \ln p(a)$, Algorithm 3 ensures $\mathbb{E}[\mathcal{R}_T] \leq \frac{H \ln |A|}{\eta} + 2\eta H^2 |X| |A| T + 4\eta H^5 T$, which is $\tilde{O}(\sqrt{H^3 |X| |A| T} + H^3 \sqrt{T})$ using the optimal η .*

Proof. First, decompose the regret in the following way

$$\begin{aligned} \mathbb{E}[\mathcal{R}_T] &= \mathbb{E} \left[\sum_{t=1}^T (V^{\pi_t}(x_{\text{init}}; \ell_t) - V_t^{\pi^*}(x_{\text{init}}; \ell_t)) \right] \\ &= \mathbb{E} \left[\sum_{t=1}^T (V^{\pi_t}(x_{\text{init}}; \ell_t - b_t) - V_t^{\pi^*}(x_{\text{init}}; \ell_t - b_t)) \right] + \mathbb{E} \left[\sum_{t=1}^T (V^{\pi_t}(x_{\text{init}}; b_t) - V_t^{\pi^*}(x_{\text{init}}; b_t)) \right] \end{aligned}$$

Algorithm 3: Policy Optimization with Bonuses

Input: parameter $\gamma = \eta H$, learning rate $\eta \leq \frac{1}{2H^2}$, and a regularizer ψ defined over $\Delta(A)$

for $t = 1, \dots, T$ **do**

 for each state x , define $\pi_t(\cdot|x) = \operatorname{argmin}_{p \in \Delta(A)} \left\langle p, \sum_{s < t} (\widehat{Q}_s(x, \cdot) - B_s(x, a)) \right\rangle + \frac{1}{\eta} \psi(p)$
 execute policy π_t and observe trajectory $\{(x_{t,h}, a_{t,h}, \ell_t(x_{t,h}, a_{t,h}))\}_{h=1}^H$
 construct Q -function estimator $\widehat{Q}_t$ as in Eq. (4) and a bonus Q -function B_t with
 $B_t(x, a) = Q^{\pi_t}(x, a; b_t)$ and $b_t(x, a') = \sum_{a \in A} \frac{2\gamma H \pi_t(a|x)}{q_t(x, a) + \gamma}$ for all $a' \in A$

$$\leq \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \langle \pi_t(\cdot|x) - \pi^*(\cdot|x), Q_t(x, \cdot) - B_t(x, \cdot) \rangle \right] + 2\gamma H |X| |A| T - \mathbb{E} \left[\sum_{t=1}^T V_t^{\pi^*}(x_{\text{init}}; b_t) \right]$$

where the last step uses Corollary 1 and the earlier calculation $V_t^{\pi_t}(x_{\text{init}}; b_t) \leq 2\gamma H |X| |A|$. Importantly, the last negative term helps us cancel the large bias and variance coming from the first term. Indeed, we can further upper bound the first term by (since $\widehat{Q}_t$ is an underestimator)

$$\mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \left(\langle \pi_t(\cdot|x) - \pi^*(\cdot|x), \widehat{Q}_t(x, \cdot) - B_t(x, \cdot) \rangle + \langle \pi_t(\cdot|x), Q_t(x, \cdot) - \widehat{Q}_t(x, \cdot) \rangle \right) \right]. \quad (6)$$

Since $b_t(x, a) \leq 2H$ and consequently $B_t(x, a) \leq 2H^2$, under the condition $\eta \leq \frac{1}{2H^2}$, we have $\eta(\widehat{Q}_t(x, a) - B_t(x, a)) \geq -1$, satisfying the condition for Hedge's local-norm regret bound. Applying it, we see that compared to Eq. (5), which is $\frac{H \ln |A|}{\eta} + \sum_{t=1}^T \mathbb{E}[V_t^{\pi^*}(x_{\text{init}}; b_t)]$, the only extra term in Eq. (6), coming from the local-norm term of Hedge, is

$$\eta \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \sum_{a \in A} \pi_t(a|x) B_t(x, a)^2 \right] \leq 4\eta H^4 \mathbb{E} \left[\sum_{x \in X} q^*(x) \sum_{t=1}^T \sum_{a \in A} \pi_t(a|x) \right] = 4\eta H^5 T.$$

Combining everything (and importantly canceling $V_t^{\pi^*}(x_{\text{init}}; b_t)$ with $-V_t^{\pi^*}(x_{\text{init}}; b_t)$) shows $\mathbb{E}[\mathcal{R}_T] \leq \frac{H \ln |A|}{\eta} + 2\gamma H |X| |A| T + 4\eta H^5 T$. Plugging in $\gamma = \eta H$ finishes the proof. $\square$

Therefore, Algorithm 3 not only enjoys the nice properties of policy optimization methods, but also achieves a regret bound that is only worse by some H factors compared to the optimal bound $\mathcal{O}(\sqrt{H|X||A|T})$ (achieved by Algorithm 1). In fact, the term $\tilde{\mathcal{O}}(H^3\sqrt{T})$ can be further improved to $\tilde{\mathcal{O}}(H^4)$ by slightly enlarging the bonus function. For this improvement, as well as how to deal with unknown transition (which uses similar ideas discussed earlier) and how to generalize the algorithm to MDPs with an infinite number states and a certain linear structure, see [Luo et al., 2021].

Question 3. Once again, in both Algorithms 2 and 3, what happens if we use Tsallis entropy, such as $\psi(p) = -2 \sum_{a \in A} \sqrt{p(a)}$, as the regularizer?

References

- Chi Jin, Tiancheng Jin, Haipeng Luo, Suvrit Sra, and Tiancheng Yu. Learning adversarial markov decision processes with bandit feedback and unknown transition. In *International Conference on Machine Learning*, pages 4860–4869. PMLR, 2020.
- Haipeng Luo, Chen-Yu Wei, and Chung-Wei Lee. Policy optimization in adversarial mdps: Improved exploration via dilated bonuses. *Advances in Neural Information Processing Systems*, 34: 22931–22942, 2021.
- Lior Shani, Yonathan Efroni, Aviv Rosenberg, and Shie Mannor. Optimistic policy optimization with bandit feedback. In *International Conference on Machine Learning*, pages 8604–8613. PMLR, 2020.

- Yi Tian, Yuanhao Wang, Tiancheng Yu, and Suvrit Sra. Online learning in unknown markov games. In *International conference on machine learning*, pages 10279–10288. PMLR, 2021.
- Alexander Zimin and Gergely Neu. Online learning in episodic markovian decision processes by relative entropy policy search. *Advances in neural information processing systems*, 26, 2013.